



南京大學

本科畢業設計

院 系 软件学院

专 业 软件工程

题 目 MAGIC4FDG——一种基于多

智能体协作与覆盖率反馈迭代

的模糊驱动自动生成系统

年 级 2021 学 号 211250071

学生姓名 柏帅

指导教师 房春荣 职 称 副教授

提交日期 2026 年 5 月 31 日

南京大学本科生毕业论文（设计、作品）中文摘要

题目：MAGIC4FDG——一种基于多智能体协作与覆盖率反馈迭代的模糊驱动自动生成系统

院系：软件学院

专业：软件工程

本科生姓名：柏帅

指导教师（姓名、职称）：房春荣 副教授

摘要：

在软件安全领域，模糊测试已经被证明是挖掘漏洞最高效的手段之一，但它的实际效果很大程度上受限于模糊驱动程序的质量。调研发现，OSS-Fuzz 等主流平台上的驱动几乎全靠人工编写，这不仅消耗大量专家时间，而且最终覆盖率往往不高。大语言模型的代码生成能力为自动化这一过程创造了新的可能性，不过现有方法普遍面临几个问题：覆盖率反馈粒度不足，只知道哪些 API 被调用了却不知道哪行代码没跑到；生成策略单一，缺乏多样性；没有建立起跨轮次的知识积累，迭代优化能力不足。更关键的是，让一个模型同时负责规划、生成、修复和分析，注意力容易被稀释，深层代码路径难以触及。

针对这些问题，本文提出了 MAGIC4FDG (Multi-Agent Guided Iteration with Coverage for Fuzz Driver Generation) 系统。设计的核心思路是将任务拆解而非让单一模型承担所有职责。系统把驱动生成分成七个子任务，分别交给专职智能体：知识提取、策略规划、代码生成、编译修复、覆盖率分析、状态持久化和缺口诊断。七个智能体组成有向无环图，每个只获取自己需要的信息，避免无关上下文的干扰和注意力稀释。生成流程分两个阶段：第一轮是探索，策略规划智能体分析 API 语义关系，从十种预定义策略中为每个驱动槽位选择最合适的方案，尽量铺开覆盖面；第二轮起进行迭代优化，缺口诊断智能体利用 llvm-cov 的行级覆盖率数据定位未覆盖代码及其原因，将结论整理为结构化约束，代码生成智能体据此做定向修改。每个槽位维护独立的知识空间，轮次之间约束条件和代码模式持续累积。所有执行环节（知识提取、编译、模糊测试、覆盖率收集）均在 Docker 容器内完成。

本文在 12 个 OSS-Fuzz 收录的开源 C/C++ 库上进行了实验，与 PromptFuzz、

Hopper 和 OSS-Fuzz-Gen 等工具以及人工驱动进行对比。结果表明 MAGIC4FDG 生成的驱动在覆盖率上明显优于单轮生成基线，消融实验也验证了多智能体架构、策略库匹配和迭代反馈机制各自的贡献。在多个目标库上，系统的表现已接近甚至超过了现有最优工具和人工编写的驱动。长时模糊测试进一步验证了驱动的实用价值：在 4 个库的 3 小时测试中触发 9 个崩溃并关联 7 个已知 CVE，还发现了一个潜在新缺陷。

关键词： 软件安全；模糊测试；模糊驱动生成；大语言模型；多智能体系统；迭代优化

南京大学本科生毕业论文（设计、作品）英文摘要

THESIS: MAGIC4FDG——An Automated Fuzz Driver Generation System Based on Multi-Agent Collaboration and Coverage-Feedback Iteration

DEPARTMENT: Software Institute

SPECIALIZATION: Software Engineering

UNDERGRADUATE: Shuai Bai

MENTOR: Associate Professor Chun Rong Fang

ABSTRACT:

Fuzz testing is among the most effective methods to find security problems in software, but how useful it is in real work is mostly limited by how good the fuzz drivers are. On common platforms like OSS-Fuzz, people write almost all drivers by hand. This takes a lot of work from experienced experts, and many times it only gets limited code coverage in the end. Big language models give new chances to make this whole process automatic, but the current methods we have now have a few issues. The coverage feedback they use is too rough, they use single big monolithic models that easily get diluted attention, they do not keep and reuse knowledge between different iterations, and they use the same generation method every time that does not account for the specific traits of individual libraries.

This thesis introduces MAGIC4FDG (Multi-Agent Guided Iteration with Coverage for Fuzz Driver Generation), which is a system that breaks down fuzz driver generation into seven specific sub-tasks. Each of these sub-tasks is handled by its own dedicated agent: knowledge extraction, strategy planning, code generation, compilation repair, coverage analysis, state persistence, and gap diagnosis. These agents make up a directed acyclic graph, and each one only gets context that matters for its specific task, this is to stop too much information from causing overload. The whole generation process works in two different phases. The first is the exploration phase, where the planner agent goes through API semantics and picks strategies from a pre-made library that holds ten different approaches. After this comes the exploitation phases, where the analyst agent uses line-level coverage data from llvm-cov to figure out which regions of

code are not covered, and then makes structured constraints to help improve the work in a targeted way. Each driver slot keeps its own separate knowledge space, where constraints and code patterns build up little by little across every iteration. Every single step of execution, including things like compilation, fuzzing, and collecting coverage data, all runs inside Docker containers.

When we ran experiments on 12 different C/C++ libraries from OSS-Fuzz, we found that MAGIC4FDG gets an average line coverage of 55.01%. This number is higher than what PromptFuzz (53.25%), Hopper (50.19%), and the single-round baseline methods can reach. We did extra ablation studies to check, and these studies confirm that the multi-agent architecture, the strategy matching step, and the iterative feedback mechanism all make their own separate contributions to the performance. When we ran longer 3-hour fuzzing tests on 4 representative libraries, the tests found 9 crashes that are connected to 7 already known CVEs. They also found one possible new defect that had not been spotted before. This shows that the generated drivers do have real practical value for security testing work.

KEYWORDS: Software Security; Fuzz testing; Fuzz driver generation; Large language models; Multi-agent systems; Iterative optimization

目 录

目 录	V
第一章 引言	1
1.1 研究背景与意义	1
1.2 国内外研究现状	2
1.2.1 基于程序分析的模糊驱动生成	2
1.2.2 基于大模型的模糊驱动生成	3
1.2.3 现有方法的不足与本文切入点	4
1.3 研究目标	5
1.4 研究内容	5
1.4.1 主要工作	5
1.4.2 论文结构	7
第二章 相关技术	9
2.1 模糊测试与模糊驱动	9
2.2 LibFuzzer 模糊引擎与覆盖率引导技术	10
2.3 大语言模型与代码生成技术	11
2.4 多智能体协作框架	13
2.5 静态程序分析与 clang AST	14
2.6 本章小结	15
第三章 覆盖率引导的多智能体模糊驱动生成系统 MAGIC4FDG	
的需求分析与概要设计	17
3.1 系统整体概述	17
3.2 需求分析	20

3.2.1	涉众分析	21
3.2.2	功能性需求分析	22
3.2.3	非功能性需求分析	24
3.2.4	系统用例图	25
3.2.5	系统用例描述	26
3.3	系统概要设计	32
3.3.1	系统整体架构设计	32
3.3.2	系统模块划分	33
3.3.3	系统逻辑视图	35
3.3.4	系统开发视图	37
3.3.5	系统进程视图	38
3.3.6	系统物理视图	40
3.4	本章小结	41
第四章 MAGIC4FDG 系统的详细设计与实现		43
4.1	知识提取模块详细设计与实现	43
4.1.1	知识提取模块详细设计	43
4.1.2	AST 解析与知识构建功能实现	46
4.1.3	差异化上下文构建功能实现	50
4.2	策略规划与代码生成模块详细设计与实现	53
4.2.1	策略规划与代码生成模块详细设计	54
4.2.2	探索阶段策略规划功能实现	56
4.2.3	多提示词策略驱动生成功能实现	58
4.3	模糊测试与覆盖率收集模块详细设计与实现	61
4.3.1	模糊测试与覆盖率收集模块详细设计	62
4.3.2	Fork 模式模糊测试与语料库重放功能实现	63
4.3.3	控制流图可达性分析功能实现	65
4.4	缺口诊断模块详细设计与实现	67
4.4.1	缺口诊断模块详细设计	67
4.4.2	覆盖率缺口诊断功能实现	69

4.4.3	跨轮次知识累积功能实现	71
4.5	本章小结	72
第五章 实验设计与结果分析		73
5.1	实验设置	73
5.1.1	实验环境	73
5.1.2	目标库选取	73
5.1.3	对比方法	74
5.1.4	评估指标	75
5.1.5	参数设置	75
5.2	主实验结果与分析	76
5.2.1	覆盖率总体结果	76
5.2.2	迭代轮次效果分析	78
5.2.3	策略有效性分析	80
5.2.4	Token 消耗分析	81
5.3	消融实验	82
5.3.1	消融实验设计	82
5.3.2	消融实验结果与分析	82
5.4	长时模糊测试与缺陷发现	84
5.4.1	长时覆盖率提升	84
5.4.2	缺陷发现	85
5.5	本章小结	86
第六章 总结与展望		87
6.1	研究工作总结	87
6.2	局限性分析	88
6.3	未来研究展望	89
参考文献		91
致 谢		95

第一章 引言

1.1 研究背景与意义

软件中潜藏的安全漏洞一旦被利用，后果往往远超开发者的预期。2017 年 WannaCry 勒索软件在短短几天内瘫痪了全球数十万台主机，四年后 Log4j 漏洞又让几乎整个 Java 生态陷入紧急修补，两起事件的经济损失合计达数十亿美元^[1]。这类案例反复说明一个现实：漏洞发现得越晚，修复代价越高。

模糊测试 (Fuzzing) 是目前应对这一问题最直接的手段之一。其原理并不复杂，就是向目标程序持续注入随机或经过变异的输入，看程序是否出现崩溃或异常行为^[2]。这种简单而高效的方法在工业界已有充分验证：Google 的 OSS-Fuzz 平台运行至今，累计为超过 1000 个开源项目捕获并修复了上万个安全漏洞^[3]。

但模糊测试的效果很大程度上取决于模糊驱动的质量。模糊驱动是一段适配程序，负责把模糊引擎产生的随机字节流转化为被测库能接受的 API 调用^[4]。写好一个驱动并不容易，开发者需要深入理解目标库的接口语义和调用约束，这往往要花数小时甚至数天^[5]。而且 OSS-Fuzz 上的驱动几乎全靠人工编写，最终平均覆盖率也只有约 30%^[3]。所以本文认为，如何自动生成高质量的模糊驱动，是提升模糊测试规模和深度的关键瓶颈。

近两年大语言模型的进步让人看到了另一种可能性。LLM 在海量代码语料上预训练后，对代码补全、生成和修复等任务表现出了相当强的泛化能力^[6]。值得注意的是，Google 开源安全团队已经在实际项目中验证了这条路径，他们用 LLM 生成的模糊驱动不仅提升了覆盖率，还在 OpenSSL 中首次发现了此前未知的安全漏洞^[7]。换句话说，LLM 辅助模糊驱动生成已经不只是学术探索，它正在产生真实的安全收益。

不过，现有的 LLM 驱动生成方法仍有明显的短板。本文观察到四个共性问题：覆盖率反馈的粒度太粗（往往只到函数或 API 组合级别），单模型承担了从规划到生成到修复的全部职责导致注意力分散，轮次之间缺乏系统性的知识传

递，以及生成策略对不同类型的库缺乏区分。这些问题共同制约了驱动质量的进一步提升。本文的思路是将多智能体协作架构与细粒度覆盖率反馈结合，让不同智能体各司其职，用行级覆盖率数据驱动精准迭代，同时在槽位隔离的知识空间中累积经验。这一方案旨在提升模糊驱动自动生成的效率和质量，从而降低人工编写驱动的成本、扩大模糊测试的适用范围。

1.2 国内外研究现状

模糊驱动的自动生成是提升模糊测试规模与深度的关键课题。围绕这一目标，研究者走出了两条主要路径：一条依赖程序分析来合成驱动，另一条借助大语言模型来生成驱动。下面分别梳理两条路径的代表性工作，分析各自的优势和局限，然后说明本文的切入点与创新点。

1.2.1 基于程序分析的模糊驱动生成

早期工作大多从已有代码中“学习”API 的使用方式。Google 的 FUDGE 系统^[8]是这个方向的开创者，它分析目标库的使用者代码，提取 API 调用模式并据此合成驱动。在 Google 内部部署中，FUDGE 为数千个库生成了上千个驱动，其中超过 200 个被集成到持续模糊测试中，帮助修复了 150 多个缺陷。Ispoglou 等人的 FuzzGen^[9]把这个思路往前推了一步，它分析多个消费者程序来构建抽象 API 依赖图（A²DG），从而推断出更丰富的调用序列组合。

一个有意思的思路来自 Zhang 等人的 APICRAFT^[10]。他们面对的是闭源 SDK，因为拿不到源码，因此只能综合利用头文件、二进制和运行痕迹等信息来收集 API 依赖，再通过多目标遗传算法组合生成驱动。APICRAFT 在 macOS SDK 上的实验很亮眼，自动生成的驱动比人工版本平均提升了 64% 的覆盖率，8 个月持续测试中发现了 142 个新漏洞，54 个获得了 CVE 编号。三星研究院的 Jeong 等人则换了个角度，他们的 UTOPIA 工具^[11]直接从单元测试用例中提取 API 调用序列。这个思路很巧妙：单元测试本身就是开发者精心编写的、验证过 API 规范用法的代码，天然避免了从消费者程序中提取时混入自定义逻辑的问题。UTOPIA 在 55 个项目中从 8000 余个单元测试生成了约 5000 个驱动，发现了 123 个漏洞。

更近期的工作开始引入形式化方法。Zhang 等人在 USENIX Security 2023 提出 RUBICK^[12]，用主动自动机学习算法把 API 使用模式建模为确定性有穷自动机，生成的驱动比 FuzzGen 多覆盖了 50.42% 的代码边。Yu 等人的 Hopper^[13]则从类型系统出发，自动推断参数间的依赖关系和取值约束，不需要消费者代码就能合成驱动。

总的来看，程序分析方法的优势在于能保证生成的驱动遵循真实的 API 调用模式。但它们的局限也很明显：必须有足够的先验样本（消费者代码、测试用例或使用实例），面对复杂语义约束和闭源库时仍然力不从心。

1.2.2 基于大模型的模糊驱动生成

大语言模型的出现为驱动生成带来了新范式。相比程序分析，LLM 方法的通用性更强，不局限于特定语言或接口。按反馈机制的精细程度，现有方法大致可以分为三个层次。

无迭代反馈的单次生成。最基础的做法是让 LLM 一次性生成驱动，不做后续改进。Deng 等人的 TitanFuzz^[14]证明了 LLM 可以零样本地为深度学习库生成模糊输入，Xia 等人的 Fuzz4All^[15]进一步把这个能力推广到任意编程语言。虽然这些工作针对的是测试输入而非驱动程序，但它们验证了 LLM 理解 API 语义的能力。在驱动生成领域，Google 的 OSS-Fuzz-Gen^[3]采用单次生成模式。Zhang 等人^[4]做了一个很有价值的大规模评估：在 30 个 C 项目的 86 个 API 上，用 5 种 LLM 和 6 种提示策略生成了 73 万多个候选驱动，跑了 3.75 CPU 年的对照实验。结论显示 LLM 生成驱动有潜力，但单次生成的质量和人工优化的驱动还有差距。问题的根源在于缺乏反馈，因为单次生成过程是开环的，无法根据执行效果做针对性改进。

粗粒度反馈的迭代生成。为了克服开环的局限，Lyu 等人的 PromptFuzz^[5]引入了覆盖率反馈来指导迭代。它通过指令化程序生成、错误自动校验和覆盖率驱动的提示变异来探索未覆盖区域。在 14 个库上的测试中，PromptFuzz 比 OSS-Fuzz 默认驱动和 Hopper 分别提高了 1.61 倍和 1.63 倍的分支覆盖率，还发现了 33 个新漏洞。但本文注意到一个关键限制，PromptFuzz 的反馈停留在 API 组合级别：它知道哪些 API 组合被覆盖了，却不知道具体哪行代码没跑到，更无法诊断未覆盖的原因。这种粗粒度反馈让 LLM 只能做相对盲目的变异，难以高效突

破覆盖率瓶颈。

知识增强的生成方法。Xu 等人的 CKGFuzzer^[16]尝试用知识图谱来增强 LLM 对被测库的理解，在 8 个库上平均提升了 8.73% 的覆盖率。但 CKGFuzzer 的知识图谱是静态构建的，不随迭代更新；其多智能体之间的协作也只是顺序串联，各智能体共享相同上下文，缺乏基于覆盖率反馈的差异化信息传递。

1.2.3 现有方法的不足与本文切入点

综合来看，程序分析方法可靠但依赖先验代码，LLM 方法灵活但可控性不足。特别是在 LLM 方法中，本文认为当前研究存在四个共性短板：

第一，覆盖率反馈粒度不足。现有迭代方法的反馈多停留在 API 组合或函数级别，无法告诉 LLM 具体哪行代码没覆盖、为什么没覆盖。理想的反馈应该精确到行和分支，并附带根因分析。

第二，单一模型职责过载。让一个 LLM 同时做规划、生成、修复和分析，上下文一长注意力就分散了。Liu 等人^[17]的研究指出 LLM 处理长上下文时存在“中间遗忘”现象，即关注首尾而忽略中间。在代码生成场景中，把 API 签名、覆盖率数据、编译错误、历史代码全塞进一个提示里，模型对关键信息的利用效率会明显下降。这个问题不是简单拆分提示词就能解决的，因为各环节需要不同的知识视图、独立的温度控制和隔离的状态管理，这些需求天然指向多智能体架构。

第三，缺乏结构化知识积累。多数方法每轮迭代相互独立，成功的代码模式、失败的尝试方向、发现的 API 约束都没有被系统性地记录和复用。后续轮次可能重蹈覆辙，也可能遗忘已验证有效的模式。

第四，生成策略单一。用统一的提示词模板生成所有驱动，忽略了不同类型库的差异。解析类库需要把原始字节直接喂给解析函数，有状态库则需要构造多步调用序列，两种场景的代码结构、输入分配和资源管理完全不同，单一提示词无法兼顾。

本文正是针对这四个不足展开工作：用多智能体架构解决注意力稀释，用行级覆盖率反馈实现精准迭代引导，用按槽位隔离的知识累积实现经验传递，用策略库加自动匹配实现对不同库的自适应生成。

1.3 研究目标

本文希望解决的核心问题是：如何让 LLM 生成的模糊驱动在反馈精度、职责分配、知识积累和策略适配这四个维度上都有实质性的提升？为此，本文设计并实现了 MAGIC4FDG (Multi-Agent Guided Iteration with Coverage for Fuzz Driver Generation)，一个端到端的模糊驱动自动生成系统。具体目标从四个维度展开。

在架构层面，本文需要实现由七个专职智能体组成的有向无环图协作架构。每个智能体只负责一件事：知识提取、策略规划、代码生成、编译修复、覆盖率分析或缺口诊断，并且只接收与自身职责相关的知识子集。这样做的目的是避免单模型上下文过载导致的注意力稀释。

在反馈层面，本文需要构建基于 llvm-cov 的行级与分支级覆盖率反馈闭环。不只是告诉系统“覆盖率是多少”，而是通过缺口诊断智能体分析具体哪些代码行没覆盖、为什么没覆盖，把分析结果转化为结构化约束，给代码生成智能体一个明确的改进方向。

在迭代层面，本文需要设计探索-利用双阶段范式。第一轮通过 LLM 场景推理和多样化策略库做广度探索，建立多条独立的生成轨迹；后续轮次基于约束反馈对各轨迹独立做增量改进。每条轨迹维护隔离的知识空间，跨轮次累积约束和有效模式，形成持续增长的经验库。

在评测层面，本文需要在 OSS-Fuzz 收录的多个开源 C/C++ 库上做系统性实验，与 PromptFuzz、OSS-Fuzz-Gen 等工具和人工驱动全面对比，并通过消融实验验证各组件的贡献。

1.4 研究内容

1.4.1 主要工作

围绕上述目标，本文工作聚焦于 MAGIC4FDG 系统的体系化设计与工程实现，核心要回答三个问题：多智能体协作如何提升生成质量？覆盖率反馈如何实现精准迭代？知识累积如何提升迭代效率？具体包括四个方面。

第一，多智能体协作架构的设计与实现。本文将驱动生成拆成七个子任务，每个交给一个专职智能体，用 DAG 调度框架按拓扑顺序执行。其中关键设计是

差异化上下文注入，即同一个知识库，不同智能体获取的上下文不一样。举例来说：策略规划智能体看到的是宏观的 API 分类摘要和覆盖率趋势，代码生成智能体看到的是具体的目标 API 签名和累积约束，缺口诊断智能体看到的是完整调用图和未覆盖的源码行。这样做的原因是如果把所有信息一并提供给每个智能体，注意力就会分散，反而降低生成质量。另外系统还设计了条件路由：变体全部编译失败就跳过模糊测试节省资源，覆盖率连续几轮不涨就自动终止。

第二，多提示词策略自适应生成范式。本文搭建了一个包含十种模糊策略的策略库，包括解析驱动、多 API 序列、往返验证、结构感知等，每种策略有 YAML 元数据描述适用条件，还有策略正文作为提示词后缀注入 LLM。同一轮里不同槽位拿到的基础模板一样，但策略后缀不同，这样产出驱动的探索风格就会不一样。探索阶段 Planner 智能体通过场景推理做策略匹配，如果推理失败还有基于特征标签的规则引擎兜底。在知识提取模块，本文用 clang AST 实现了五步流程：语法树解析、宏解析、调用图构建、语义标注和分类推断。

第三，覆盖率反馈迭代与知识累积机制。覆盖率数据的采集依赖 llvm-cov，精确到源码行级别。模糊测试本身在 Docker 容器内以 fork 模式运行（这样单个输入触发的崩溃不会中断整个测试进程），测试结束后再通过语料库重放来收集完整的 profraw 数据。每条生成轨迹拥有独立的知识空间，随轮次推进不断累积正负模式，比如“必须先调用 cJSON_Parse 才能操作返回的对象”属于正模式，“传入零长度缓冲区会触发断言失败”则是负模式。缺口诊断智能体的职责被刻意限定为只做分析、不写代码，它需要指出哪些行没覆盖并给出可能的原因，输出结构化约束交给生成智能体去落实。这种诊断与生成的分离是有意为之的，混在一起容易导致角色混淆、输出质量下降。收敛判定分两个层级：某条轨迹连续若干轮覆盖率不涨，就尝试更换策略；如果所有轨迹的整体涨幅都停滞了，则终止任务避免浪费算力。

第四，系统工程化实现与实验评估。工程层面，所有执行环节（编译、模糊测试、覆盖率采集）都在容器内完成，宿主机只负责调度。构建缓存机制让同一个库只需编译一次，后续轮次直接挂载缓存产物。效率方面做了多层并行：LLM 调用通过异步 API 轮询实现并发，多个变体的编译修复和模糊测试也是并行执行的。实验在 12 个 OSS-Fuzz 收录的 C/C++ 库上展开，对比对象包括 PromptFuzz、OSS-Fuzz-Gen 以及各库已有的人工驱动，同时设计了消融实验来量化各组件（多

智能体拆分、行级反馈、知识累积、策略库)的独立贡献。

1.4.2 论文结构

全文共六章，组织如下：

第一章是引言，交代研究背景和动机，梳理国内外研究现状，指出现有方法的不足，明确研究目标和内容。

第二章介绍相关技术基础，包括模糊测试与 **LibFuzzer** 引擎、覆盖率引导技术、大语言模型代码生成、多智能体协作框架以及 **clang AST** 静态分析，为后续设计提供技术铺垫。

第三章是 **MAGIC4FDG** 的需求分析与概要设计。先用系统流程图展示整体运行逻辑，再做涉众分析、功能性与非功能性需求分析和用例建模，最后通过“4+1”架构视图完成概要设计，明确模块职责和协作关系。

第四章是详细设计与实现。按模块逐一阐述各智能体的设计方案和核心实现，每个模块包含类图、顺序图和关键代码片段。

第五章是实验设计与结果分析。说明实验设置后，围绕系统性能和组件有效性进行多组实验，并通过长时模糊测试验证生成驱动的缺陷发现能力，用定量数据验证系统。

第六章总结全文，分析局限性，并展望未来该领域可能的方向。

第二章 相关技术

MAGIC4FDG 系统的设计融合了多个领域的技术。本章介绍系统依赖的五项核心技术基础：模糊测试与模糊驱动的基本原理、LibFuzzer 引擎的覆盖率引导机制、大语言模型的代码生成能力、多智能体协作框架，以及基于 clang AST 的静态分析。

这些技术看似分属不同领域，但在本系统中它们各司其职又紧密配合：模糊测试定义了任务目标和质量标准，覆盖率引导提供迭代反馈信号，大语言模型负责代码合成和约束推理，多智能体框架实现职责分离和协作编排，静态分析则提供结构化的上下文知识。

2.1 模糊测试与模糊驱动

模糊测试的历史可以追溯到 1990 年。Miller 等人向 90 个 Unix 工具喂入随机字符串，发现约 24% 的程序崩溃了，这种用随机输入触发异常的方法就此得名“fuzzing”^[18]。它的核心思想在于尽可能广泛地探索程序的输入空间来暴露缺陷，几乎不需要人工编写测试用例。三十多年过去，模糊测试已经从纯随机输入演化为覆盖率引导的智能化方法，是当前发现安全漏洞最有效的技术之一^[19]。

从运行机制看，模糊测试围绕一个不断迭代的循环展开。引擎拿到初始种子后，对其施加各种变异，如位翻转、字节插入删除、整数边界替换等，并批量产出测试用例^[20]。这些用例被逐一送入经过插桩的被测程序执行。插桩代码在运行时记录覆盖率信息，同时系统监控是否出现崩溃或内存越界。覆盖率引导模式下的核心判据是：某个输入是否触发了此前未见的代码边。触发了就留下继续变异，没有就丢弃。值得一提的是，AFL 和 LibFuzzer 等主流引擎采用的都是边覆盖而非路径覆盖^[21]。也就是说，它们关心控制流图中基本块之间的转移边有没有被走到，并不试图区分完整的执行路径序列。

在模糊测试系统中，三大核心组件各有分工：输入生成器基于种子做变异，

变异质量直接决定探索效率；执行器把输入送入目标程序；缺陷监控器通过 AddressSanitizer、UBSan 等工具捕获异常，同时采集覆盖率反馈^[22]。通常，输入生成器和缺陷监控器合在一起构成模糊引擎，而执行器的核心就是模糊驱动。

模糊驱动的角色是把模糊引擎产生的原始字节流转化为被测库 API 能接受的合法调用序列^[19]。它需要完成输入解析、环境初始化、API 调用和资源清理这几件事。以 LibFuzzer 为例，驱动要实现一个 LLVMFuzzerTestOneInput 函数，接收字节数据和长度，在函数体内把字节转化为有效的 API 调用^[23]。

驱动质量对测试效果的影响是决定性的。好的驱动能把随机输入映射到深层代码路径，触发复杂行为；差的驱动让大量输入在浅层校验处就被拒绝。驱动本身的正确性也很关键，错误的参数初始化或遗漏前置条件会导致虚假崩溃或漏掉真实漏洞。目前工业界的驱动主要靠人工编写，耗时且覆盖有限^[5]。这正是本文研究自动生成的出发点。

2.2 LibFuzzer 模糊引擎与覆盖率引导技术

LibFuzzer 是 LLVM 项目提供的进程内、覆盖率引导、进化式模糊测试引擎^[23]。和 AFL 这类进程外模糊器不同，LibFuzzer 把自身和被测库链接成同一个二进制，在单进程内反复调用目标函数并变异输入，省去了进程创建销毁的开销，吞吐量很高。如图2-1所示，它要求用户实现一个固定签名的目标函数，引擎运行时反复调用这个函数，每次传入不同的变异数据。

```
1 int LLVMFuzzerTestOneInput(const uint8_t *Data, size_t Size) {  
2     // 1. 输入数据解析与参数构造  
3     if (Size < 4) return 0;  
4     int flags = *(int*)Data;  
5     const char *payload = (const char*)(Data + 4);  
6     size_t payload_len = Size - 4;  
7     // 2. 环境初始化  
8     context_t *ctx = target_init(flags);  
9     // 3. 目标API调用  
10    target_process(ctx, payload, payload_len);  
11    // 4. 资源清理  
12    target_cleanup(ctx);  
13    return 0;  
14 }
```

图 2-1 LibFuzzer 模糊驱动代码结构示例

LibFuzzer 的覆盖率引导依赖 LLVM 的 SanitizerCoverage 插桩框架^[24]。编译阶段，编译器在每个基本块入口插入回调代码，运行时记录哪些位置被执行。引擎内部用一个 8 位边计数器位图来跟踪状态：每条控制流边占一个槽位，记录的“不是”是否走过”而是”命中次数落在哪个区间”。一旦某个新输入让位图出现了之前未见的模式（触发了新边，或让某条边的命中次数跳到新区间），引擎就将其保留继续变异。这种设计比单纯的覆盖/未覆盖二值判断更精细，因为它能区分循环执行一次和执行多次的行为差异。

实际使用中，LibFuzzer 支持 fork 模式（`-fork=N`）来隔离崩溃。主进程管理语料库，依次启动子进程跑模糊测试；子进程独立运行一段时间后退出现，把新覆盖输入汇报给主进程。某个输入触发崩溃只影响当前子进程，主进程随即启动新的继续探索。这对长时间运行很重要，单个崩溃不会中断对其他路径的探索。但 fork 模式也带来一个问题：工作进程被信号终止时，`profraw` 文件可能没写完整。本系统通过语料库重放解决这个问题，fork 模式结束后，用收集到的语料库在非 fork 模式下重跑一次（`-runs=0`），确保覆盖率数据完整。

精确的覆盖率数据收集依赖 LLVM 的 `llvm-cov` 工具链^[25]。具体做法是：编译阶段给目标加上 `-fprofile-instr-generate -fcoverage-mapping` 两个标志，启用源码级插桩。程序运行结束后会落盘一份 `profraw` 文件，记录原始的计数器数据。随后用 `llvm-profdata merge` 将可能存在的多份 `profraw` 合并为统一的 `profdata` 格式。最终通过 `llvm-cov export` 导出 JSON 报告，报告中包含每一行的执行次数以及每个分支条件的真/假覆盖状态。

这种粒度的数据对本系统而言不可或缺。粗粒度反馈（比如只告诉你哪些函数被调用过、哪些 API 组合被覆盖了）无法定位具体的薄弱点。而行级加分支级的信息能够直接告诉缺口诊断智能体：某个 `if` 分支从未走过 `false` 路径，或者某段错误处理代码完全没被触及。有了这样明确的分析目标，后续的迭代改进才有据可依。

2.3 大语言模型与代码生成技术

大语言模型（LLM）是基于 Transformer 架构^[26]的深度神经网络，通过在海量文本上预训练获得能力。Transformer 最关键的创新是自注意力机制：处理序

列中任何一个位置时都能直接关注其他所有位置，有效捕捉长距离依赖。这对代码生成尤为重要，因为变量定义和使用、函数声明和调用之间经常相隔数十甚至上百行，传统 RNN 难以建模这种远距离关联。从 GPT^[27]、BERT^[28]到 GPT-4^[29]，参数量从数亿增长到数千亿，”预训练 + 指令微调 + RLHF”的训练范式让模型既具备丰富知识储备，又能遵循复杂指令^[30]。

代码生成是检验 LLM 能力的一个重要维度。编程语言语法严格、语义明确，对模型的精确推理能力要求很高。OpenAI 的 Codex 在 HumanEval 上达到 28.8% 的单次通过率^[6]，此后 GPT-4 等模型持续提升。本文采用的 Anthropic Claude 系列^[31]在代码生成和推理上表现优异，Claude Opus 能在单次生成中产出数百行完整程序，多轮交互中对复杂约束的理解保持一致，这对本系统的多轮迭代生成至关重要。

在软件测试领域，LLM 的应用越来越广。TitanFuzz^[14]证明了 LLM 可以零样本生成深度学习库的测试输入，Fuzz4All^[15]把这个能力推广到任意语言。在驱动生成方面，PromptFuzz^[5]用迭代提示引导 LLM 持续生成新驱动，CKGFuzzer^[16]用知识图谱增强模型对被测库的理解。这些工作表明 LLM 确实具备理解 API 语义、推断调用约束并生成规范测试代码的能力。

不过 LLM 用于代码生成并非没有代价。最突出的问题是”幻觉”：模型有时会自信地写出根本不存在的 API 名称，或者给函数传入类型不匹配的参数。另一个现实约束来自上下文窗口：当需要同时塞入 API 签名、覆盖率报告、编译错误日志和上一轮生成的代码时，窗口容量捉襟见肘，关键信息被挤出注意力范围的情况并不罕见^[17]。此外，没有执行反馈的开环生成本质上是在”盲写”，正确性缺乏保障。本系统针对这些问题做了对应设计：多智能体架构将信息负载分散到不同角色，覆盖率反馈闭环提供执行级验证信号，迭代改进机制则让质量逐轮提升。

关于提示工程，LLM 的输出质量与提示设计密切相关^[32]，这一点在驱动生成场景中体现得尤为明显。提示需要在信息量和简洁性之间找到合适的位置：给得太少，模型缺乏必要的类型和约束信息，生成的代码编译都过不了；给得太多，注意力被稀释，模型反而忽略关键约束。本文的做法是差异化上下文注入，不同智能体根据各自职责获取不同侧面的信息。生成智能体看到的是 API 签名和策略描述，分析智能体看到的是覆盖率缺口和历史约束，各取所需而不互相干扰。

MAGIC4FDG 采用 Claude Opus 作为核心模型，选型论证在第三章展开。

2.4 多智能体协作框架

多智能体系统（MAS）的核心思路是将一个复杂任务拆分给多个自主智能体协作完成。在 LLM 驱动的多智能体系统中，每个智能体由一个 LLM 实例驱动，有特定的角色定义和知识边界，彼此通过结构化消息通信。相比单模型方案，多智能体架构有三方面优势：职责分离避免单个智能体的注意力稀释；各实例可以使用不同的生成参数针对不同任务优化；模块化设计便于独立测试和替换。

协作架构的选择直接影响系统效率。常见模式包括：流水线（严格按顺序依次处理）、黑板（共享全局状态空间）、合同网（动态竞标分配任务）和 DAG（按依赖关系决定执行顺序，支持条件分支和并行）。在模糊驱动生成场景中，各环节之间有明确的先后依赖：知识提取完成后才能规划策略，编译通过后才能执行模糊测试，但又不是纯线性的，中间存在条件判断和并行机会，因此 DAG 是最合适的架构选择。

近年来，LLM 多智能体框架在软件工程领域进展很快。MetaGPT^[33]模拟软件公司组织结构，通过产品经理、架构师、工程师等角色协作完成开发。Chat-Dev^[34]模拟团队沟通模式，用对话驱动协调。微软的 AutoGen^[35]采用可定制的对话式架构，在代码生成和数学推理上表现不错，但对话驱动的交互更适合探索性任务，对于有明确依赖关系的固定流程（如编译-测试-分析循环）不如 DAG 高效。在模糊测试领域，CKGFuzzer^[16]引入了多智能体，但其协作只是顺序串联，各智能体共享相同上下文，缺乏差异化知识视图和基于反馈的动态调整。

LangGraph^[36]是 LangChain 团队推出的多智能体编排框架，专门面向有状态的多参与者 LLM 应用。其核心抽象是 StateGraph，以共享状态为中心的有向图，节点代表智能体或处理函数，边代表执行流转。本文选择 LangGraph 主要基于两点考虑：一是条件路由能力，可以根据当前状态动态决定下一步执行哪个节点（如编译失败则跳回修复、覆盖率未提升则提前终止）；二是状态持久化支持，任意节点都能保存和恢复执行状态，为长时间迭代任务提供断点恢复能力。

本文选择 LangGraph 的 DAG 架构作为系统的协作基础设施。模糊驱动生成

的各环节（知识提取 → 策略规划 → 代码生成 → 编译修复 → 模糊测试 → 覆盖率分析 → 缺口诊断）有明确的依赖关系，天然适合 DAG 表达；条件路由实现自适应流程控制；状态持久化支持跨轮次知识累积和断点恢复。具体设计决策在第三章展开。

2.5 静态程序分析与 clang AST

静态程序分析是在不执行程序的前提下，通过对源代码进行结构和行为建模来推断程序性质的技术^[37]。其优势在于不受特定输入限制，理论上能覆盖所有可能的执行路径。常用技术包括词法分析、语法分析、类型推导、数据流分析等^[38]。在本文的场景中，静态分析的价值在于从被测库源码中提取结构化的 API 知识，为 LLM 提供精确的上下文信息。

AST（抽象语法树）是静态分析中最基础的程序表示^[39]。它将源代码组织为树形结构，每个节点对应一个语法单元（函数定义、变量声明、表达式等）。相比原始文本，AST 消除了空白和注释等无关信息，只保留语义结构，适合做类型分析和调用关系提取。

本文的知识提取基于 clang 编译器前端的 AST JSON 导出（`clang -Xclang -ast-dump=json`）。之所以选 clang 而不是 tree-sitter 之类的轻量解析器，根本原因在于 C/C++ 的复杂性远超表面语法。宏展开、模板实例化、typedef 链解析、条件编译，这些都需要经过完整的编译语义分析才能得到正确结果，tree-sitter 做不到这一点。而 clang 的 JSON 输出中，每个 AST 节点都带有 id、kind、type、loc 等字段，函数签名和参数类型可以直接读取，无需二次推断。

知识提取的具体流程如下：先从 FunctionDecl 节点中筛选出公开 API 签名；再通过预处理器输出识别宏常量和枚举值；然后分析 CallExpr 节点构建函数间的调用图；之后根据类型特征和命名模式对 API 做语义标注（区分构造/析构、读/写、配置类接口等）；最后按功能相关性对 API 进行分组。各阶段的实现细节在第四章展开。

关于替代方案的排除：tree-sitter 的设计目标是快速增量解析以支持编辑器高亮，不经过完整的编译语义分析，无法提供宏展开后的真实结构和完整类型信息，而这些恰恰是驱动生成所必需的。libclang 的 Python 绑定虽然提供了 AST

访问接口，但遍历效率低且处理复杂模板时稳定性不足。`clang AST JSON` 导出在 `Docker` 容器内一次性完成编译和导出，既保证精确性又实现了环境隔离。

2.6 本章小结

本章介绍了 `MAGIC4FDG` 系统依赖的五项核心技术。模糊测试和模糊驱动定义了任务目标；`LibFuzzer` 和 `llvm-cov` 提供了精确的行级覆盖率反馈；大语言模型的代码生成能力是系统的核心智能；多智能体框架提供了职责分离和协作编排的基础设施；`clang AST` 静态分析为生成过程提供了结构化上下文知识。这五项技术共同构成了系统的技术基础，为后续的设计和实验提供支撑。

第三章 覆盖率引导的多智能体模糊驱动生成系统 MAGIC4FDG 的需求分析与概要设计

在进入实现之前，需要先将系统的需求和架构梳理清楚。本章分三部分展开：先通过流程图给出系统运行的全貌，再做需求分析明确系统该做什么、做到什么程度，最后用”4+1”视图把架构设计讲清楚。

3.1 系统整体概述

前面提到，现有 LLM 驱动生成方法的几个痛点（反馈粒度粗、单模型过载、缺乏知识积累、策略单一），归根结底是把太多事情交给了一个模型。本文的解决思路是将任务拆分，让不同的智能体各司其职，通过共享状态协调，形成”知识提取 → 策略规划 → 代码生成 → 编译修复 → 模糊测试与覆盖率收集 → 状态持久化 → 缺口诊断”的闭环。整个系统由七个节点组成有向无环图，其中四个（Planner、Generation、Patching、Analyst）调用大语言模型执行推理任务，属于 LLM 智能体；另外三个（Knowledge、Coverage、Checkpoint）执行确定性计算或 I/O 操作，本质上是计算节点，但在 DAG 调度框架中统一以”节点”身份参与编排。第一轮做广度探索，后续轮次做深度利用，这就是 MAGIC4FDG 的基本运行逻辑。

如图3-1所示，系统运行流程分为四个阶段：知识准备、驱动生成与编译修复、模糊测试与覆盖率收集、缺口诊断与迭代控制。每个阶段由对应的智能体负责，阶段之间通过条件路由做自适应的流程控制。

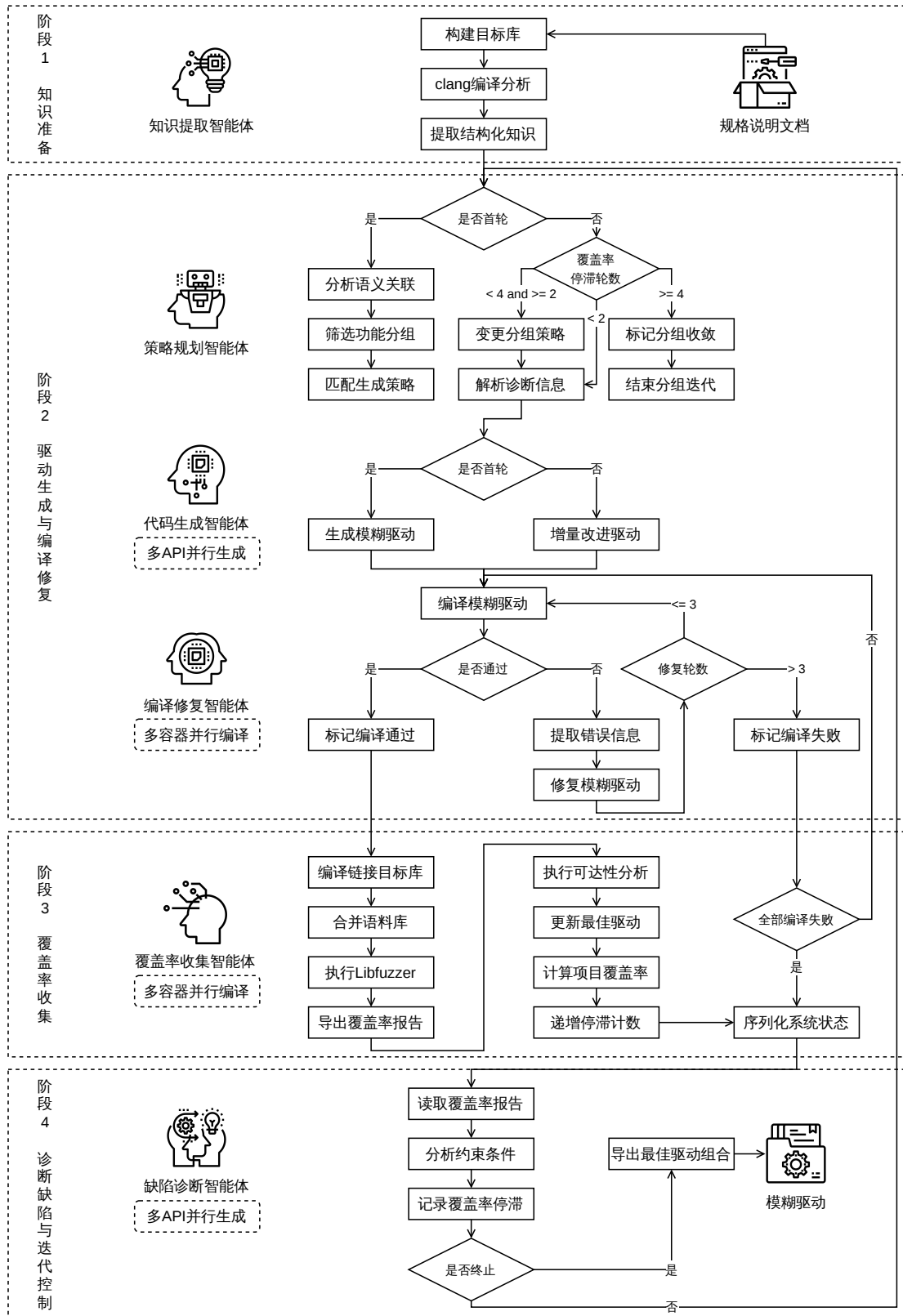


图 3-1 MAGIC4FDG 系统整体流程图

知识准备阶段。流水线的第一步交给知识提取智能体（Knowledge Agent）完成。该智能体先在 Docker 容器内编译目标库，随后借助 clang 前端执行 AST 导

出。导出内容涵盖公开 API 的函数签名、参数类型与语义标注、函数间调用图，以及宏常量和枚举定义。值得一提的是，首次构建的产物会被缓存到宿主机上，后续轮次只需以只读方式挂载，省去重复编译的开销。整个知识准备仅在流水线启动时执行一次，其产出构成全局知识库，贯穿后续所有迭代轮次。

驱动生成与编译修复阶段。这个阶段涉及三个智能体的协作：策略规划 (Planner Agent)、代码生成 (Generation Agent) 和编译修复 (Patching Agent)。

策略规划智能体在知识提取完成后启动工作。首轮迭代属于探索阶段，它加载策略库里十种预定义策略的元数据，每种策略用结构化格式声明适用条件、最佳场景和不兼容约束，同时附带一段策略正文作为提示词后缀。智能体调用 LLM 分析目标库 API 之间的语义关系和功能分组，把策略元数据的适用条件跟 API 特征做匹配，给每个驱动槽位选一个最合适的策略。这里的关键在于，不同槽位会拿到完全不同的策略后缀，从而在代码生成环节产出风格迥异的驱动。如果 LLM 调用失败，系统回退到基于调用图的规则引擎做确定性匹配。到了后续轮次（利用阶段），策略规划智能体不再调 LLM，而是根据各槽位的覆盖率停滞情况决定是否切换策略或标记收敛，同时从缺口诊断结果中提取额外的目标 API 补充到对应槽位的生成上下文里。

代码生成智能体根据规划结果并行生成驱动代码。它的提示由三部分拼接而成：基础模板（库名、接口规范、通用约束）、API 知识上下文（目标 API 的签名、类型、调用关系）、策略提示词后缀（由 Planner 匹配的策略正文）。首轮是从零生成，LLM 基于这三部分合成完整驱动；后续轮次是增量改进，提示里额外包含当前最佳代码、Analyst 输出的结构化约束、以及跨轮次累积的经验知识，要求模型保留已有覆盖路径并定向扩展新路径。

编译修复智能体对所有生成的变体做编译验证和错误修复。每个变体在 Docker 容器内用带 ASan 插桩的方式编译，跟目标库静态链接。编译通过则标记成功；未通过的，提取编译器报错，将代码和错误信息注入修复提示模板由 LLM 修复，最多尝试三次。

编译修复完成后，系统做第一个条件路由判断，只要有至少一个变体编译通过，就进入下一阶段；如果全部失败，则跳过模糊测试直接进入状态持久化，避免无效执行。

模糊测试与覆盖率收集阶段。覆盖率收集智能体 (Coverage Agent) 负责跑

模糊测试并收集精确的覆盖率数据。对每个编译通过的变体，它在 **Docker** 容器里依次做这几件事：先用覆盖率插桩标志重新编译驱动并链接目标库；然后合并种子语料库和历史累积语料库，以 **LibFuzzer** 的 **fork** 模式跑模糊测试。**fork** 模式让每个输入在独立子进程里执行，某个输入触发崩溃不会终止整个测试进程；测试结束后通过语料库重放收集完整覆盖率数据（**fork** 模式下 **profrw** 可能写不完整，因此需要重放一遍）；最后用 **LLVM** 工具链合并原始数据并导出行级与分支级覆盖率报告。另外，这个智能体还会做控制流图可达性分析，把结构上不可达的代码行标记出来，免得后面 **Analyst** 在这些行上浪费精力。覆盖率收集完毕后，系统逐槽位比对本轮与历史最佳覆盖率。若有提升则替换最佳版本，否则将该槽位的停滞计数加一。项目级覆盖率取所有槽位覆盖行的并集。

随后流水线进入状态持久化节点（**Checkpoint Node**）。这一节点不涉及 **LLM** 调用，职责单一，即把当前完整的流水线状态（含各变体源码）序列化为检查点文件写入磁盘。有了这份快照，用户可以从任意已完成的轮次恢复执行。

缺口诊断与迭代控制阶段。这一阶段由 **Analyst Agent** 主导。它逐一审视每个活跃槽位的覆盖率缺口，尝试定位根因。具体做法是：读取该槽位当前最佳代码、未覆盖行列表和函数级覆盖率数据，连同 **API** 知识上下文一并注入分析提示模板，交由 **LLM** 完成诊断。**LLM** 返回的是结构化结果，输出分为三部分：约束列表（要触发某个分支需满足的前置条件）、未覆盖代码簇（按功能分组的未覆盖区域）、以及新发现的知识点。这些内容会追加到该槽位自身的知识空间里，轮次之间持续累积，使系统在迭代中逐步加深对目标库的理解。

Analyst 完成分析后，系统做第二个条件路由判断，检查四个终止条件：项目级覆盖率达到用户设定的目标、迭代轮次到上限、所有槽位都已收敛、或者项目级覆盖率连续多轮没有明显提升。满足任何一个就终止迭代，输出各槽位的最佳版本并生成实验报告；否则回到策略规划智能体开始新一轮，进入利用阶段的闭环优化。

3.2 需求分析

MAGIC4FDG 的核心功能是为 **C/C++** 开源库自动生成高质量模糊驱动，并通过覆盖率反馈迭代持续优化。除此之外，系统还需具备可扩展性、可靠性、可

恢复性等非功能性需求。下面从涉众分析出发，明确谁会使用本系统、他们关心什么，再逐一梳理功能性和非功能性需求，最后给出用例图和用例描述。

3.2.1 涉众分析

根据技术背景和使用目的的不同，MAGIC4FDG 系统的涉众可以分为三类：目标项目开发者、模糊测试使用者和模糊测试研究人员。表3-1列出了这三类涉众的特征和期望。

表 3-1 涉众分析结果

涉众名称	涉众特征	涉众期望
目标项目开发者	被选为测试对象的开源项目的维护者，通常具备较强的 C/C++ 工程能力，熟悉自身接口设计与使用语义，了解模糊测试平台的基本要求。该类涉众直接参与项目构建与接口演进，对系统产出的驱动可用性与行为一致性有要求。	希望系统能够在不破坏接口契约的前提下，自动生成结构完整、逻辑合理、覆盖真实调用路径的模糊测试驱动，用于集成测试、安全审计或接口验证，降低人工编写负担并提高测试质量。
模糊测试使用者	使用 LibFuzzer、AFL 或 OSS-Fuzz 等平台执行模糊测试的测试工程师或安全研究人员，关注模糊驱动的适配性与执行效果。该类涉众具备模糊测试实践经验，熟悉驱动编写规范与覆盖率评估方法，但不一定深入了解目标库的内部实现。	希望系统生成的驱动能够直接用于主流模糊测试引擎，具备较高的代码覆盖率和缺陷发现能力；同时期望系统提供覆盖率报告和迭代过程记录，便于评估测试效果和定位潜在漏洞。
模糊测试研究人员	从事模糊测试自动化、驱动生成或大语言模型辅助测试等方向研究的学者或研究工程师。该类涉众关注系统的技术方法与创新点，通常将本系统与其他工具进行对比实验以验证方法的有效性。	希望系统具备良好的可复现性和可配置性，能够在不同目标库上稳定运行；同时期望系统提供详细的实验数据（覆盖率变化曲线、迭代轮次统计、消融实验支持等），便于进行公平的横向对比和方法分析。

三类涉众的差异主要体现在技术背景和关注重点上。目标项目开发者关注驱动是否符合 API 使用规范及缺陷发现能力；模糊测试使用者关注覆盖率、崩溃数量及与现有测试基础设施的兼容性；模糊测试研究人员则更关注实验数据和配置灵活性。三者共同期望系统能够在减少人工干预的同时保证驱动质量。

3.2.2 功能性需求分析

在调研现有工具的基础上，本文梳理出七个方面的功能性需求：目标库配置与知识提取、策略规划与驱动生成、编译修复、模糊测试与覆盖率收集、缺口诊断与迭代优化、状态持久化与恢复、实验报告生成。具体内容见表3-2。

表 3-2 功能性需求列表

需求编号	需求名称	需求描述
R1	目标库配置与知识提取	用户可以通过配置文件指定目标 C/C++ 库的构建方式、头文件路径和静态库路径，系统自动在 Docker 容器内完成构建并提取 API 知识
R2	策略规划与驱动生成	系统应能够根据目标库的 API 特征自动匹配合适的模糊策略，并基于策略指导生成多个风格互补的模糊驱动代码
R3	编译修复	系统应能够对生成的驱动代码执行编译验证，对编译失败的变体自动诊断错误原因并尝试修复
R4	模糊测试执行	系统应能够对编译成功的驱动在隔离环境中执行模糊测试，支持崩溃隔离和语料库累积
R5	覆盖率收集与分析	系统应能够收集行级与分支级覆盖率数据，执行可达性分析过滤不可达代码，并维护每个驱动的最佳版本记录
R6	缺口诊断与迭代优化	系统应能够对覆盖率缺口进行根因分析，输出结构化约束描述，并基于分析结果在后续轮次中定向改进驱动代码
R7	状态持久化与恢复	系统应能够在每轮迭代后保存完整的流水线状态，支持从任意检查点恢复执行
R8	实验报告生成	系统应能够在迭代结束后生成包含覆盖率统计、迭代历史和最佳驱动代码的实验报告

目标库配置与知识提取 (R1) 是一切的起点。用户编写一个 JSON 配置文件，将库名、构建命令、头文件路径、静态库路径等信息填入即可。系统拿到配置后在 Docker 里编译目标库，然后用 clang AST 导出拿到结构化的 API 知识，包括函数签名、参数类型、调用图、宏常量等。这步的产出是一个全局知识库，后续所有智能体都从这里获取信息。

策略规划与驱动生成 (R2) 是整个系统最核心的部分。规划环节分析 API 的特征和适合的测试策略，然后给每个槽位分配一个；生成环节拿着分配好的策略后缀、API 知识和基础模板去调 LLM，让它写出完整的模糊驱动代码。第一轮从零开始写，后续轮次则在已有代码上做增量改进。

编译修复 (R3) 的目的很直接——保证代码能编译通过。变体逐个送入 Docker 容器，以带 ASan 插桩的方式编译并与目标库静态链接。编译未通过时，系统提取编译器报错交给 LLM 尝试修复，上限三次，超过即放弃。之所以设定三次而非更多，是在修复成本与收益之间做的权衡——实践中绝大多数可修复的错误在前两轮就能解决，第三轮仍失败的往往涉及设计层面的问题。保住尽可能多的变体，后续模糊测试才有足够的样本量。

模糊测试执行 (R4) 与覆盖率收集与分析 (R5) 共同构成测试与反馈环节。前者在 Docker 容器内以 fork 模式运行 LibFuzzer，每个输入在独立子进程中执行，单个崩溃不会拖垮整体测试进程。后者负责精确度量，通过语料库重放（非 fork 模式）配合 LLVM 工具链，导出行级与分支级覆盖率数据。此外，系统还执行控制流图可达性分析，将结构上不可达的代码行预先标记，避免后续诊断环节在这些行上做无用功。

缺口诊断与迭代优化 (R6) 是覆盖率能够持续攀升的关键所在。Analyst 针对每个槽位的未覆盖代码做根因分析，产出两类结构化信息：约束列表（触发特定分支所需的前置条件）和未覆盖代码簇（按功能聚类的未覆盖区域）。这些信息被写入各槽位独立的知识空间，轮次间不断累积。到了后续迭代，Generation Agent 读取这些约束作为增量改进的精确指引，相当于把“哪里没覆盖、为什么没覆盖”的诊断结论直接转化为生成指令。

状态持久化与恢复 (R7) 保障容错能力。每轮迭代完成后，系统把完整的流水线状态序列化为检查点文件，包括所有槽位的最佳代码、覆盖率数据、累积知识和迭代计数。异常中断时用户可以指定检查点恢复执行，无需从头开始。

实验报告生成（R8）满足用户对结果的查阅需求。迭代终止后系统自动生成包含项目级覆盖率统计、各槽位覆盖率对比、迭代历史和最佳驱动源码的结构化报告，方便评估效果和做横向对比。

3.2.3 非功能性需求分析

功能性需求之外，非功能性需求同样重要。本文将其分为质量属性、性能需求和约束三部分，具体见表3-3。

表 3-3 非功能性需求列表

需求编号	需求类型	需求名称	需求描述
QA1	质量属性	可扩展性	系统应支持通过配置文件接入新的目标库，无需修改系统源码
QA2	质量属性	可靠性	系统生成的驱动在经过编译修复后应达到 60% 以上的编译通过率
QA3	质量属性	可恢复性	系统应支持从任意轮次的检查点恢复执行，恢复后状态与中断前一致
QA4	质量属性	隔离性	所有编译、模糊测试和覆盖率收集操作应在 Docker 容器内执行，不影响宿主机环境
QA5	质量属性	可复现性	在相同配置和相同大语言模型下，系统的覆盖率结果应具备统计意义上的可复现性
PR1	性能需求	并行处理能力	系统应支持多个驱动变体的并行生成、并行编译和并行模糊测试
PR2	性能需求	构建缓存	目标库首次构建后应缓存产物，后续轮次的编译和模糊测试应复用缓存，避免重复构建
CS1	约束	开发语言	系统使用 Python 语言进行开发
CS2	约束	容器化依赖	系统运行依赖 Docker 引擎和 LLVM 工具链

质量属性方面本文列出五条，其中最重要的两条是可扩展性（QA1）和隔离性（QA4）。可扩展性的含义是，用户更换目标库时只需修改配置文件，系统代码本身不动。这一点直接决定了系统能否脱离实验室环境、真正被他人使用。隔离性则依赖 **Docker** 容器化来兑现，构建、模糊测试均在容器内完成，宿主机环境不受影响，多个库并行运行时也彼此隔离。可靠性（QA2）要求编译修复环节切实有效，每轮需保证足够数量的变体编译通过，否则后续测试无从谈起。可恢

复性（QA3）通过检查点机制实现，中断后从最近的检查点恢复即可继续。可复现性（QA5）是实验评估的基本前提，本文通过固定随机种子与确定性调度两项措施加以保障。

性能需求方面，系统需要在代码生成、编译修复和模糊测试三个环节支持多变体并行执行（PR1）。这三个环节的计算相互独立，并行化能显著缩短单轮迭代耗时，充分利用多核资源。构建缓存（PR2）则要求目标库的编译产物在首次构建后持久化存储，后续轮次一律通过只读挂载复用，杜绝每轮重复构建带来的时间浪费。

约束方面，系统用 Python 开发（CS1），利用其丰富的生态支持 LLM 调用和流水线编排；运行依赖 Docker 引擎提供容器化执行环境，依赖 LLVM 工具链提供覆盖率插桩和数据导出能力（CS2）。

3.2.4 系统用例图

基于前面的涉众分析和功能性需求，本文对相关描述进行了实体识别、用例拆分和筛选，构建了系统用例图，如图3-2所示。

系统包含 10 个用例：配置目标库、提取 API 知识、规划模糊策略、生成模糊驱动、修复编译错误、执行模糊测试、收集覆盖率数据、诊断覆盖率缺口、恢复中断任务、查看实验报告。参与者有五类：三类人类参与者（目标项目开发者、模糊测试使用者、模糊测试研究人员）和两类外部系统参与者（Docker 引擎、大语言模型服务）。

三类人类参与者都可以使用配置目标库、恢复中断任务和查看实验报告。提取 API 知识、规划模糊策略、生成模糊驱动、修复编译错误、执行模糊测试、收集覆盖率数据和诊断覆盖率缺口这七个用例是系统内部自动执行的流程，用户启动任务后由系统自动编排。Docker 引擎参与知识提取、编译修复、模糊测试和覆盖率收集四个用例；大语言模型服务参与策略规划、驱动生成、编译修复和缺口诊断四个用例。

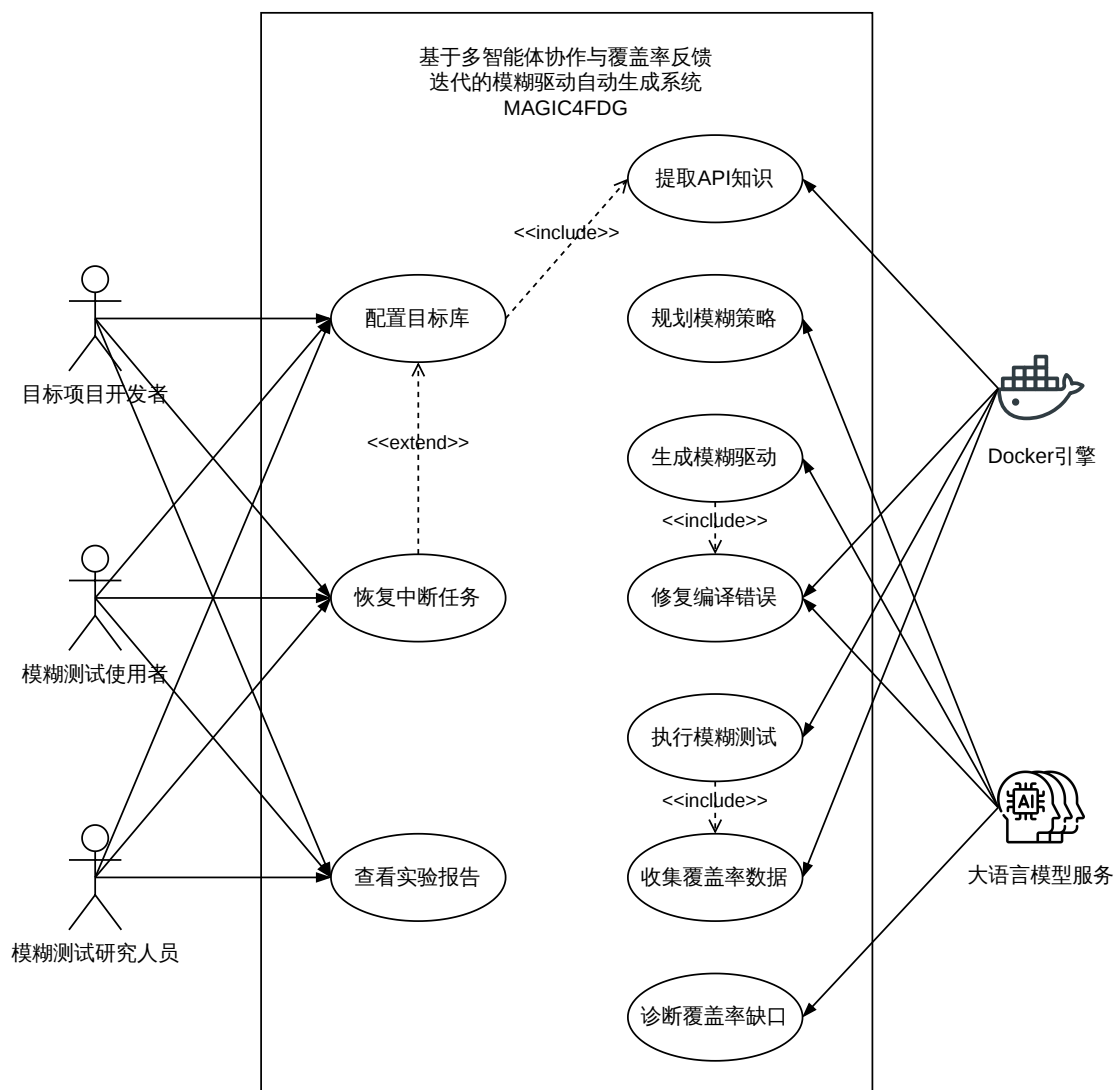


图 3-2 MAGIC4FDG 系统用例图

3.2.5 系统用例描述

上面的用例图从用户视角展示了系统行为与需求之间的关系。下面逐一描述每个用例的细节，帮助理解系统功能并指导后续设计。

配置目标库是用户使用系统的入口。用户编写 **JSON** 配置文件声明目标库的构建方式和路径信息，通过命令行参数控制迭代轮次、目标覆盖率等运行时行为，系统据此启动自动化流程。表3-4是该用例的详细描述。

表 3-4 用例”配置目标库”详细描述

用例编号	UC1
用例名称	配置目标库
参与者	目标项目开发者、模糊测试使用者、模糊测试研究人员
前置条件	目标库源码可通过网络获取，用户了解目标库的构建方式
后置条件	系统获得有效的目标库配置，可启动自动化生成流程
基本流程	1. 用户编写目标库配置文件，指定库名、构建命令、头文件路径、静态库路径和覆盖率源文件路径 2. 用户通过命令行指定配置文件路径和运行参数（最大轮次、目标覆盖率、模糊测试时长等） 3. 系统验证配置文件格式和必填字段的完整性 4. 系统启动流水线执行
扩展流程	3a. 配置文件格式错误或缺少必填字段时，系统输出错误提示并终止

提取 API 知识是获取目标库结构化信息的基础操作。系统利用 clang 前端的 AST 导出能力，在 Docker 容器内精确提取函数签名、参数类型、调用图和宏常量定义，构建产物缓存到宿主机避免后续重复构建。表3-5是该用例的详细描述。

表 3-5 用例”提取 API 知识”详细描述

用例编号	UC2
用例名称	提取 API 知识
参与者	Docker 引擎
前置条件	系统获得有效的目标库配置
后置条件	系统获得目标库的结构化 API 知识（函数签名、调用图、类型信息）
基本流程	1. 系统在 Docker 容器内执行目标库的构建命令 2. 系统通过 clang AST 导出提取头文件和源文件中的 API 信息 3. 系统解析 AST JSON，提取函数签名、参数类型、调用图和宏定义 4. 系统将构建产物缓存至宿主机 5. 系统将提取结果存入全局知识库
扩展流程	1a. 构建失败时，系统输出构建日志并终止流程

规划模糊策略是为每个槽位选择最合适生成策略的决策环节。探索阶段，Planner 通过 LLM 把策略库中十种策略与目标库 API 特征做匹配；利用阶段，根据覆盖率停滞情况动态调整策略分配。表3-6是该用例的详细描述。

表 3-6 用例”规划模糊策略”详细描述

用例编号	UC3
用例名称	规划模糊策略
参与者	大语言模型服务
前置条件	系统已完成 API 知识提取
后置条件	每个模糊驱动获得匹配的生成策略和目标 API 列表
基本流程	1. 系统加载策略库中十种模糊策略的元数据 2. 系统将 API 知识注入规划提示模板 3. 系统调用大语言模型分析 API 语义关系并匹配策略 4. 系统为每个模糊驱动分配目标 API 和对应策略
扩展流程	3a. 大语言模型调用失败时，系统回退至基于调用图的规则引擎进行确定性匹配

生成模糊驱动是核心的代码生成环节。每个槽位接收差异化的策略后缀，引导 LLM 采用不同的代码结构和 API 调用模式；首轮从零生成，后续轮次切换为基于约束和累积经验的增量改进。表3-7是该用例的详细描述。

表 3-7 用例”生成模糊驱动”详细描述

用例编号	UC4
用例名称	生成模糊驱动
参与者	大语言模型服务
前置条件	策略规划完成，每个驱动已分配目标 API 和策略
后置条件	系统获得多个模糊驱动代码变体
基本流程	1. 系统为每个驱动组装提示（基础模板 +API 知识 + 策略后缀） 2. 系统并行调用大语言模型生成驱动代码 3. 系统从模型输出中提取代码块并保存为变体文件
扩展流程	2a. 后续轮次，提示额外包含最佳代码、约束列表和累积经验

修复编译错误保证生成的代码能通过编译。编译修复智能体在 Docker 容器内做带 ASan 插桩的编译，失败的变体提取错误信息让 LLM 修复，最多三次。表3-8是该用例的详细描述。

表 3-8 用例”修复编译错误”详细描述

用例编号	UC5
用例名称	修复编译错误
参与者	Docker 引擎、大语言模型服务
前置条件	代码生成智能体已输出驱动变体
后置条件	变体被标记为编译成功或编译失败（已耗尽重试次数）
基本流程	1. 系统在 Docker 容器内对变体执行带插桩的编译 2. 编译成功，标记变体为通过
扩展流程	1a. 编译失败时，系统提取错误信息并调用大语言模型生成修复代码 1b. 重新编译修复后的代码，最多重试三次 1c. 三次重试均失败，标记变体为编译失败

执行模糊测试是对目标库做自动化安全测试的核心环节。`fork` 模式让每个输入在独立子进程里跑，单个崩溃不会终止整个测试；每轮发现的有效输入持久化存储供后续轮次复用。表3-9是该用例的详细描述。

表 3-9 用例”执行模糊测试”详细描述

用例编号	UC6
用例名称	执行模糊测试
参与者	Docker 引擎
前置条件	存在至少一个编译成功的驱动变体
后置条件	系统获得模糊测试产生的语料库和崩溃样本
基本流程	1. 系统以覆盖率插桩标志重新编译驱动变体 2. 系统合并种子语料库与历史累积语料库 3. 系统以 <code>fork</code> 模式运行 <code>LibFuzzer</code> 执行模糊测试 4. 系统收集测试产生的新语料库和崩溃样本
扩展流程	3a. 所有变体均编译失败时，跳过本用例直接进入状态持久化

收集覆盖率数据是获取精确反馈的关键环节。`fork` 模式下 `profraw` 文件可能写不完整，所以系统通过语料库重放在非 `fork` 模式下重新执行所有输入来保证数据完整性，同时维护每个槽位的最佳版本记录和停滞计数。表3-10是该用例的详细描述。

表 3-10 用例”收集覆盖率数据”详细描述

用例编号	UC7
用例名称	收集覆盖率数据
参与者	Docker 引擎
前置条件	模糊测试执行完成，语料库已生成
后置条件	系统获得行级与分支级覆盖率报告，最佳版本记录已更新
基本流程	1. 系统通过语料库重放机制收集完整的覆盖率原始数据 2. 系统通过 LLVM 工具链合并数据并导出覆盖率报告 3. 系统执行控制流图可达性分析，标记不可达代码行 4. 系统比较当前覆盖率与历史最佳，更新最佳版本记录 5. 系统计算项目级覆盖率并更新停滞计数
扩展流程	4a. 当前覆盖率未超过历史最佳时，递增该驱动的停滞计数

诊断覆盖率缺口是实现精准迭代改进的智能分析环节。Analyst 只输出结构化的约束描述和未覆盖代码簇分析，把”诊断”和”生成”的职责分开；诊断结果持久化到每个槽位的隔离知识空间，跨轮次累积形成经验库。表3-11是该用例的详细描述。

表 3-11 用例”诊断覆盖率缺口”详细描述

用例编号	UC8
用例名称	诊断覆盖率缺口
参与者	大语言模型服务
前置条件	覆盖率数据收集完成
后置条件	每个活跃驱动获得结构化的约束列表和未覆盖代码簇分析
基本流程	1. 系统读取每个驱动的最佳代码和未覆盖代码行列表 2. 系统将覆盖率数据与 API 知识注入分析提示模板 3. 系统调用大语言模型进行根因分析 4. 系统将输出的约束和知识追加到驱动的隔离知识空间
扩展流程	3a. 大语言模型输出格式异常时，系统跳过该驱动的本轮分析

恢复中断任务让用户能从之前保存的检查点继续执行。MAGIC4FDG 的单次完整运行可能持续数小时（取决于目标库复杂度、迭代轮次和模糊测试时长），期间可能因网络波动、LLM 服务不可用或系统资源不足而中断。检查点恢复机

制确保用户无需从头开始，可以从最近一次成功完成的轮次继续。表3-12是该用例的详细描述。

表 3-12 用例” 恢复中断任务” 详细描述

用例编号	UC9
用例名称	恢复中断任务
参与者	目标项目开发者、模糊测试使用者、模糊测试研究人员
前置条件	系统存在之前保存的检查点文件
后置条件	系统从指定检查点恢复完整的流水线状态并继续迭代
基本流程	1. 用户通过命令行参数指定检查点文件路径 2. 系统反序列化检查点文件，恢复流水线状态 3. 系统从恢复的轮次继续执行迭代流程
扩展流程	2a. 检查点文件损坏或格式不兼容时，系统输出报错并终止

查看实验报告让用户了解系统的运行结果和测试效果。对三类涉众来说，报告是评估系统产出质量的主要途径——开发者通过它了解驱动覆盖了哪些 API 路径、发现了哪些缺陷；测试使用者通过它评估覆盖率水平和崩溃发现能力；研究人员通过其中的定量数据做横向对比。报告同时以 Markdown 和 JSON 两种格式输出，兼顾可读性和自动化处理需求。表3-13是该用例的详细描述。

表 3-13 用例” 查看实验报告” 详细描述

用例编号	UC10
用例名称	查看实验报告
参与者	目标项目开发者、模糊测试使用者、模糊测试研究人员
前置条件	系统迭代终止（满足任一终止条件）
后置条件	用户获得包含覆盖率统计和最佳驱动代码的结构化报告
基本流程	1. 系统在迭代终止后自动生成 Markdown 和 JSON 格式的实验报告 2. 报告包含项目级覆盖率、各驱动覆盖率对比、迭代历史和最佳驱动源码 3. 用户查阅报告评估测试效果
扩展流程	1a. 用户可通过配置选择报告中包含的内容项

3.3 系统概要设计

本节用”4+1”视图方法描述 MAGIC4FDG 的架构。”4+1”视图包含逻辑视图、开发视图、进程视图、物理视图和场景视图五部分，其中场景视图就是 3.2 节的用例视图，已经在需求分析阶段完成了。下面先介绍整体架构设计，再从模块划分、逻辑视图、开发视图、进程视图和物理视图四个维度展开。

3.3.1 系统整体架构设计

图3-3展示了 MAGIC4FDG 的整体架构。用户提供目标库配置后，系统在 Docker 容器内依次完成知识提取、驱动生成、编译修复、模糊测试与覆盖率收集、缺口诊断等环节。各环节的产出经由共享状态对象流转，形成迭代反馈闭环以持续优化覆盖率。图中按模块划分了系统功能，箭头标明模块间的依赖方向。

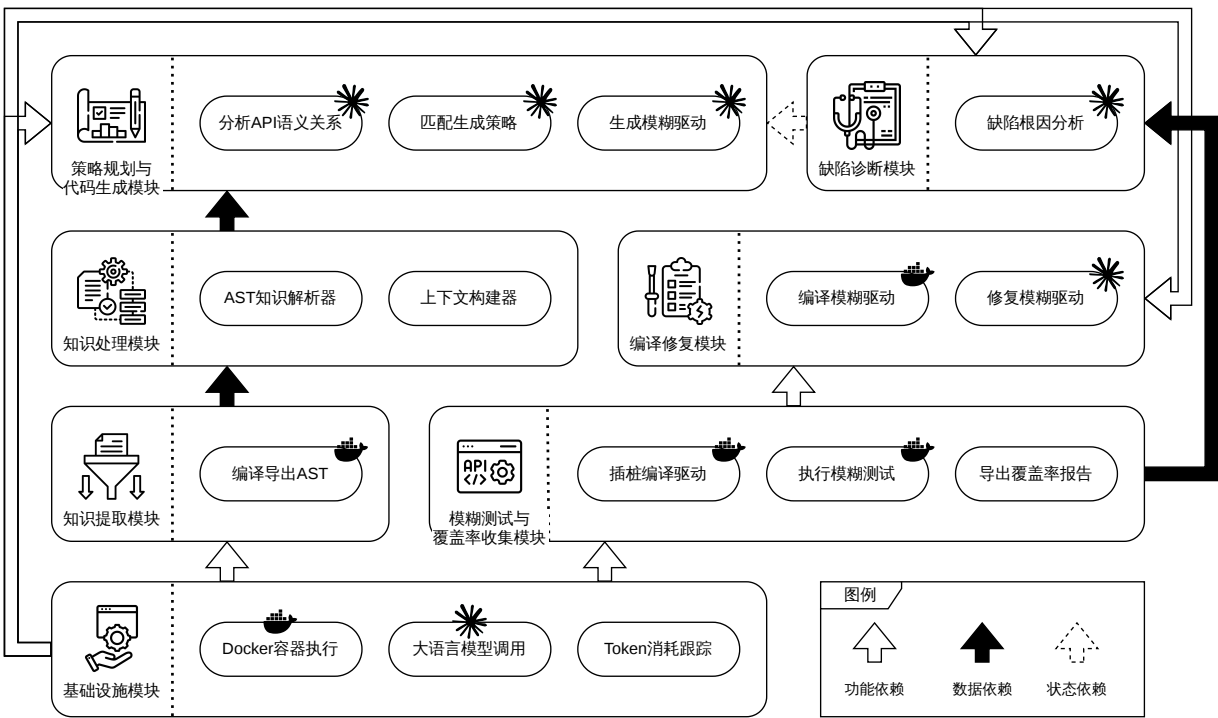


图 3-3 MAGIC4FDG 系统整体架构图

架构自底向上分层，共包含七个功能模块，具体包括：基础设施、知识处理、知识提取、策略规划与代码生成、编译修复、模糊测试与覆盖率收集、缺口诊断。

基础设施模块是底座，提供 Docker 执行、LLM 调用和令牌追踪三项服务，其他模块都依赖它。知识处理模块解析 clang AST 输出，将原始的语法树信息转换为各智能体能直接使用的文本片段。知识提取模块在 Docker 里编译目标库、

做 AST 导出，产出全局知识库。策略规划与代码生成模块是核心，负责分析 API 语义、匹配策略、并行生成多个风格不同的驱动。编译修复模块负责将编译失败的变体修复。模糊测试与覆盖率收集模块执行测试、收集数据。缺口诊断模块分析哪里没覆盖到、为什么没覆盖到，输出约束供下一轮使用。各模块的详细设计在下一小节展开。

3.3.2 系统模块划分

在需求分析的基础上做模块化划分，可以有效降低系统复杂度，让结构更清晰。MAGIC4FDG 按功能职责分为七个模块，下面逐一介绍，最后解释模块间的依赖关系。

基础设施模块。该模块并不直接对应某个功能性需求或用例，但它是其他所有模块赖以运行的技术底座。内部包含三个核心组件。其一，**Docker 执行器**，封装知识提取、编译、模糊测试、覆盖率收集四类容器化操作的命令构建与执行逻辑，同时负责构建缓存的创建与挂载管理，对应 QA4（隔离性）和 PR2（构建缓存）两项非功能性需求。其二，**LLM 工厂**，以统一接口创建带重试机制的模型客户端实例，集中管理配置参数、超时策略和错误处理。其三，**令牌追踪器**，按智能体类型分类记录每次模型调用的令牌消耗，供成本监控和实验分析使用。

知识处理模块。对应 R1 中的知识组织部分。三个组件各负责一项工作：提取器负责解析 clang AST 的 JSON 输出，从中提取函数签名、参数类型、调用图和宏常量；上下文构建器根据“谁要用”来决定“给什么”——代码生成智能体需要看目标 API 的完整签名和调用关系，Analyst 需要看未覆盖代码行的上下文，同一份知识给不同智能体的视图是不一样的；分组引擎做基于调用图的 API 分组和规则引擎策略匹配，当 LLM 策略规划失败时作为确定性回退。

知识提取模块。对应 R1 和 UC2。功能较为明确，即在 Docker 容器内编译目标库，用 clang 前端做 AST 导出，结果交给知识处理模块去解析。构建产物会缓存到宿主机，后续的编译和模糊测试容器只读挂载即可，无需每轮重新构建。该模块仅执行一次。

策略规划与代码生成模块。对应 R2、UC3 和 UC4，是整个系统的核心。探索阶段加载十种策略的元数据，让 LLM 分析 API 之间的语义关系，给每个槽位匹配策略；然后用差异化的提示词并行生成多个驱动。利用阶段则根据覆盖率

停滞情况动态调整策略，将 **Analyst** 的约束和历史经验注入提示词做增量改进。

编译修复模块。对应 R3 和 UC5。对每个变体在 **Docker** 容器内做带 **ASan** 插桩的编译和静态链接。编译通过则标记成功；未通过的提取错误信息由 **LLM** 修复，最多三次。该模块的存在让系统对 **LLM** 代码生成质量有容错能力，保证每轮有足够变体进入模糊测试。

模糊测试与覆盖率收集模块。对应 R4、R5、UC6 和 UC7。对每个编译通过的变体，在 **Docker** 容器内依次做：覆盖率插桩重编译、合并语料库、fork 模式模糊测试、语料库重放收集完整覆盖率、**LLVM** 工具链导出行级与分支级报告。还做控制流图可达性分析过滤不可达代码行，并维护各槽位的最佳版本记录和停滞计数。

缺口诊断模块。对应 R6 和 UC8。对每个活跃槽位，读取最佳代码和未覆盖代码行列表，把覆盖率数据与 **API** 知识注入分析提示模板让 **LLM** 做根因分析。输出结构化的约束列表和未覆盖代码簇，追加到对应槽位的隔离知识空间，跨轮次累积，为后续的策略规划与代码生成提供精确的改进指导。

模块之间的依赖关系。图3-3的架构是基于需求分析得到的功能性需求和用例做模块化划分的结果，模块间存在数据依赖、功能依赖和状态依赖三类关系。整体上看，系统采用分层架构，上层依赖下层，下层不依赖上层，符合依赖倒置原则，有效降低了耦合度。

具体来说：知识提取模块对基础设施模块是功能依赖（调用 **Docker** 执行器在容器内完成构建和 **AST** 导出），对知识处理模块是数据依赖（调用提取器解析 **AST JSON** 并构建知识库）。策略规划与代码生成模块、缺口诊断模块对知识处理模块是数据依赖，它们需要知识处理模块提供的结构化 **API** 知识和格式化上下文文本。所有涉及 **LLM** 调用的模块对基础设施模块是功能依赖，基础设施模块提供 **LLM** 调用服务和 **Docker** 执行服务。编译修复模块和模糊测试与覆盖率收集模块对基础设施模块同样是功能依赖，都通过 **Docker** 执行器在容器内完成各自的工作。缺口诊断模块与策略规划与代码生成模块之间是状态依赖，**Analyst** 的输出（约束列表和累积知识）通过共享的流水线状态对象传递给后者，后者在后续轮次中据此调整生成策略和提示词内容。

3.3.3 系统逻辑视图

逻辑视图关注功能性需求，描述系统的主要组件、类和它们之间的关系，通常用类图表示。图3-4是 MAGIC4FDG 的逻辑视图。

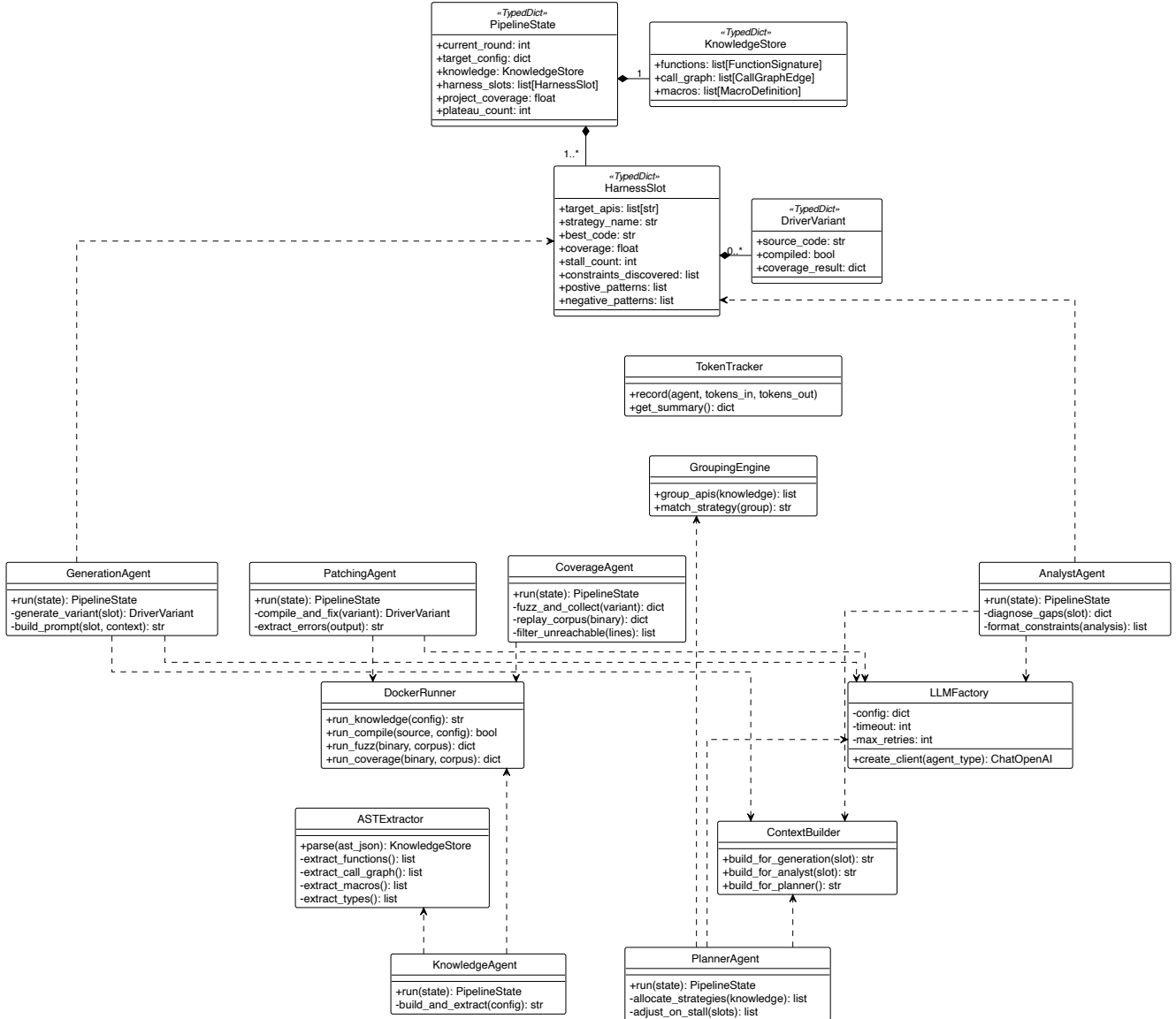


图 3-4 MAGIC4FDG 系统逻辑视图

这个逻辑视图用概念类图展示系统的静态结构。在看各模块的类之前，先说明全局状态，这是理解整个系统数据流的关键。MAGIC4FDG 用共享状态架构，模块之间靠一个统一的状态对象传数据。这个状态由四个类型支撑：**PipelineState** 是最外层的容器，**TypedDict** 定义，包含当前轮次、目标库配置、知识库、槽位列表等信息，所有模块都通过它交换信息；**HarnessSlot** 代表一个独立的驱动槽位，里面有目标 **API** 列表、策略名、最佳代码、覆盖率、停滞计数和累积知识，这是

整个系统中最核心的数据结构，因为多驱动并行迭代全靠它；`DriverVariant` 相对简单，表示单轮生成的一个变体，包含源码、编译状态和覆盖率；`KnowledgeStore` 存储全局知识，包括函数签名、调用图、宏定义等。

下面按 3.3.2 的模块划分，分别介绍各模块的核心类。

基础设施模块包含 `DockerRunner`、`LLMFactory` 和 `TokenTracker` 三个类。`DockerRunner` 封装四类 `Docker` 操作（知识提取、编译、模糊测试、覆盖率收集），每类对应一个方法，内部负责命令构建、卷挂载管理和容器输出解析。`LLMFactory` 用工厂模式根据配置创建带重试和超时控制的 `ChatOpenAI` 客户端。`TokenTracker` 通过回调记录每次模型调用的令牌数，按智能体类型分类统计。

知识处理模块包含 `ASTExtractor`、`ContextBuilder` 和 `GroupingEngine` 三个类。`ASTExtractor` 解析 `clang` 导出的 `AST JSON`，提取 `FunctionSignature`、`CallGraphEdge` 和 `MacroDefinition` 等结构化对象。`ContextBuilder` 根据调用方智能体的类型把原始知识格式化为不同粒度的文本片段。`GroupingEngine` 基于调用图连通分量做 `API` 分组，并通过规则引擎把策略元数据的适用条件与 `API` 分组特征匹配。

知识提取模块的核心类是 `KnowledgeAgent`，封装了 `Docker` 容器内目标库构建和 `AST` 导出的完整流程，调用 `DockerRunner` 执行容器操作，把结果写入全局状态的 `KnowledgeStore`。

策略规划与代码生成模块里有两个核心类：`PlannerAgent` 和 `GenerationAgent`。前者负责加载策略库元数据、让 `LLM` 分析 `API` 语义然后做策略匹配，最终给出每个槽位该用什么策略。后者拿到分配方案后组装提示词，用线程池并行调 `LLM`，一次生成多个变体。

编译修复模块的核心类是 `PatchingAgent`。逻辑较为清晰，在 `Docker` 里编译，通过则标记成功，未通过则将错误信息提供给 `LLM` 修复，最多三次机会。

模糊测试与覆盖率收集模块的 `CoverageAgent` 较为复杂，它将模糊测试执行、语料库重放、覆盖率导出和 `CFG` 可达性分析串联在一起，同样通过线程池并行处理多个变体。

缺口诊断模块的 `AnalystAgent` 读各槽位的覆盖率数据和未覆盖行列表，调 `LLM` 做根因分析，输出约束和未覆盖代码簇，然后追加到对应槽位的知识空间里。

3.3.4 系统开发视图

开发视图关注代码的模块化结构和组织方式，描述代码如何被分解为模块、组件和包。图3-5是 MAGIC4FDG 的开发视图。

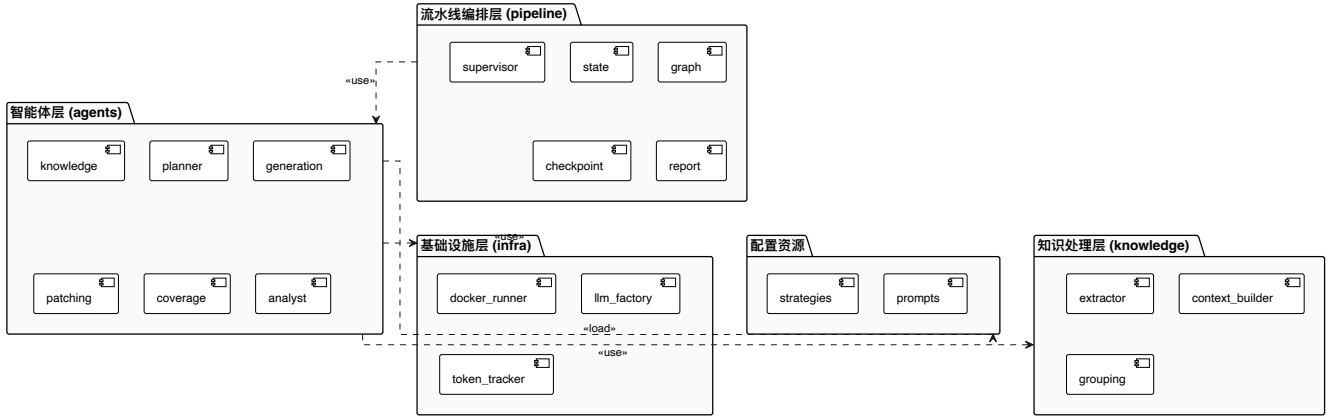


图 3-5 MAGIC4FDG 系统开发视图

这个开发视图用包图展示了代码组织结构，面向开发人员设计。它遵循分层原则，自底向上分为四个代码层次：基础设施层、知识处理层、智能体层和流水线编排层。需要说明的是，代码层次划分是从软件工程的代码组织角度出发的，与 3.3.2 中按业务功能划分的七个模块存在映射关系：基础设施层对应基础设施模块，知识处理层对应知识处理模块，智能体层包含知识提取、策略规划与代码生成、编译修复、模糊测试与覆盖率收集、缺口诊断五个模块的实现，流水线编排层负责全局状态定义、DAG 图构建、条件路由和检查点管理。

基础设施层 (infra) 包含 docker_runner、llm_factory 和 token_tracker 三个包，分别封装 Docker 操作、LLM 客户端创建和令牌统计。

知识处理层 (knowledge) 包含 extractor、context_builder 和 grouping 三个包，分别负责 AST JSON 解析、按需格式化提示词文本、以及基于调用图的 API 分组和规则引擎策略匹配。

智能体层 (agents) 包含 knowledge、planner、generation、patching、coverage 和 analyst 六个包，每个包对应一个专职智能体，实现独立的节点函数，内部封装了提示词加载、LLM 调用和结果解析的完整逻辑。

流水线编排层 (pipeline) 包含 state、graph、supervisor、checkpoint 和 report 五个包。state 定义全局状态的 TypedDict 结构；graph 构建 LangGraph 状态图并注册节点和条件路由；supervisor 提供命令行入口和参数解析；checkpoint 实现状

态的 JSON 序列化与反序列化；report 生成 Markdown 和 JSON 格式的实验报告。

另外还有两个辅助目录：strategies 目录存储十种策略的 Markdown 文件（含 YAML 前置元数据），prompts 目录存储各智能体的提示词模板。这两个目录作为配置资源，被智能体层在运行时动态加载。

3.3.5 系统进程视图

进程视图关注系统运行时的行为，描述并发性、进程间通信和同步机制。

图3-6是 MAGIC4FDG 的进程视图。

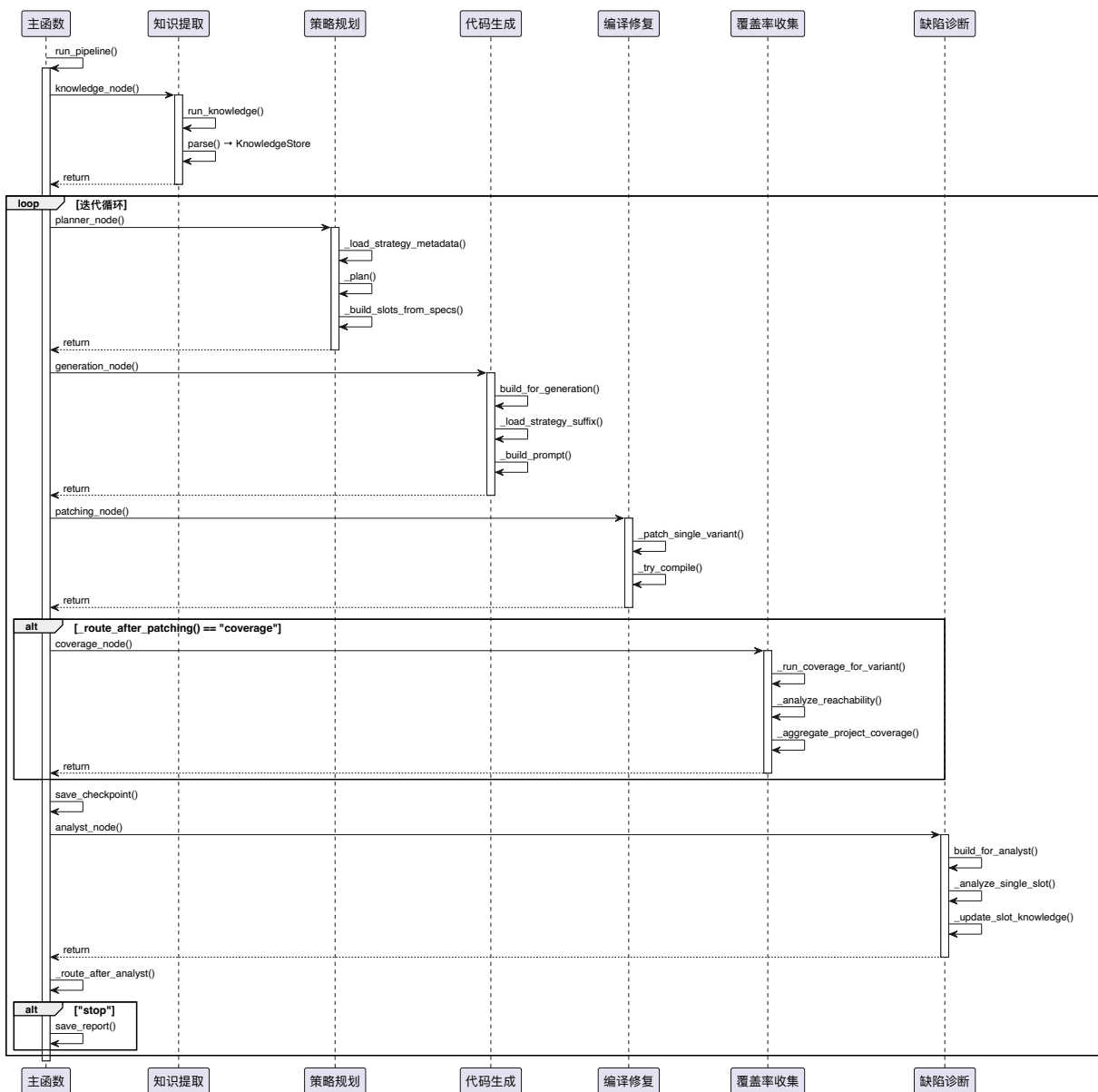


图 3-6 MAGIC4FDG 系统进程视图

这个进程视图用顺序图展示系统运行时各组件的交互过程。重点展示事件触发和数据流转。系统共有八个组件在主函数的调度下协同工作：知识提取、知识处理、策略规划、代码生成、编译修复、覆盖率收集、状态持久化、缺口诊断。

主函数调用 `run_pipeline()` 启动流水线后，先调用知识提取组件的 `knowledge_node()`。知识提取组件内部调用 `run_knowledge()` 在 Docker 容器内编译目标库并做 AST 导出，再调用知识处理组件的 `parse()` 解析 AST JSON，把结构化知识存入 `KnowledgeStore`，构建产物缓存到宿主机。这一步只在首次运行时执行，后续通过检查缓存是否存在来决定是否跳过。

知识提取完成后进入迭代循环。每轮迭代中，主函数先调用策略规划组件的 `planner_node()`。该组件依次调用 `_load_strategy_metadata()` 加载策略库元数据，首轮调用 `_plan_initial()` 通过 LLM 做 API 语义分析和策略匹配，后续轮次调用 `_plan_iterate()` 根据停滞计数调整策略，最后调用 `_build_slots_from_specs()` 构建槽位。

然后主函数调用代码生成组件的 `generation_node()`。该组件先调用知识处理组件的 `build_for_generation()` 获取格式化的 API 知识上下文，再调用 `_load_strategy_suffix()` 加载策略后缀，最后根据轮次调用 `_build_fresh_prompt()` 或 `_build_incremental_prompt()` 组装提示词并调用 LLM 生成代码。

接着主函数调用编译修复组件的 `patching_node()`。该组件通过线程池对每个变体并行调用 `_patch_single_variant()`，内部调用 `_try_compile()` 在 Docker 容器内编译，失败的提取错误让 LLM 修复，最多三次。

编译修复完成后，主函数调用 `_route_after_patching()` 做条件路由。有编译通过的变体就调用覆盖率收集组件的 `coverage_node()`，该组件通过线程池并行调用 `_run_coverage_for_variant()` 执行模糊测试和覆盖率收集，再调用 `_analyze_reachability()` 做 CFG 可达性分析，最后调用 `_aggregate_project_coverage()` 算项目级覆盖率。全部编译失败则跳过直接进入下一步。

之后主函数调用状态持久化组件的 `checkpoint_node()`，内部调用 `save_checkpoint()` 把流水线状态序列化为 JSON 检查点文件，同时保存各变体源码。

最后主函数调用缺口诊断组件的 `analyst_node()`。该组件先调用知识处理组件的 `build_for_analyst()` 获取未覆盖代码行的上下文，再通过线程池对每个活跃槽位并行调用 `_analyze_single_slot()` 做根因分析，最后调用 `_update_`

slot_knowledge() 把结果追加到对应槽位的知识空间。

每轮迭代结束时，主函数调用_route_after_analyst() 检查终止条件（覆盖率达标、轮次上限、全部收敛、项目级停滞）。满足任一条件就调用 save_report() 生成报告并终止；否则进入下一轮。

3.3.6 系统物理视图

物理视图关注部署架构，描述软件如何映射到硬件资源。图3-7是 MAGIC4FDG 的物理视图。

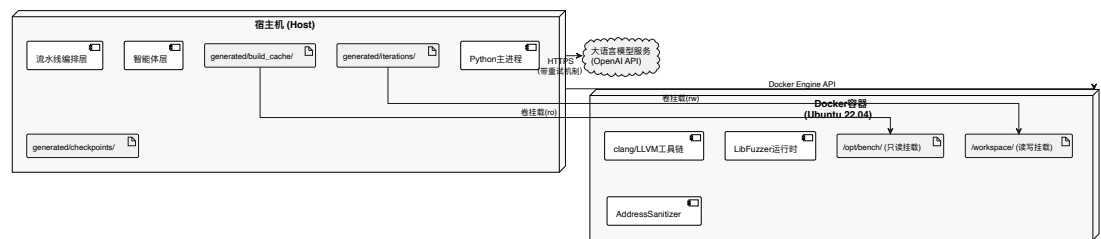


图 3-7 MAGIC4FDG 系统物理视图

部署结构分三块看：宿主机、Docker 容器、LLM 服务。

宿主机运行的是 Python 主进程，负责流水线编排和智能体调度。计算密集型操作（构建目标库、编译驱动、执行模糊测试、收集覆盖率）全部在 Docker 容器内完成。这一设计基于实际考虑——macOS 上 clang 的行为与 Linux 不完全一致，直接在宿主机编译容易出现链接兼容性问题，因此统一使用 Ubuntu 22.04 容器，预装好 clang/LLVM 工具链和 LibFuzzer。宿主机与容器之间通过卷挂载交换数据，构建缓存挂载为只读（/opt/bench），驱动源码和语料库通过读写卷双向传递。

LLM 服务通过 HTTPS 调用，策略规划、代码生成、编译修复、缺口诊断四个环节都需要使用。本系统使用兼容 OpenAI API 格式的端点，配置文件中指定地址和密钥。考虑到网络波动，系统采用指数退避重试机制，实测中偶有超时但不会导致流水线中断。

3.4 本章小结

这一章完成了需求分析和概要设计，为后续的详细设计和编码实现提供了依据。

回顾一下本章内容。开头用流程图将系统运行的四个阶段阐述清楚，即知识准备、驱动生成与编译修复、模糊测试与覆盖率收集、缺口诊断与迭代控制，每个阶段中哪些智能体参与、如何配合都做了说明。

需求分析部分，本章梳理了三类涉众各自关心什么，再从中提炼出 8 项功能性需求和 9 项非功能性需求。用例分析画了用例图，选取了 10 个关键用例做了详细描述。用例描述较为详细，目的是为后续实现提供明确的验收标准。

概要设计部分，本章将系统划分为七个功能模块，分析了它们之间的数据依赖、功能依赖和状态依赖。然后用“4+1”视图从不同角度描述架构：逻辑视图描述类和依赖关系，开发视图描述代码组织与分层，进程视图描述运行时调用顺序和并行点，物理视图描述部署结构及三者间的通信方式。场景视图即前述用例图，此处不再赘述。

第四章 MAGIC4FDG 系统的详细设计与实现

第三章完成了需求分析和概要设计，本章在此基础上对核心模块进行详细设计和实现展示。本章将逐一剖析知识提取模块、策略规划与代码生成模块、模糊测试与覆盖率收集模块、缺口诊断模块的内部结构和实现机制。每节先用类图和顺序图说明设计思路，再结合关键代码片段展示核心功能的具体实现。

4.1 知识提取模块详细设计与实现

知识提取模块是整个系统的基础，没有它提供的结构化 API 知识，后面的智能体就无从下手。它的职责是在 Docker 容器内编译目标库并做 clang AST 导出，从中提取结构化的 API 知识，再根据不同智能体的需求把知识格式化为适合注入提示词的文本片段。下面先通过类图和顺序图说明设计思路，再深入解析 AST 解析与知识构建、差异化上下文构建两个核心功能的实现。

4.1.1 知识提取模块详细设计

图4-1是知识提取模块的详细设计类图，展示了关键类及其属性和方法。

模块入口为 `extract_knowledge` 函数，接收目标库配置后依次执行 AST 解析、宏提取、调用图构建、语义标注、注释附加五个阶段，最终产出 `KnowledgeStore` 对象。`KnowledgeStore` 在设计上充当全局知识容器，内含四个字段。其中 `api_entries` 存放全部公开 API 条目，每条对应一个 `APIEntry` 实例，记录函数名、签名、返回类型、参数列表等基本信息，同时包含功能类别、前置条件和文档注释。`call_graph` 用邻接表存储函数间的调用关系。其余两个字段分别管理类型定义（结构体、枚举、类型别名）和宏常量，宏常量还按标志位、普通常量、函数式宏做了分类，方便后续生成智能体按需取用。

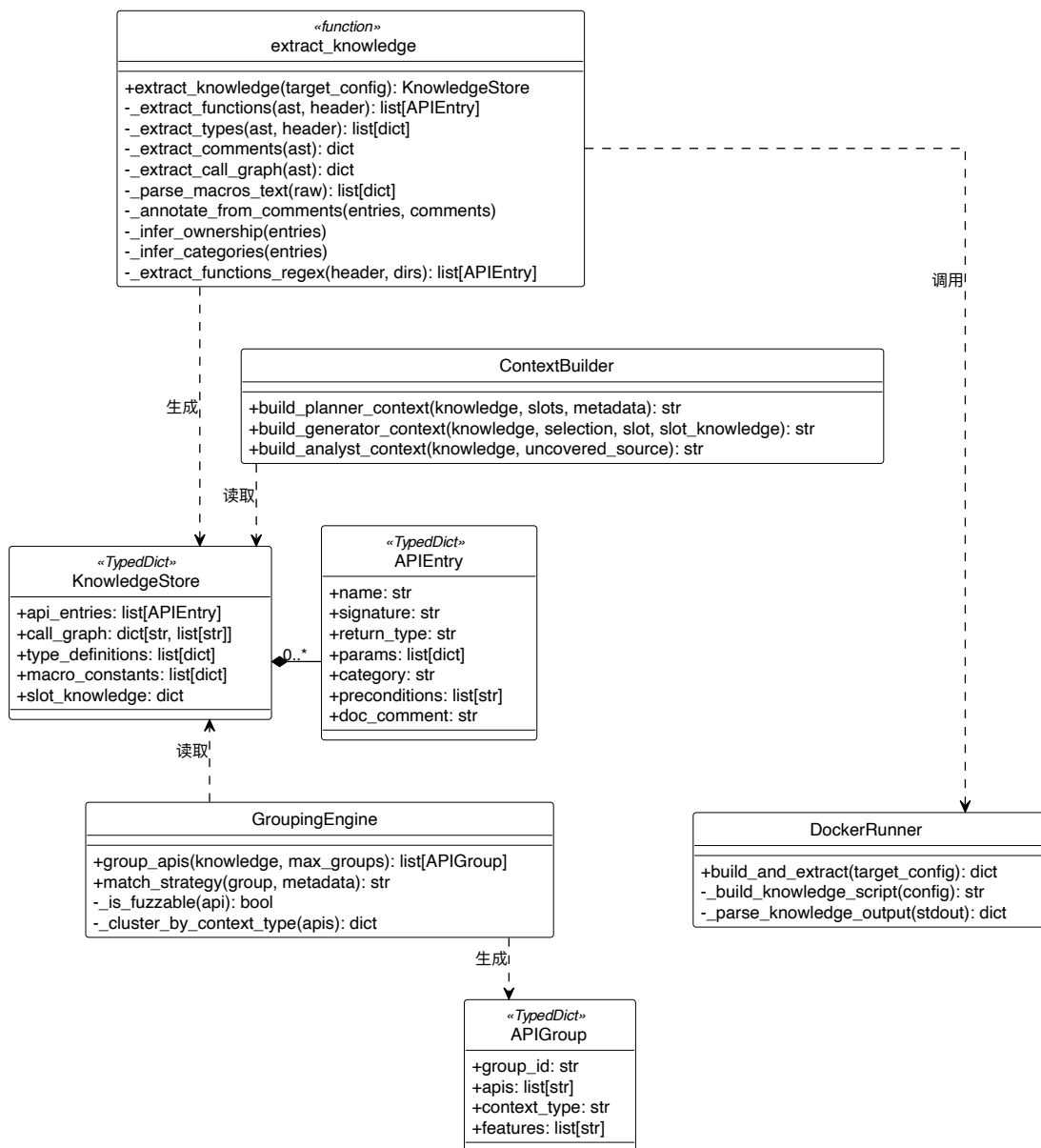


图 4-1 知识提取模块详细设计类图

`ContextBuilder` 类是知识处理的核心组件，负责把原始 `KnowledgeStore` 转换为适合注入提示词的文本片段。它提供三个方法：`build_planner_context` 为 Planner 生成 API 分类概览和调用图摘要；`build_generator_context` 为 Generation 生成目标 API 完整签名、参数语义和类型定义的详细上下文；`build_analyst_context` 为 Analyst 生成完整 API 清单和未覆盖源码行的诊断上下文。三种上下文的信息粒度和侧重点各不相同，这就是差异化上下文注入的体现。

`GroupingEngine` 类实现确定性的 API 分组和策略匹配，作为 LLM 策略规划失败时的回退方案。`group_apis` 方法基于调用图连通分量和参数类型相似性做

API 聚类，match_strategy 方法用规则引擎把分组特征与策略元数据的适用条件匹配。

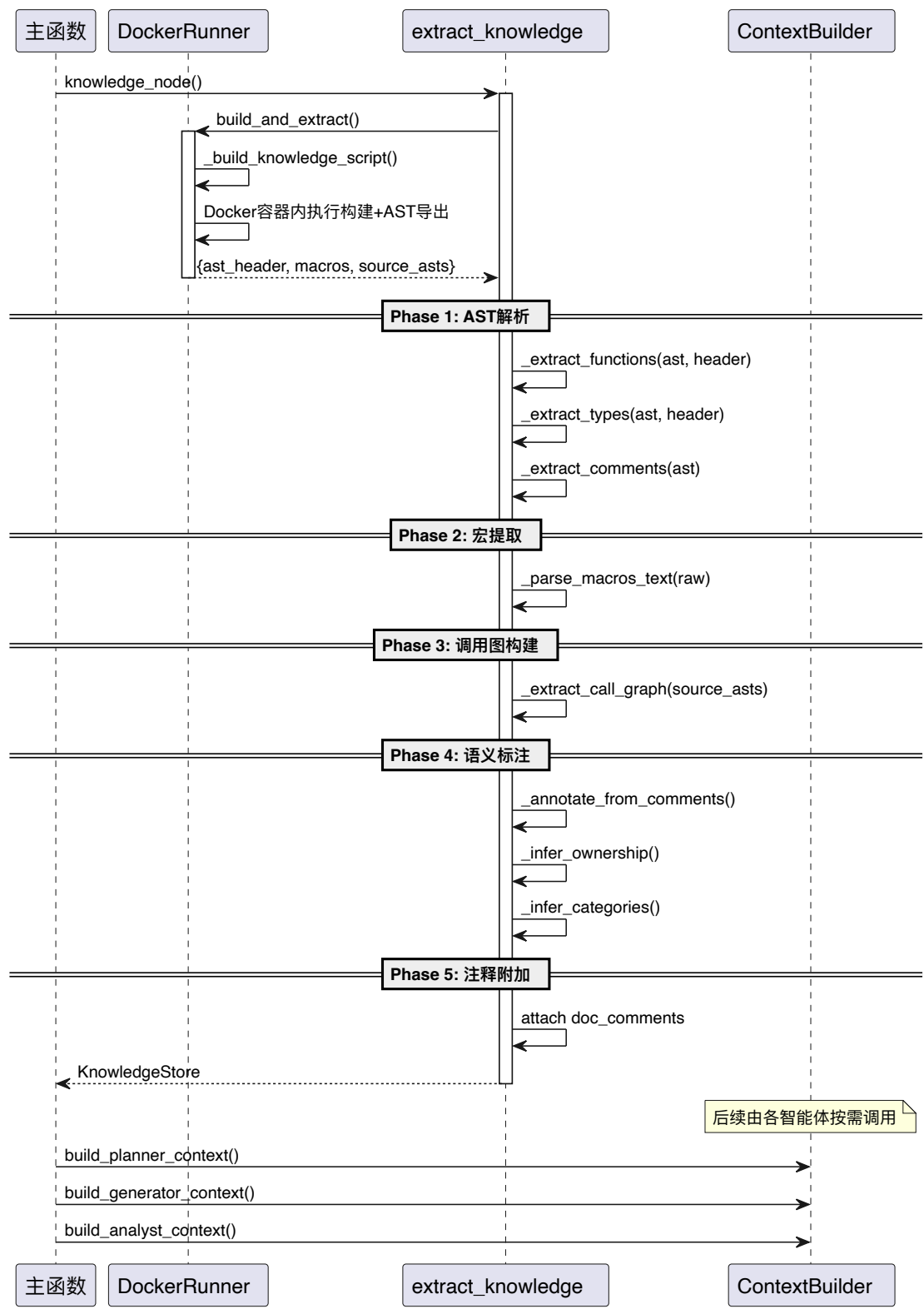


图 4-2 知识提取模块详细设计顺序图

图4-2是知识提取模块的顺序图，描绘了各功能类之间的协作时序。

流程从主函数调用 `knowledge_node` 开始。该函数读取流水线状态中的目标库配置，随即委托 `extract_knowledge` 完成实际工作。后者首先通过 `Docker-Runner` 的 `build_and_extract` 方法，在 `Docker` 容器内完成四项操作：目标库编译、头文件 `AST` 导出、预处理器宏导出、源文件 `AST` 导出。容器执行结果经标准输出的分隔符协议传回宿主机。

获取原始数据后进入五阶段处理流水线：第一阶段调用 `_extract_functions` 解析头文件 `AST JSON`，遍历 `FunctionDecl` 节点提取公开 `API` 签名；第二阶段调用 `_parse_macros_text` 解析预处理器输出，提取并分类宏常量；第三阶段对每个源文件 `AST` 调用 `_extract_call_graph`，该步骤的核心逻辑在于递归查找 `CallExpr` 节点、将调用关系填入邻接表。第四阶段较为复杂，需要连续执行三个语义标注方法：`_annotate_from_comments`、`_infer_ownership`、`_infer_categories`，从注释和命名规则中推断所有权语义、前置条件和功能类别。第五阶段将文档注释附加到对应 `API` 条目。如果 `AST` 解析阶段未能提取到任何函数声明，系统自动回退到基于正则表达式的头文件扫描方案，保证知识提取不会完全失败。

4.1.2 AST 解析与知识构建功能实现

这一小节聚焦知识提取模块中 `AST` 解析与知识构建的实现细节。本节依次说明 `clang` `AST` 深度优先遍历算法、函数签名提取、调用图构建和语义标注这四个功能的具体做法。

AST 深度优先遍历

`clang` 以 `JSON` 格式导出的抽象语法树本质上是一棵嵌套树，每个节点通过 `inner` 字段持有子节点列表。本模块实现了一个通用的深度优先遍历生成器 `_walk_ast`，它是后续所有 `AST` 分析功能的基础。这里有一个实现细节：生成器采用 `Python` 的 `yield from` 语法做惰性遍历，不会一次性把整棵 `AST` 展开到内存里。对于大型库（如 `OpenSSL`，`AST JSON` 可达数百 `MB`），这一点至关重要。该遍历器的实现如图4-3所示。

```

1 def _walk_ast(node: dict):
2     if not node:
3         return
4     yield node
5     for child in node.get("inner", []):
6         if child:
7             yield from _walk_ast(child)

```

图 4-3 AST 深度优先遍历生成器

函数签名提取

函数签名提取是知识构建中最关键的一步，目标在于从 AST 中定位所有公开 API 的函数声明，并提取函数名、完整类型签名、返回类型及参数列表。核心实现如图4-4所示。

实现中采用了逐层收窄的过滤策略。首先，通过 `kind` 字段筛出 `FunctionDecl` 节点，排除变量声明和类型定义。接着检查 `loc` 字段中的文件路径，只有声明在目标头文件中的函数才予以保留，系统头文件和第三方依赖引入的声明在此被剔除。最后一道过滤针对函数名前缀：以下划线开头的内部实现函数不进入结果集。通过这三道过滤的函数声明，系统从 `type.qualType` 字段取出完整函数类型签名，并调用 `_extract_params` 方法遍历子节点中的 `ParmVarDecl` 节点，逐一提取参数名、类型、是否可空和所有权语义。

当 AST 解析未能提取到任何函数声明时（比如目标库使用了 clang 无法处理的非标准扩展），系统会自动回退到基于正则表达式的头文件扫描方案。这个回退方案用多行正则匹配 C 函数声明的标准模式。精度虽然不如 AST 解析，但能保证知识提取流程不会因为单一工具链的局限而彻底失败。

```

1 def _extract_functions(ast: dict, header_path: str) -> list[APIEntry]:
2     entries: list[APIEntry] = []
3     header_name = Path(header_path).name if header_path else ""
4     for node in _walk_ast(ast):
5         if node.get("kind") != "FunctionDecl":
6             continue
7         loc = node.get("loc", {})
8         file_field = (loc.get("file", "") or
9                     loc.get("expansionLoc", {}).get("file", ""))
10        if file_field and header_name and header_name not in file_field:
11            continue
12        name = node.get("name", "")
13        if not name or name.startswith("_"):
14            continue
15        qual_type = node.get("type", {}).get("qualType", "")
16        params = _extract_params(node)
17        return_type = _infer_return_type(qual_type)
18        entries.append(APIEntry(
19            name=name, signature=qual_type,
20            return_type=return_type, params=params,
21            category="utility", preconditions=[],
22            group="", doc_comment="",
23        ))
24    return entries

```

图 4-4 基于 AST 的函数签名提取

调用图构建

调用图记录了函数之间的调用关系，是策略规划智能体做 API 分组和场景设计时的重要依据。系统通过递归遍历源文件 AST 中的 `CallExpr` 节点来构建调用图，核心实现如图4-5所示。

算法维护一个 `current_func` 状态变量，遍历过程中持续跟踪当前所处的函数作用域。遇到 `FunctionDecl` 节点时更新当前函数名；遇到 `CallExpr` 节点时通过 `_resolve_callee` 解析被调用函数的名称，把调用关系记录到邻接表中。值得说明的是，`_resolve_callee` 方法需要处理两种常见的 AST 表示：直接的 `DeclRefExpr` 引用，以及经过 `ImplicitCastExpr` 隐式转换的间接引用。系统对所有源文件的调用图做合并和去重，最终生成全局的函数调用关系图。

```

1 def _extract_calls_recursive(node: dict, current_func: str,
2                               graph: dict) -> None:
3     kind = node.get("kind", "")
4     if kind == "FunctionDecl":
5         current_func = node.get("name", "")
6     elif kind == "CallExpr" and current_func:
7         for child in node.get("inner", []):
8             callee = _resolve_callee(child)
9             if callee:
10                graph.setdefault(current_func, []).append(callee)
11                break
12     for child in node.get("inner", []):
13         if child:
14             _extract_calls_recursive(child, current_func, graph)

```

图 4-5 调用图递归构建

语义标注

语义标注阶段要解决的问题是，光有函数签名还不够，代码生成智能体需要知道如何正确调用这些 API。因此本模块从文档注释和函数命名规则两个渠道推断所有权语义、前置条件和功能类别。具体分为三个独立的标注方法。

`_annotate_from_comments` 方法的思路比较直接，用预定义的正则模式集扫描文档注释。系统预定义了三类正则模式集。一类匹配返回值所有权描述，如“caller is responsible to free”；一类匹配参数所有权转移，如“takes ownership”；还有一类匹配前置条件，如“must not be NULL”。注释文本一旦命中某个模式，对应 API 条目即被打上相应标记。这种模式匹配方案精度有限，但对文档规范的库而言效果尚可。

`_infer_ownership` 方法走的是另一条路径——从函数名本身推断语义。具体而言，名字中包含 Create、New、Alloc 等关键词的函数，其返回值被标记为调用者负责释放；名字中包含 Delete、Free、Destroy 的函数，其指针参数被标记为所有权转移。实现如图4-6所示。

`_infer_categories` 方法把每个 API 归入六个功能类别：解析、创建、销毁、修改、查询、序列化。分类逻辑是按优先级匹配函数名中的关键词，首个命中的类别就是最终结果。这个分类信息后面策略规划智能体会用到，举个例子，系统会把解析类和序列化类 API 配对来构建往返测试场景。

```

1 def _infer_ownership(entries: list[APIEntry]) -> None:
2     create_pattern = re.compile(
3         r"(Create|New|Duplicate|Copy|Clone|Alloc)", re.I)
4     delete_pattern = re.compile(
5         r"(Delete|Free|Destroy|Release|Close)", re.I)
6     for entry in entries:
7         name = entry["name"]
8         if create_pattern.search(name) and "*" in entry["return_type"]:
9             precondition = "Return value must be freed by caller"
10            if precondition not in entry["preconditions"]:
11                entry["preconditions"].append(precondition)
12            if delete_pattern.search(name):
13                for p in entry["params"]:
14                    if "*" in p["type"] and p["ownership"] == "borrow":
15                        p["ownership"] = "transfer"
16                break

```

图 4-6 基于命名规则的所有权推断

4.1.3 差异化上下文构建功能实现

本小节解析 `ContextBuilder` 模块的实现。它在 `KnowledgeStore` 和大语言模型提示词之间起到翻译作用，针对不同智能体的任务特点，从完整知识库中筛选相关子集并格式化为结构化 `Markdown` 文本。本模块提供三种差异化上下文视图。背后的设计原则可以概括为信息不对称：让每个智能体只看到完成任务所需的最小充分信息集，避免无关内容干扰大语言模型推理。

策略规划上下文

`build_planner_context` 方法为策略规划智能体构建宏观视角的知识上下文。核心思路是提供“广度优先”的信息，让规划智能体了解目标库的整体 `API` 分布和函数间关系，而不是单个 `API` 的实现细节。实现如图4-7所示。

输出的上下文分为四个信息层次。第一层是按功能类别分组的 `API` 概览，展示每个类别的函数数量和签名摘要，帮助规划智能体把握库的功能分布。第二层是调用图摘要，只展示公开 `API` 之间的调用关系（内部实现函数被过滤掉），帮助识别 `API` 间的依赖链。第三层是关键类型定义摘要，展示结构体字段和枚举值。第四层是当前各槽位的状态信息（仅在第二轮及之后包含），展示每个槽位的策略历史和覆盖率变化趋势，供规划智能体做决策。

```

1 def build_planner_context(knowledge: KnowledgeStore,
2                           slots: list[HarnessSlot],
3                           strategy_metadata: list[dict]) -> str:
4     lines = ["## Target Library API Surface\n"]
5     by_category: dict[str, list] = {}
6     for api in knowledge["api_entries"]:
7         by_category.setdefault(api["category"], []).append(api)
8     for cat in ["parse", "create", "modify", "query",
9                "delete", "serialize", "utility"]:
10         apis = by_category.get(cat, [])
11         if apis:
12             lines.append(f"### {cat.title()} ({len(apis)} functions)\n")
13             for api in apis:
14                 params_str = ", ".join(
15                     f"{p['type']} {p['name']}" for p in api.get("params", []))
16                 lines.append(
17                     f"- {api['return_type']} {api['name']}({params_str})`")
18     # ... 调用图 + 类型定义 + slot状态
19     return "\n".join(lines)

```

图 4-7 策略规划上下文构建

代码生成上下文

`build_generator_context` 方法为代码生成智能体构建“深度优先”的详细上下文。与规划上下文的宏观视角不同，生成上下文聚焦于规划智能体指定的目标 API 子集，提供生成正确模糊驱动代码所需的全部细节。核心实现如图4-8所示。

```

1 def build_generator_context(knowledge: KnowledgeStore,
2                             selection: StrategySelection,
3                             slot: HarnessSlot | None = None,
4                             slot_knowledge: dict | None = None) -> str:
5     lines = ["## API Knowledge\n"]
6     target_api_names = set(selection.get("target_apis", []))
7     relevant_apis = list(knowledge["api_entries"])
8     if target_api_names:
9         relevant_apis = [a for a in knowledge["api_entries"]
10                          if a["name"] in target_api_names]
11     for api_name in list(target_api_names):
12         callees = knowledge["call_graph"].get(api_name, [])
13         for callee in callees:
14             for a in knowledge["api_entries"]:
15                 if a["name"] == callee and a not in relevant_apis:
16                     relevant_apis.append(a)
17     # ... 详细API信息 + 类型定义 + 宏常量 + 累积约束
18     return "\n".join(lines)

```

图 4-8 代码生成上下文构建

这一步的关键在于三个设计决策。第一，API 范围做了聚焦：如果规划智能体指定了 `target_apis`，生成上下文就只包含目标 API 和它们在调用图中直接调用的函数，其余一概不提供。此处的设计考虑是，无关 API 信息会稀释大语言模型的注意力，少即是多。第二，信息排序做了优化：同一功能类别内，有文档注释的 API 排前面，让模型优先看到信息最丰富的接口。第三，跨轮次知识会注入进来：从该槽位隔离的 `SlotKnowledge` 中取出累积的约束、正向模式和负向模式一并注入，使大语言模型能利用历史经验，避免反复触发同一类错误。

缺口诊断上下文

`build_analyst_context` 方法为缺口诊断智能体构建以覆盖率缺口为中心的诊断上下文。设计目标是提供诊断覆盖率问题所需的完整信息链，从未覆盖的源码行出发，结合 API 清单和调用图，帮助诊断智能体推断覆盖率缺口的根因。核心实现如图4-9所示。

```
1 def build_analyst_context(knowledge: KnowledgeStore,
2                           uncovered_source: dict | None = None) -> str:
3     lines = ["## Full API Inventory\n"]
4     for api in knowledge["api_entries"]:
5         ownership_mark = ""
6         if any(p.get("ownership") == "transfer"
7               for p in api.get("params", [])):
8             ownership_mark = " [takes ownership]"
9         lines.append(
10             f"- `{api['signature']}` [{api['category']}] {ownership_mark}")
11     # 调用图 + 宏常量标志 + 未覆盖源码行
12     if uncovered_source:
13         lines.append("## Uncovered Source Lines\n")
14         for filename, line_map in list(uncovered_source.items())[:5]:
15             lines.append(f"### {filename}")
16             for line_no, code in sorted(line_map.items())[:20]:
17                 lines.append(f" {line_no}: {code.rstrip()}")
18     return "\n".join(lines)
```

图 4-9 缺口诊断上下文构建

与前两种上下文不同，诊断上下文传递完整的 API 清单而非子集。原因在于诊断智能体需要判断哪些 API 尚未被任何驱动覆盖。另一个特点是，该上下文包含未覆盖源码行的原始文本（每个文件最多展示 20 行），让诊断智能体能直接观察未覆盖代码的具体逻辑，从而给出精确的改进建议。调用图信息则帮助诊断智能体理解函数间的依赖关系，比如某个函数未被覆盖，可能是因为其

前置调用函数未被正确初始化。

表4-1总结了三种上下文视图的信息差异。

表 4-1 三种差异化上下文视图的信息对比

信息维度	规划上下文	生成上下文	诊断上下文
API 范围	全量（按类别分组）	目标子集（含被调用者）	全量（含所有权标记）
参数详情	类型摘要	完整（含 ownership）	无
调用图	公开 API 间关系	无	完整调用图
类型定义	摘要	完整（含字段）	无
宏常量	无	完整（分类展示）	仅标志位
覆盖率信息	槽位历史趋势	无	未覆盖源码行
累积知识	无	约束 + 正/负模式	无

4.2 策略规划与代码生成模块详细设计与实现

策略规划与代码生成模块是整个 MAGIC4FDG 系统中对大语言模型能力要求最高的部分。其核心任务是将前面提取出来的结构化 API 知识转化为能编译、能运行的模糊驱动代码。内部由两个智能体协作完成：规划智能体利用大语言模型的语义推理来设计测试场景、分配策略；生成智能体根据规划结果和策略库的指导调用大语言模型编写具体代码。本节先用类图和顺序图阐明模块设计，再分别展开探索阶段策略规划和多提示词策略驱动生成的实现。

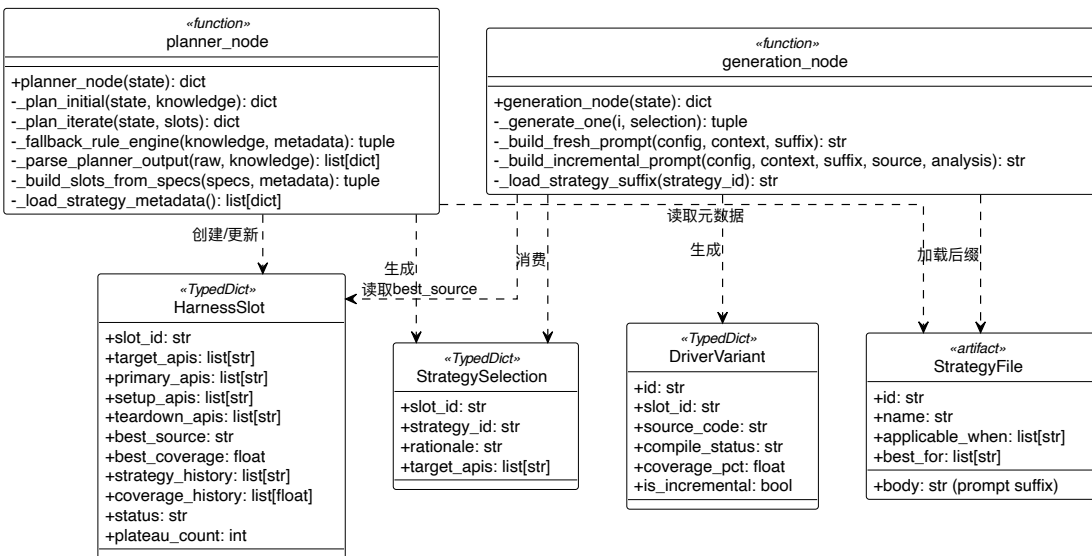


图 4-10 策略规划与代码生成模块详细设计类图

4.2.1 策略规划与代码生成模块详细设计

图4-10展示了策略规划与代码生成模块的详细设计类图。

该模块包含两个核心函数类和四个数据类型。`planner_node` 函数类封装了策略规划的全部逻辑，公开方法 `planner_node` 作为 `LangGraph` 节点入口，内部根据当前轮次分派到 `_plan_initial`（探索阶段）或 `_plan_iterate`（利用阶段）。`_plan_initial` 方法承担探索阶段的核心工作：借助大语言模型进行场景推理。具体做法是将完整的 API 知识、调用图、类型定义注入提示词模板，由 LLM 分析 API 间的语义关系并据此设计测试场景。LLM 返回的 JSON 结构先经 `_parse_planner_output` 解析，再由 `_build_slots_from_specs` 转化为具体槽位对象。为防止 LLM 调用失败导致流水线中断，此处还设置了容错保障，`_fallback_rule_engine` 可接管工作，通过 `GroupingEngine` 做确定性降级。

`generation_node` 函数类封装了代码生成的全部逻辑，通过 `ThreadPoolExecutor` 并行调用大语言模型，为每个策略选择生成一个模糊驱动变体。内部区分两种提示词构建路径：`_build_fresh_prompt` 用于首轮从零生成，`_build_incremental_prompt` 用于后续轮次的增量改进。`_load_strategy_suffix` 方法从策略文件中加载 YAML 前置元数据之后的正文部分，作为提示词的策略后缀。

`HarnessSlot` 是系统的核心状态单元，代表一个独立的模糊测试场景。每个槽位维护目标 API 集合（分为 `primary`、`setup`、`teardown` 三类）、历史最佳驱动源码、覆盖率历史、策略切换记录和停滞计数器。`StrategySelection` 记录规划智能体为每个槽位做出的策略决策，包括策略 ID、决策理由和目标 API 列表。`DriverVariant` 记录生成智能体产出的每个驱动变体的完整信息。`StrategyFile` 表示策略库中的策略文件，包含 YAML 前置元数据（适用条件、最佳场景）和 Markdown 正文（生成指导）。

图4-11展示了策略规划与代码生成模块的详细设计顺序图，可以看到探索阶段和利用阶段采用不同的执行路径。

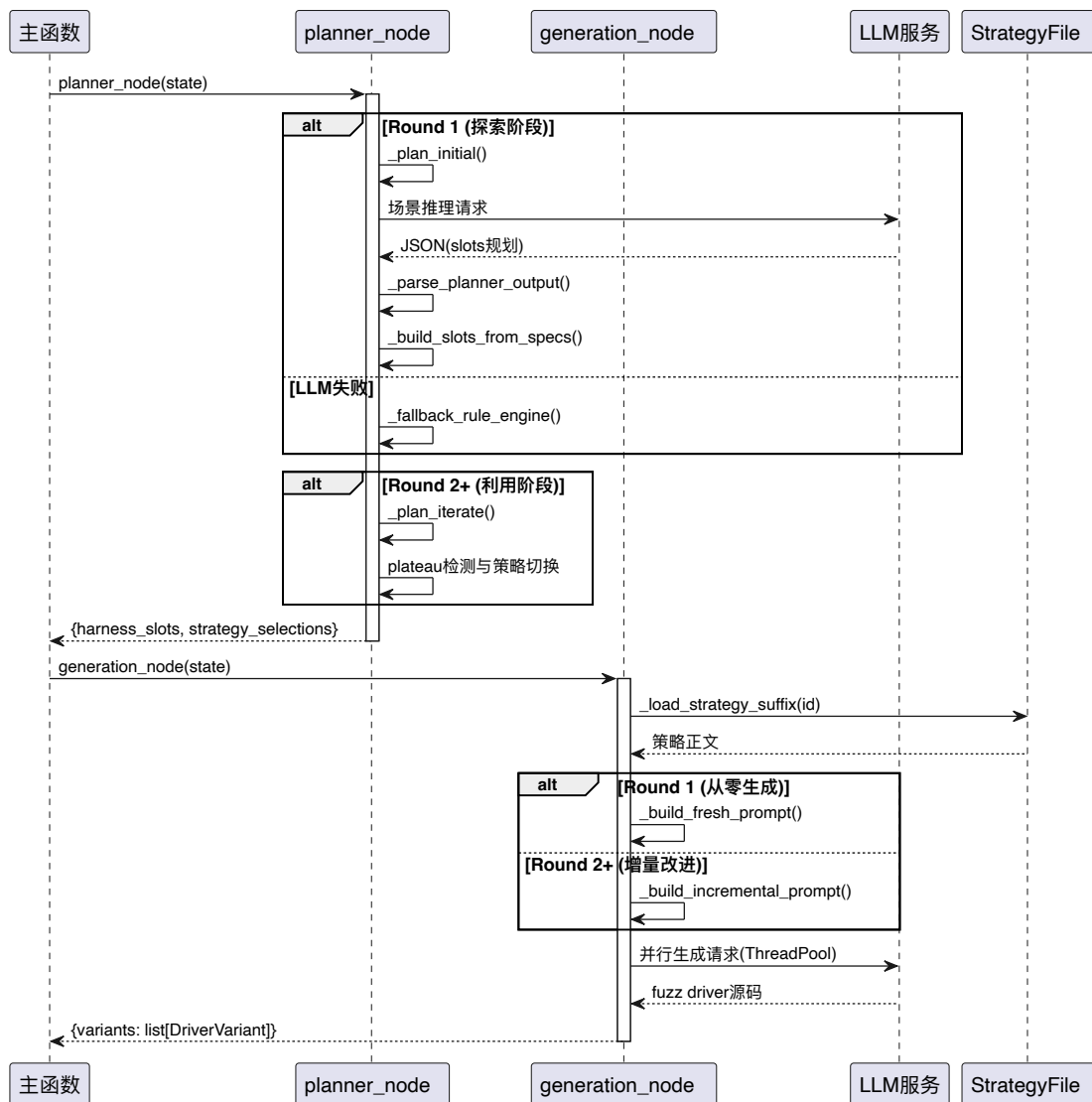


图 4-11 策略规划与代码生成模块详细设计顺序图

主函数先调用 `planner_node` 执行策略规划。探索阶段（Round 1）的做法是把完整 API 知识、调用图和类型定义全部注入提示词模板，让大语言模型做场景推理。LLM 返回一个 JSON 结构，其中包含多个槽位的规划方案。系统对其做解析校验，通过后创建 `HarnessSlot` 和 `StrategySelection` 对象。若 LLM 调用因网络超时或返回格式不合法而失败，则降级到规则引擎做确定性分组。进入利用阶段（Round 2+）后，规划逻辑有所不同：遍历所有活跃槽位，依据停滞计数器判断是否需要切换策略。系统设定的阈值是连续两轮覆盖率无提升则切换至 `targeted-expansion`；若连续四轮仍无进展，则标记该槽位已收敛，不再投入计算资源。

随后主函数调用 `generation_node` 启动代码生成。生成智能体先加载对应

的策略文件后缀，然后根据当前轮次选择提示词构建路径：首轮使用三部分组装的从零生成提示词，后续轮次使用包含现有代码和诊断约束的增量改进提示词。最终通过线程池并行调用大语言模型，为每个策略选择生成一个模糊驱动变体。

4.2.2 探索阶段策略规划功能实现

这一小节解析策略规划智能体在探索阶段（Round 1）的核心实现。本节依次说明 LLM 场景推理、输出解析校验和确定性回退机制。

LLM 场景推理

探索阶段的核心任务是将目标库的 API 知识转化为若干独立的模糊测试场景。系统将完整的 API 分类概览、调用图和类型定义注入规划提示词模板，要求大语言模型分析 API 间的语义关系，设计互不重叠的测试场景，并为每个场景选择最匹配的策略。_plan_initial 方法的实现如图4-12所示。

```
1 def _plan_initial(state: PipelineState, knowledge: dict) -> dict:
2     target_config = state["target_config"]
3     lib_name = target_config.get("library_name", "?")
4     strategy_metadata = _load_strategy_metadata()
5     context = build_planner_context(knowledge, [], strategy_metadata)
6     strategies_text = _format_strategies(strategy_metadata)
7     prompt = PLANNER_TEMPLATE.replace("{{LIBRARY_NAME}}", lib_name)
8     prompt = prompt.replace("{{KNOWLEDGE_CONTEXT}}", context)
9     prompt = prompt.replace("{{CALL_GRAPH}}", _format_call_graph(knowledge))
10    prompt = prompt.replace("{{TYPE_DEFINITIONS}}", _format_type_defs(knowledge))
11    prompt = prompt.replace("{{STRATEGIES}}", strategies_text)
12    try:
13        llm = create_llm(temperature=max(state.get("temperature", 0.4) - 0.1, 0.1))
14        response = llm.invoke([...])
15        slot_specs = _parse_planner_output(response.content, knowledge)
16        slots, selections = _build_slots_from_specs(slot_specs, strategy_metadata)
17    except Exception:
18        slots, selections = _fallback_rule_engine(knowledge, strategy_metadata)
19    return {"harness_slots": slots, "strategy_selections": selections}
```

图 4-12 探索阶段策略规划核心流程

提示词组装采用模板变量替换机制，将五个知识维度注入固定模板：库名称、API 知识上下文、调用图、类型定义和可用策略列表。值得注意的是温度参数的设置，比全局温度低 0.1（最低 0.1），目的是获得更确定性的规划输出。

输出解析与校验

大语言模型返回的规划结果为 JSON 格式，系统通过 `_parse_planner_output` 方法做严格的解析和校验。实现如图4-13所示。

```
1 def _parse_planner_output(raw: str, knowledge: dict) -> list[dict]:
2     text = raw.strip()
3     match = re.search(r"```(?:json)?\s*\n(?:.+?)```", text, re.DOTALL)
4     if match:
5         text = match.group(1).strip()
6         data = json.loads(text)
7         slots_raw = data.get("slots", [])
8         valid_apis = {a["name"] for a in knowledge.get("api_entries", [])}
9         valid_strategies = {"parse-centric", "multi-api-sequence",
10                             "roundtrip", "targeted-expansion", ...}
11     result = []
12     for spec in slots_raw:
13         primary = [a for a in spec.get("primary_apis", []) if a in valid_apis]
14         if not primary:
15             continue
16         strategy = spec.get("strategy_id", "multi-api-sequence")
17         if strategy not in valid_strategies:
18             strategy = "multi-api-sequence"
19         result.append({...})
20     return result
```

图 4-13 LLM 规划输出解析与校验

该方法的容错设计体现在多个层面。在格式层面，它兼容两种 LLM 输出形式，既能从 Markdown 代码块中提取 JSON，也能直接处理纯 JSON 文本。在语义层面，对 API 名称做有效性校验，将 LLM 幻觉产生的不存在函数名过滤掉。在策略层面，对策略 ID 做合法性检查，遇到无效策略则回退到默认的 `multi-api-sequence`。经过这些校验后，只有至少包含一个有效 `primary` API 的槽位才会被保留。

确定性回退机制

当 LLM 调用失败时（无论是网络超时、配额耗尽还是返回格式无法解析），系统降级到基于规则引擎的确定性回退方案。这保证了流水线不会因单次 LLM 失败而中断。回退机制的实现如图4-14所示。

```

1 def _fallback_rule_engine(knowledge: dict,
2                           strategy_metadata: list[dict]
3                           ) -> tuple[list[HarnessSlot], list[StrategySelection]]:
4     groups = group_apis(knowledge, max_groups=10)
5     slots: list[HarnessSlot] = []
6     selections: list[StrategySelection] = []
7     for i, group in enumerate(groups):
8         strategy_id = match_strategy(group, strategy_metadata)
9         slot_id = f"slot_{i}"
10        slots.append(HarnessSlot(
11            slot_id=slot_id, group_id=group["group_id"],
12            target_apis=group["apis"], primary_apis=group["apis"],
13            strategy_history=[strategy_id], status="active", ...
14        ))
15        selections.append(StrategySelection(
16            slot_id=slot_id, strategy_id=strategy_id,
17            target_apis=group["apis"], ...
18        ))
19    return slots, selections

```

图 4-14 确定性回退规则引擎

回退方案调用 `GroupingEngine` 的 `group_apis` 方法，基于调用图连通分量和参数类型相似性将 API 聚类为最多 10 个分组，再通过 `match_strategy` 方法基于规则评分为每个分组匹配最适合的策略。这个方案缺乏 LLM 的语义推理能力，但保证了确定性和可重复性。

4.2.3 多提示词策略驱动生成功能实现

这一小节深入代码生成智能体的实现，依次说明策略文件的组织方式、三部分提示词的拼装机制以及增量改进生成的做法。

策略文件结构

MAGIC4FDG 系统的策略库包含 10 个策略文件，每个文件采用 YAML 前置元数据加 Markdown 正文的双层结构。前置元数据定义策略的适用条件和最佳场景，供规划智能体做策略匹配；正文部分包含具体的生成指导，作为提示词后缀注入代码生成请求。表 4-2 列出了全部 10 种策略的名称、核心思路和典型适用场景。

表 4-2 MAGIC4FDG 策略库（10 种策略）

策略 ID	核心思路	典型适用场景
parse-centric	将原始模糊字节直接喂给解析 API 的各种变体	解析类库（JSON、XML、图像）
roundtrip	解析 → 修改 → 序列化 → 再解析，验证往返一致性	序列化库（JSON、XML、YAML）
multi-api-sequence	链式调用多个 API，构造有状态的调用序列	数据结构库
structure-aware	构造符合格式规范的结构化输入	输入语法复杂的解析库
stateful	跨调用维护上下文状态，测试状态机转换	加密库（基于上下文的加解密）
differential	对比同一库中不同 API 实现的行为差异	加密库（不同模式、填充方案）
targeted-expansion	基于覆盖率反馈定向扩展未覆盖区域	利用阶段的增量改进
error-path	故意构造非法输入，覆盖错误处理路径	错误处理逻辑复杂的库
callback-driven	注册回调函数并触发事件驱动的执行路径	事件驱动型库
resource-boundary	测试内存/缓冲区边界条件和资源极限	压缩/解压缩库

图4-15展示了策略文件的结构示例。

```

1 ---
2 id: parse-centric
3 name: Parse-Centric Input Fuzzing
4 applicable_when:
5   - "library has parsing functions that accept raw bytes"
6   - "API includes multiple parse variants"
7 best_for:
8   - "parser libraries (JSON, XML, image)"
9   - "format decoders"
10 ---
11 ## Strategy: Parse-Centric
12 Feed raw fuzz bytes directly to ALL parsing API variants.
13 - Call every parse function variant
14 - Use different option flag combinations
15 - Exercise error paths by feeding malformed data
16 - Use FuzzedDataProvider to split input

```

图 4-15 策略文件结构示例（parse-centric）

系统通过 `_load_strategy_suffix` 方法加载策略正文。该方法遍历策略目录下的所有 Markdown 文件，通过匹配 `id` 字段定位目标策略，然后截取 YAML 前置元数据结束标记（第二个 ‘`---`’）之后的全部内容作为策略后缀。

从零生成提示词组装

首轮生成采用三部分组装机制构建提示词：基础模板提供通用的生成框架和约束，知识上下文提供目标 API 的详细信息，策略后缀提供特定策略的生成指导。`_build_fresh_prompt` 方法的实现如图4-16所示。

```

1 def _build_fresh_prompt(target_config: dict, knowledge_context: str,
2                         strategy_suffix: str) -> str:
3     include_dirs = target_config.get("include_dirs", [])
4     prompt = (
5         GENERATION_TEMPLATE
6         .replace("{{LIBRARY_NAME}}", target_config.get("library_name", ""))
7         .replace("{{LANGUAGE}}", target_config.get("language", "C"))
8         .replace("{{HEADER}}", target_config.get("header", ""))
9         .replace("{{INCLUDE_DIRS}}", ", ".join(include_dirs))
10        .replace("{{RESEARCH_SUMMARY}}", knowledge_context)
11        .replace("{{COVERAGE_FEEDBACK}}", "(first round)")
12    )
13    if strategy_suffix:
14        prompt += "\n\n" + strategy_suffix
15    return prompt

```

图 4-16 从零生成提示词三部分组装

基础模板 `GENERATION_TEMPLATE` 对模糊驱动提出了若干通用约束：必须包含 `LLVMFuzzerTestOneInput` 入口函数，头文件引用路径正确，资源管理需满足内存安全。知识上下文由 `build_generator_context` 方法构建，涵盖目标 API 的完整签名、参数语义、类型定义和宏常量。策略后缀则给出特定测试策略的具体指导，以 `parse-centric` 策略为例，它会要求“把原始模糊字节直接喂给所有解析 API 变体”。三部分各有侧重，拼合后形成完整的生成请求。

增量改进提示词组装

第二轮及之后的迭代采用增量改进模式，基于该槽位的历史最佳驱动做修改，而非从零重新生成。这样做的好处是保留了已有的覆盖率成果，同时针对诊断智能体识别的覆盖率缺口做定向改进。`_build_incremental_prompt` 方法的实现如图4-17所示。

```

1 def _build_incremental_prompt(target_config: dict, knowledge_context: str,
2                               strategy_suffix: str, base_source: str,
3                               coverage_analysis: dict) -> str:
4     constraints_text = ""
5     if coverage_analysis:
6         constraints = coverage_analysis.get("constraints", [])
7         if constraints:
8             constraints_text = "\n".join(
9                 f"- {c.get('target', '?')}: {c.get('precondition', '')}"
10                for c in constraints[:10])
11     clusters_text = ""
12     if coverage_analysis:
13         clusters = coverage_analysis.get("uncovered_clusters", [])
14         if clusters:
15             clusters_text = "\n".join(
16                 f"- {' '.join(c.get('functions', []))}: {c.get('root_cause', '')}"
17                for c in clusters[:5])
18     prompt = f"""## Task: Improve Fuzz Driver
19                ## Current Driver (to be improved)
20                ```cpp
21                {base_source}
22                ```
23                ## Analyst Constraints
24                {constraints_text or "(no specific constraints)"}
25                ## Uncovered Code Clusters
26                {clusters_text or "(no cluster data)"}
27                ## API Knowledge
28                {knowledge_context}
29                ## Strategy Guidance
30                {strategy_suffix}"""
31     return prompt

```

图 4-17 增量改进提示词组装

增量提示词分五个层次组织信息。最上面放现有驱动源码，作为修改的基础。然后是诊断约束，最多放 10 条，每条指明需要覆盖的具体目标和前置条件。接着是未覆盖代码簇，最多 5 个，附带根因分析。再往下是 API 知识上下文和策略指导。提示词中明确指定了“保留现有驱动中有效的部分，仅添加新的代码路径”，这一步至关重要，否则大语言模型容易将已有的覆盖率成果破坏。

4.3 模糊测试与覆盖率收集模块详细设计与实现

模糊测试与覆盖率收集模块构成了 MAGIC4FDG 系统的执行层与反馈源。它负责在 Docker 容器内运行生成的模糊驱动、收集细粒度代码覆盖率数据，并借助控制流图可达性分析剔除结构性不可达的代码路径。该模块的输出直接驱动系统的迭代决策——覆盖率提升则继续优化，停滞则切换策略或终止。本节

先通过类图和顺序图阐述模块的详细设计，再分别解析 Fork 模式模糊测试与语料库重放、控制流图可达性分析两个核心功能的实现。

4.3.1 模糊测试与覆盖率收集模块详细设计

图4-18展示了模糊测试与覆盖率收集模块的详细设计类图。

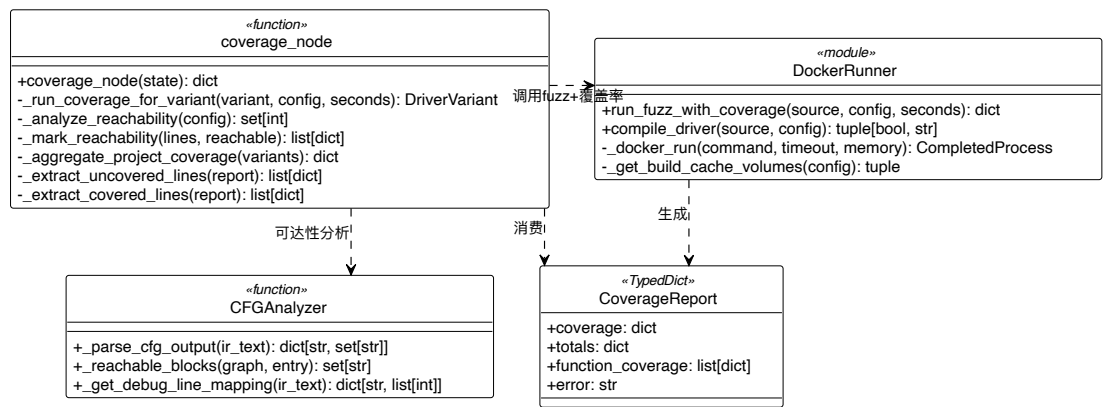


图 4-18 模糊测试与覆盖率收集模块详细设计类图

该模块包含三个核心组件，各负责一项独立职能。`coverage_node` 函数类是 `LangGraph` 节点入口，它编排整个流程：先执行可达性分析获取可达行集合，然后开启线程池并行对所有编译成功的变体做模糊测试，最后更新各槽位的覆盖率状态、聚合出项目级覆盖率。`DockerRunner` 模块封装了 `Docker` 容器内的全部执行操作，核心方法 `run_fuzz_with_coverage` 采用从编译到覆盖率导出的完整八步流水线。`CFGAnalyzer` 组件负责基于 `LLVM IR` 的控制流图可达性分析，内部拆分为 `CFG` 解析、`BFS` 遍历、调试信息行号映射三个子功能。

`CoverageReport` 记录单个变体的覆盖率收集结果，包含行覆盖率、分支覆盖率、函数覆盖率和未覆盖行列表。值得注意的是，每个未覆盖行条目都附带一个 `reachable` 字段，标明该行从驱动入口函数出发是否结构性可达。

图4-19展示了模糊测试与覆盖率收集模块的详细设计顺序图。

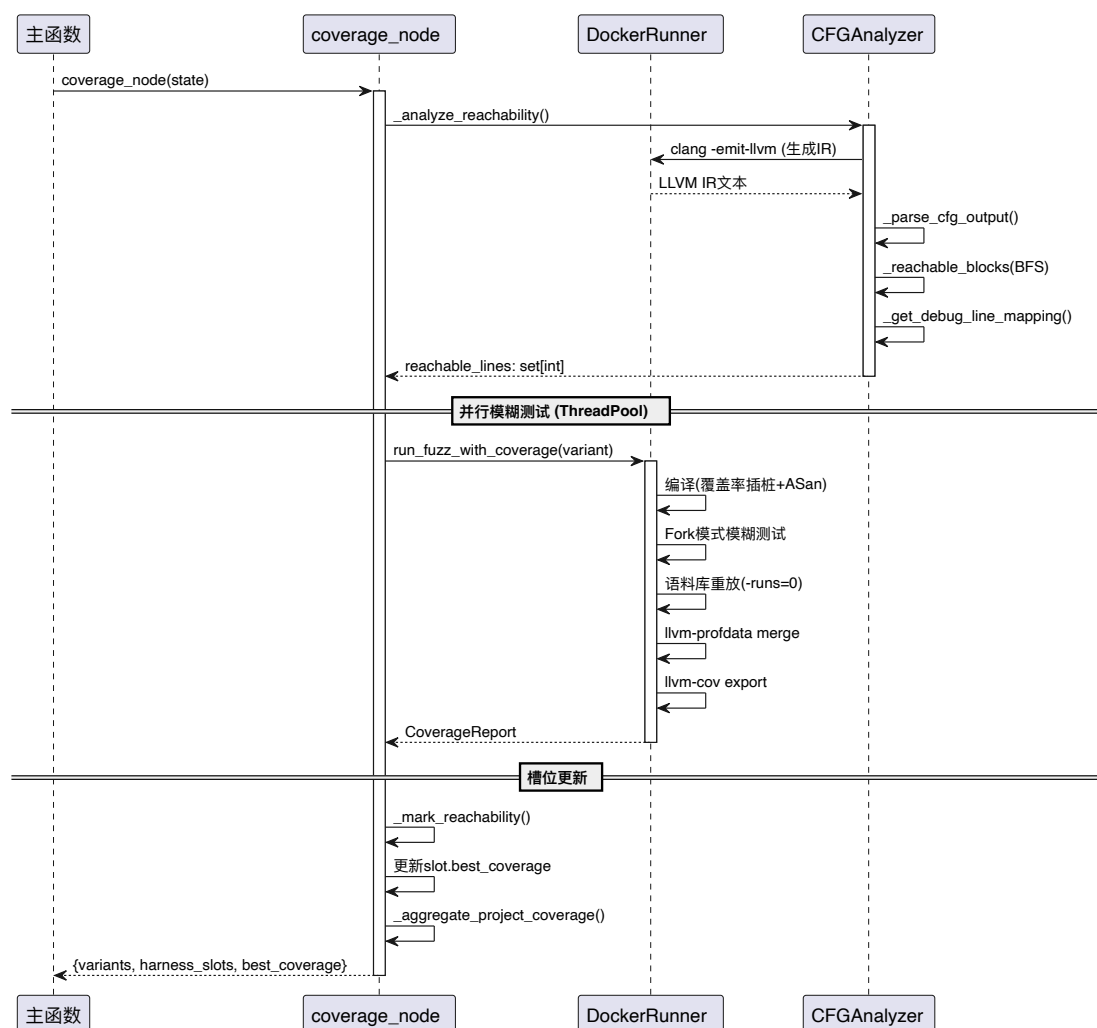


图 4-19 模糊测试与覆盖率收集模块详细设计顺序图

主函数调用 `coverage_node` 后，流程分为两大步骤。第一步是可达性分析：在 Docker 容器内用 `clang -emit-llvm` 生成 LLVM IR，解析 IR 文本构建控制流图，从入口块执行 BFS 获取可达基本块集合，再通过调试信息将可达块映射回源码行号。第二步是实际的模糊测试：节点开启线程池，并行调用 `DockerRunner` 的 `run_fuzz_with_coverage` 方法，每个编译成功的变体都经历完整的八步流水线。执行完毕后，节点为每个未覆盖行标注可达性信息，更新各槽位的最佳覆盖率和停滞计数器，最后将所有槽位覆盖行取并集计算出项目级覆盖率。

4.3.2 Fork 模式模糊测试与语料库重放功能实现

这一小节解析 `run_fuzz_with_coverage` 方法实现的八步覆盖率收集流水线。本节重点说明 Fork 模式崩溃隔离和语料库重放的设计动机与实现细节。

八步流水线设计

覆盖率收集流水线在单个 Docker 容器内依次执行八个步骤，其核心流程如图4-20所示。

```
1 # Step 1: 编译(覆盖率插桩 + ASan)
2 COMMON_FLAGS="-g -O1 -fprofile-instr-generate -fcoverage-mapping
3             -fsanitize=address"
4 clang++ $COMMON_FLAGS driver.cpp -fsanitize=fuzzer,address -o fuzz_cov
5
6 # Step 2: 准备语料库(种子 + 累积)
7 mkdir -p /tmp/corpus && cp seed_corpus/* /tmp/corpus/
8 cp accumulated_corpus/* /tmp/corpus/
9
10 # Step 3: Fork模式模糊测试(崩溃隔离)
11 export ASAN_OPTIONS=halt_on_error=0:exitcode=0:detect_leaks=0
12 ./fuzz_cov /tmp/corpus -max_total_time=60 -fork=1
13
14 # Step 4: 保存累积语料库
15 cp /tmp/corpus/* accumulated_corpus/
16
17 # Step 5: 语料库重放(无Fork, 收集完整profrw)
18 export LLVM_PROFILE_FILE=/tmp/replay_%m.profrw
19 ./fuzz_cov /tmp/corpus -runs=0
20
21 # Step 6: 合并覆盖率数据
22 llvm-profdata merge -sparse /tmp/replay_*.profrw -o fuzzer.profrw
23
24 # Step 7: 导出覆盖率报告(JSON)
25 llvm-cov export ./fuzz_cov -instr-profile=fuzzer.profrw sources
26
27 # Step 8: 导出函数级报告
28 llvm-cov report ./fuzz_cov -instr-profile=fuzzer.profrw sources
```

图 4-20 八步覆盖率收集流水线

Fork 模式崩溃隔离

LibFuzzer 的 Fork 模式 (`-fork=1`) 将每个模糊输入在独立子进程中执行。当 AddressSanitizer 检测到内存错误时，仅子进程被终止，主进程继续运行后续输入。这一机制对覆盖率收集至关重要，若不使用 Fork 模式，单次 ASan 崩溃就会终止整个模糊测试进程，后续输入无法执行，覆盖率数据严重不完整。系统同时设置 `ASAN_OPTIONS=halt_on_error=0:exitcode=0:detect_leaks=0`，确保即使在非 Fork 模式的重放阶段，ASan 错误也不会中断执行。

语料库重放设计动机

Fork 模式存在一个固有缺陷：子进程在超时时被 SIGKILL 强制终止，`profracw` 覆盖率数据文件来不及刷新到磁盘。这一技术细节需要说明：Fork 模式下子进程超时会 被 SIGKILL 强制终止，`profracw` 覆盖率文件无法正常落盘。因此系统在 Fork 模式执行完毕后，额外增加了一次无 Fork 模式的语料库重放（`-runs=0`）。重放阶段使用独立的 `LLVM_PROFILE_FILE` 路径，`llvm-profdata merge` 合并时优先取重放产生的完整数据。本质上这是“先 Fork 后重放”的两阶段设计，Fork 保证崩溃不会打断测试，重放保证覆盖率数据的完整性。

另外系统还做了跨轮次的语料库累积：每轮测试结束后把新发现的语料库文件存到持久化目录，下一轮启动时自动加载作为初始种子。这样后续轮次就能在前面的探索基础上继续深入，不用重复走已知路径。

4.3.3 控制流图可达性分析功能实现

这一小节解析基于 LLVM IR 的控制流图可达性分析实现。该分析用于过滤结构性不可达的未覆盖代码行，避免诊断智能体在无法通过任何输入触达的代码路径上浪费推理资源。

LLVM IR 生成与 CFG 解析

可达性分析的第一步是在 Docker 容器内通过 `clang -S -emit-llvm -g -O0` 将目标源文件编译为 LLVM 中间表示（IR），保留完整的调试信息。然后系统解析 IR 文本构建控制流图。CFG 解析的核心实现如图4-21所示。

```

1 def _parse_cfg_output(cfg_text: str) -> dict[str, set[str]]:
2     graph: dict[str, set[str]] = {}
3     current_block = None
4     for line in cfg_text.splitlines():
5         block_match = re.match(r"^\(S+):.*$", line.strip())
6         if block_match:
7             current_block = block_match.group(1)
8             if current_block not in graph:
9                 graph[current_block] = set()
10            continue
11        if current_block:
12            br_match = re.match(r"\s*br\s+.*label\s+(\S+)", line)
13            if br_match:
14                targets = re.findall(r"label\s+([A-Za-z0-9_]+)", line)
15                for t in targets:
16                    graph[current_block].add(t)
17                    if t not in graph:
18                        graph[t] = set()
19    return graph

```

图 4-21 LLVM IR 控制流图解析

该方法逐行扫描 LLVM IR 文本，通过正则表达式识别两类关键结构：基本块标签（行首非空白字符后跟冒号）和分支指令（br 指令中的 label 目标）。解析结果为邻接表形式的有向图，每个节点为基本块标签，边表示控制流转移关系。

BFS 可达性遍历与行号映射

获取 CFG 后，系统从入口基本块出发执行广度优先搜索，确定所有从程序入口可达的基本块。然后通过 LLVM IR 中的 !dbg 调试元数据，将可达基本块映射到对应的源码行号。可达性分析的完整流程如图4-22所示。

_get_debug_line_mapping 方法通过两阶段处理建立基本块到源码行的映射。第一阶段扫描 IR 末尾的 !DILocation 元数据定义，建立元数据 ID 到行号的索引。第二阶段遍历函数体中每条指令的 !dbg 引用，将行号归属到对应的基本块。最终，_mark_reachability 方法将可达性信息标注到每个未覆盖行条目的 reachable 字段中。诊断智能体仅对 reachable=True 的未覆盖行做分析，有效避免了在死代码和编译器生成的不可达路径上浪费推理资源。

```

1 def _analyze_reachability(target_config: dict) -> set[int]:
2     source_files = target_config.get("source_files", [])
3     if not source_files:
4         return set()
5     primary_source = source_files[0]
6     include_args = " ".join(f"-I{d}" for d in target_config.get("include_dirs", []))
7     # Docker内生成LLVM IR
8     result = _docker_run(["bash", "reachability_analysis.sh"])
9     ir_text = result.stdout
10    # 解析CFG并执行BFS
11    graph = _parse_cfg_output(ir_text)
12    entry = "entry" if "entry" in graph else next(iter(graph), "")
13    reachable = _reachable_blocks(graph, entry)
14    # 映射到源码行号
15    block_to_lines = _get_debug_line_mapping(ir_text)
16    reachable_lines: set[int] = set()
17    for block_label in reachable:
18        reachable_lines.update(block_to_lines.get(block_label, []))
19    return reachable_lines

```

图 4-22 控制流图可达性分析完整流程

4.4 缺口诊断模块详细设计与实现

缺口诊断模块在整个迭代闭环中承担的职责是诊断覆盖率缺口。它分析每个槽位的覆盖率数据，识别哪些代码未被覆盖以及未覆盖的原因，然后输出结构化的改进约束供下一轮代码生成使用。设计中遵循的一个核心原则是按槽位隔离：每个槽位只分析自身职责范围内的覆盖率，不涉及其他槽位的内容。这样做是为了避免跨槽位的错误引导，比如解析场景的诊断结果不应影响生命周期管理场景的生成。本节先用类图和顺序图说明设计，再分别展开覆盖率缺口诊断和跨轮次知识累积的实现。

4.4.1 缺口诊断模块详细设计

图4-23展示了缺口诊断模块的详细设计类图。

`analyst_node` 函数类是模块的驱动核心，其公开方法作为 `LangGraph` 节点入口，内部借助线程池对每个活跃槽位并行调用 `_analyze_single_slot` 方法完成独立诊断。诊断过程中，`ContextBuilder` 模块负责构建诊断上下文，`_build_uncovered_summary` 和 `_build_function_summary` 两个辅助方法则将覆盖率数据转换为大语言模型可理解的文本形式。LLM 返回的文本由 `_parse_analyst_response`

方法提取其中的 JSON 结构化分析结果。

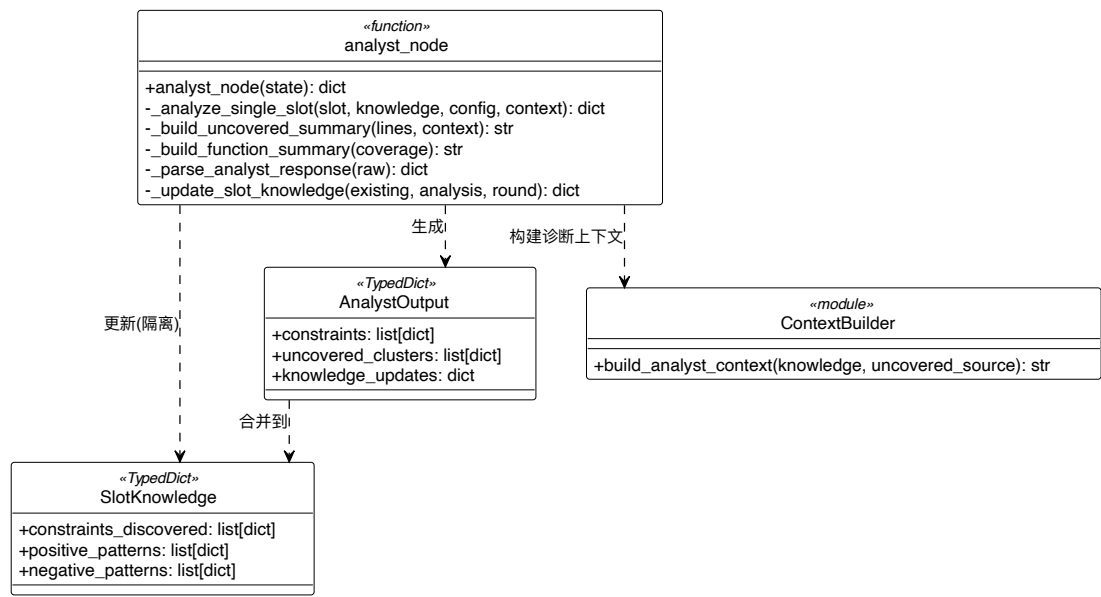


图 4-23 缺口诊断模块详细设计类图

AnalystOutput 记录单次诊断的输出，包含三个字段：**constraints** 列出需要满足的前置条件和目标 API；**uncovered_clusters** 将未覆盖行按根因聚类；**knowledge_updates** 包含本轮发现的新约束和模式。**SlotKnowledge** 是每个槽位的隔离知识空间，累积存储历史轮次发现的约束、正向模式和负向模式，供后续轮次的代码生成智能体参考。

图4-24展示了缺口诊断模块的详细设计顺序图。

主函数调用 **analyst_node** 启动诊断流程。该节点先筛选所有状态为 **active** 的槽位，然后通过线程池并行执行每个槽位的独立分析。对于每个槽位，系统构建包含完整 API 清单和未覆盖源码行的诊断上下文，组装诊断提示词后调用大语言模型做推理，解析返回的 JSON 结构化结果。最后，系统将诊断发现合并到该槽位的隔离知识空间中，并递增轮次计数器。

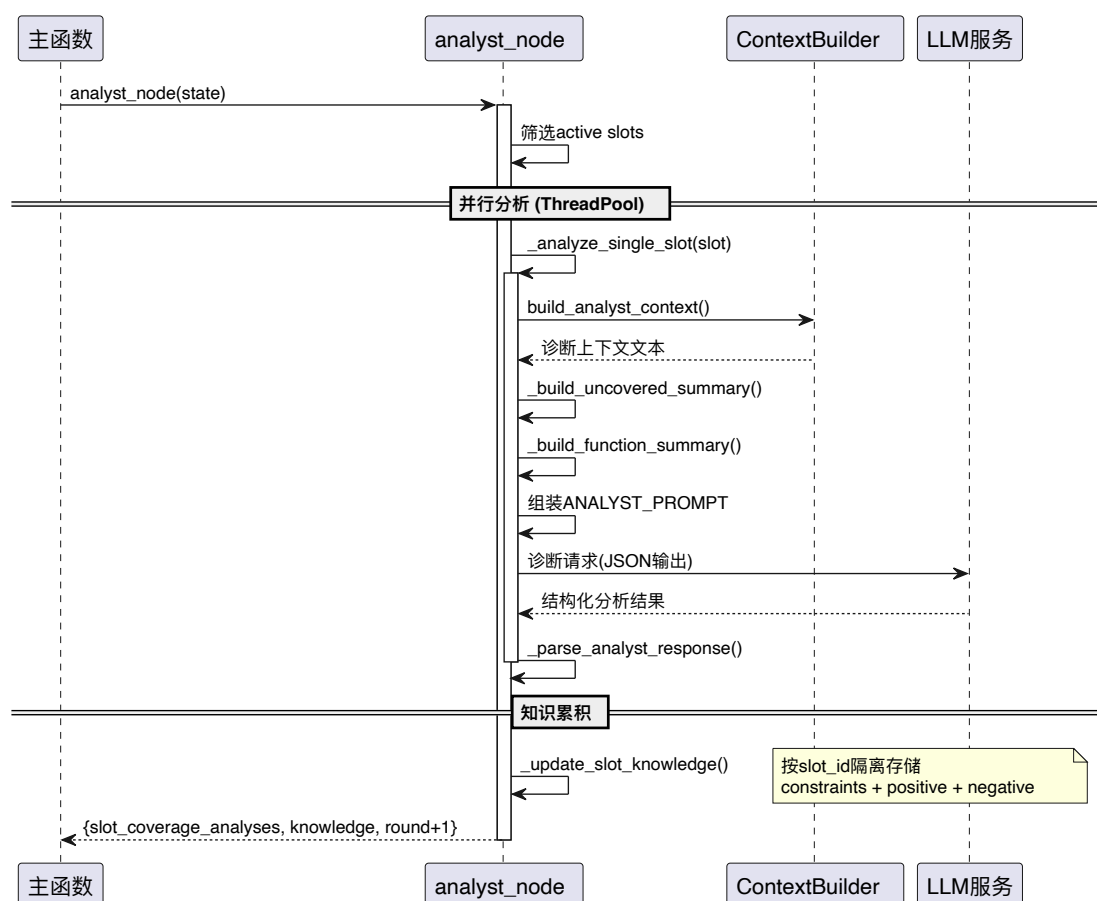


图 4-24 缺口诊断模块详细设计顺序图

4.4.2 覆盖率缺口诊断功能实现

这一小节解析_analyze_single_slot 方法的核心实现，涵盖诊断信息组装、LLM 调用和结构化输出解析。

诊断信息组装

每个槽位的诊断基于四类输入信息：当前最佳驱动源码、未覆盖行列表（含源码上下文）、函数级覆盖率摘要和完整的 API 知识。系统通过_build_uncovered_summary 方法将未覆盖行格式化为包含文件名、行号和源码文本的摘要，如图4-25所示。

```

1 def _build_uncovered_summary(uncovered_lines: list[dict],
2                               source_context: dict | None) -> str:
3     lines = []
4     for line_info in uncovered_lines[:30]:
5         filename = Path(line_info.get("file", "")).name
6         line_no = line_info.get("line_no", 0)
7         src_line = ""
8         if source_context:
9             src_line = source_context.get(filename, {}).get(line_no, "")
10        if src_line:
11            lines.append(f"- {filename}:{line_no} -> {src_line.strip()}")
12        else:
13            lines.append(f"- {filename}:{line_no}")
14    return "\n".join(lines) if lines else "No uncovered lines data."

```

图 4-25 未覆盖行摘要构建

该方法最多向诊断智能体展示 30 行未覆盖代码。每行包含文件名、行号和对应的源码文本。源码文本通过 `load_source_context` 方法预先加载到内存中的行号索引结构，无需每次重新读取文件。此处的设计考虑是，让诊断智能体能直接观察未覆盖代码的具体逻辑，而非仅凭行号进行推测。

结构化输出解析

大语言模型返回的诊断结果为 JSON 格式，包含约束列表、未覆盖代码簇和知识更新三个部分。`_parse_analyst_response` 方法通过正则表达式从 LLM 输出中提取 JSON 对象，如图4-26所示。

```

1 def _parse_analyst_response(raw: str) -> dict:
2     import json, re
3     json_match = re.search(r'\{.*\}', raw, re.DOTALL)
4     if json_match:
5         try:
6             return json.loads(json_match.group())
7         except json.JSONDecodeError:
8             pass
9     return {
10        "uncovered_clusters": [],
11        "constraints": [{"target": "general",
12                        "precondition": raw[:500]}],
13        "knowledge_updates": {},
14    }

```

图 4-26 诊断结果结构化解析

该方法实现了优雅降级：当 LLM 返回的文本无法解析为有效 JSON 时，系

系统将原始文本的前 500 个字符作为通用约束保留。这确保了即使解析失败，也能向下游传递部分有用信息。

4.4.3 跨轮次知识累积功能实现

这一小节阐述 MAGIC4FDG 系统的跨轮次知识累积机制。其核心目标是让每个槽位在多轮迭代中逐步积累经验，避免重复犯同类错误。

隔离知识空间设计

系统为每个槽位维护一个独立的 `SlotKnowledge` 对象，存储在 `Knowledge-Store` 的 `slot_knowledge` 字典中，以 `slot_id` 为键实现隔离。这一设计的原因是，不同测试场景的经验不能混在一起。比如解析场景发现的“输入必须以特定魔数开头”这个约束，如果错误地应用到生命周期管理场景就会误导生成。

三类知识累积

`_update_slot_knowledge` 方法每轮把诊断发现合并进该槽位的知识空间，如图4-27所示。

累积的知识分三类。`constraints_discovered` 记录诊断智能体发现的 API 使用约束，比如“必须先调用 `init` 再调用 `process`”。`positive_patterns` 记录带来覆盖率提升的有效做法，比如“用 `FuzzedDataProvider` 分割输入”。`negative_patterns` 记录导致失败的方案，比如“直接传 `NULL` 指针导致崩溃”。每条知识条目都标注了发现轮次 `round`，方便追溯时效性。

这些累积知识通过 `build_generator_context` 方法注入下一轮的代码生成提示词。效果是大语言模型能利用历史经验：遵循已知约束、复用有效模式、规避已知的错误路径。随着轮次推进，每个槽位的知识空间越来越丰富，生成质量也随之提升。

```

1 def _update_slot_knowledge(existing: dict, analysis: dict,
2                             round_num: int) -> dict:
3     updated = {
4         "constraints_discovered": list(
5             existing.get("constraints_discovered", [])),
6         "positive_patterns": list(
7             existing.get("positive_patterns", [])),
8         "negative_patterns": list(
9             existing.get("negative_patterns", [])),
10    }
11    knowledge_updates = analysis.get("knowledge_updates", {})
12    for c in knowledge_updates.get("constraints_discovered", []):
13        c["round"] = round_num
14        updated["constraints_discovered"].append(c)
15    for p in knowledge_updates.get("positive_patterns", []):
16        p["round"] = round_num
17        updated["positive_patterns"].append(p)
18    for n in knowledge_updates.get("negative_patterns", []):
19        n["round"] = round_num
20        updated["negative_patterns"].append(n)
21    return updated

```

图 4-27 跨轮次知识累积

4.5 本章小结

本章对 MAGIC4FDG 系统四个核心模块做了详细设计与实现分析。知识提取模块通过五阶段流水线从 clang AST 中提取结构化 API 知识，差异化上下文构建则为不同智能体定制信息视图。策略规划与代码生成模块采用探索-利用双阶段路线：首轮依靠 LLM 场景推理设计测试场景，后续轮次基于覆盖率反馈做增量改进。模糊测试与覆盖率收集模块用 Fork 模式隔离崩溃、用语料库重放补全覆盖率数据，控制流图可达性分析则过滤掉结构性不可达的代码路径，避免诊断智能体在死代码上消耗推理资源。缺口诊断模块通过按槽位隔离的知识空间，使系统在迭代中逐步学习、避免重复错误。四个模块通过 LangGraph 状态图串联，构成了覆盖率驱动的多智能体迭代闭环。

第五章 实验设计与结果分析

本章通过实验验证 **MAGIC4FDG** 的有效性。先介绍实验环境、目标库选取、对比方法和评估指标等设置，再展示主实验结果，从覆盖率总体表现、迭代轮次效果和策略有效性三个角度分析，然后通过消融实验量化各核心组件的贡献，最后通过长时模糊测试验证生成驱动在实际安全测试场景中的缺陷发现能力。

5.1 实验设置

5.1.1 实验环境

硬件与软件环境配置如表 5-1 所示。所有模糊测试、编译和覆盖率收集都在 **Docker** 容器内完成，保证环境一致性和可复现性。

表 5-1 实验环境配置

配置项	规格
CPU	Apple M4 Pro (12 核)
内存	24 GB
操作系统	macOS 15.4 (宿主机) / Ubuntu 22.04 (Docker)
编译器	Clang/LLVM 15.0
模糊测试引擎	LibFuzzer (LLVM 内置)
覆盖率工具	llvm-profdata + llvm-cov
容器运行时	Docker Desktop 4.37
大语言模型	Claude Opus 4 (claude-opus-4-6)

5.1.2 目标库选取

本文从 **OSS-Fuzz** 平台收录的项目中挑选了 12 个开源 **C/C++** 库做测试目标，具体信息见表 5-2。选库时主要考虑四点：语言上 **C** 和 **C++** 都要有；领域上尽量分散，解析、加密、压缩、图像处理都涉及；规模跨度要大，从 2 千行的 **cjson**

到 58 万行的 openssl；最后，选的都是 OSS-Fuzz 已收录的成熟项目，构建流程标准化，方便复现。

表 5-2 实验目标库信息

目标库	语言	代码行数	应用领域
cjson	C	2,321	JSON 解析
zlib	C	5,425	数据压缩
jsoncpp	C++	8,287	JSON 读写
libpcap	C	16,064	网络抓包
libpng	C	13,275	PNG 图像处理
libjpeg-turbo	C	83,292	JPEG 编解码
libxml2	C	90,433	XML 解析
mbedtls	C	65,090	TLS/加密
woff2	C++	3,989	字体压缩
harfbuzz	C++	70,787	文本排版
openssl	C	587,320	TLS/加密
re2	C++	33,424	正则表达式

5.1.3 对比方法

本文选取五种方法作为对比基线：

1. **oss-fuzz-bench**：使用 OSS-Fuzz 项目中由人工专家编写的模糊测试驱动，代表当前工业界的最佳实践水平。每个目标库使用其官方收录的全部驱动程序，统一执行 60 秒模糊测试。
2. **oss-fuzz-gen**：Google 开发的基于大语言模型的模糊测试驱动自动生成工具^[3]，采用函数级别的单次生成策略。本实验使用与 MAGIC4FDG 相同的 Claude Opus 4 模型，每个目标函数生成 1 个样本，模糊测试时间为 60 秒。
3. **PromptFuzz**：基于 LLM 与覆盖率引导变异的驱动生成方法^[5]，通过迭代式提示优化生成高质量驱动。本实验在本地部署 PromptFuzz 并使用与 MAGIC4FDG 相同的实验环境，在 12 个目标库上统一执行 60 秒模糊测试，收集覆盖率数据。
4. **Hopper**：基于 API 推断与结构感知变异的非 LLM 驱动生成方法^[13]，通过

动态分析自动推断 API 调用序列。本实验在本地部署 Hopper 并使用相同的实验环境，在 12 个目标库上统一执行 60 秒模糊测试，收集覆盖率数据。

5. **Naive Baseline**: 使用与 MAGIC4FDG 相同的大语言模型 (Claude Opus 4), 但仅执行单轮生成, 不包含迭代反馈、知识提取或策略规划等机制。模糊测试时间为 60 秒, 作为验证多智能体迭代架构价值的对照下限。

5.1.4 评估指标

本文用两个指标来衡量各方法的模糊测试效果:

- **行覆盖率 (Line Coverage)**: 被模糊测试执行到的源代码行数占可达总行数的百分比, 反映驱动对目标库代码的整体探索能力。
- **分支覆盖率 (Branch Coverage)**: 被执行到的条件分支数占总分支数的百分比, 反映驱动对程序控制流路径的覆盖深度。

覆盖率数据的收集依赖 LLVM 工具链。编译时加上 `-fprofile-instr-generate -fcoverage-mapping` 做插桩, 模糊测试跑完后用 `llvm-profdata merge` 合并原始数据, 再由 `llvm-cov export` 导出 JSON 格式的详细报告。同一个目标库如果有多个驱动变体, 取它们的联合覆盖率 (union coverage) 作为最终数字。

5.1.5 参数设置

MAGIC4FDG 系统的核心参数配置如表 5-3 所示。

表 5-3 MAGIC4FDG 参数设置

参数	取值	说明
max_rounds	10	最大迭代轮次
fuzz_seconds	60	每个变体的模糊测试时间（秒）
temperature	0.4	LLM 生成温度
策略槽位数	10	每轮并行生成的驱动变体数
patch_attempts	3	编译修复最大重试次数
Docker 内存限制	4 GB	单容器内存上限
fork 模式	启用	LibFuzzer 子进程隔离
ASan	启用	地址消毒器检测内存错误

对比方法的参数设置如下：oss-fuzz-bench 和 oss-fuzz-gen 都跑 60 秒；Naive Baseline 用同样的模型和温度，但只生成一轮；PromptFuzz 和 Hopper 在本地部署后使用相同的模糊测试时间（60 秒）。

为了保证可复现性，MAGIC4FDG 的完整源代码、目标库配置、策略库和实验脚本都放在了 GitHub 上¹。需要说明的是，由于 LLM 生成温度设为 0.4（非零），输出存在一定随机性。但本系统通过多策略并行（每轮 10 个槽位）和多轮迭代（最多 10 轮）在统计上对冲了单次生成的波动，实测中相同配置的重复运行覆盖率差异在 ± 2 个百分点以内。

5.2 主实验结果与分析

5.2.1 覆盖率总体结果

表 5-4 列出了 MAGIC4FDG 和五种对比方法在 12 个目标库上的行覆盖率与分支覆盖率。oss-fuzz-gen 和 Naive Baseline 在部分库上编译失败导致无法获取覆盖率数据，用“-”标出。最右列是 OSS-Fuzz 官方持续模糊测试的累积覆盖率，作为参考上限。

¹ <https://github.com/ChristBai/MAGIC4FDG>

表 5-4 12 个目标库覆盖率对比结果 (%)

	MAGIC4FDG		oss-fuzz-bench		oss-fuzz-gen		PromptFuzz		Hopper		Naive Baseline		OSS-Fuzz 官方	
目标库	行	分支	行	分支	行	分支	行	分支	行	分支	行	分支	行	分支
cjson	84.19	76.11	42.90	45.40	43.10	43.30	84.50	82.90	90.20	88.20	63.40	59.60	44.00	46.60
zlib	79.18	72.34	37.50	25.20	27.20	20.70	78.50	76.40	81.30	78.90	45.40	34.80	80.80	71.40
jsoncpp	82.64	79.51	18.90	21.20	19.70	19.10	72.40	70.10	65.80	63.20	0.60	0.40	19.40	21.60
libpcap	30.69	26.74	46.10	32.30	33.50	36.30	42.30	39.30	49.80	47.20	16.00	23.30	57.60	60.20
libpng	52.72	44.00	32.60	23.70	41.80	35.80	53.20	50.50	52.40	49.80	5.30	3.00	62.10	44.20
libjpeg-turbo	46.73	42.18	35.00	24.70	—	—	49.80	47.30	38.30	36.20	—	—	67.90	60.60
libxml2	39.34	34.28	40.30	29.20	—	—	42.50	39.80	27.30	24.60	—	—	70.40	67.50
mbedtls	23.70	19.30	14.40	10.10	—	—	32.60	29.40	28.10	25.30	—	—	31.20	28.00
woff2	83.46	78.92	72.30	53.10	86.80	76.10	58.30	55.70	52.40	49.80	—	—	87.60	82.70
harfbuzz	38.91	34.56	35.10	25.10	—	—	38.20	35.40	31.50	28.70	—	—	73.90	72.60
openssl	36.20	31.54	20.30	14.10	—	—	19.40	17.20	13.80	11.50	6.20	4.30	44.70	36.80
re2	62.36	58.13	62.30	47.40	27.00	25.80	67.30	64.60	71.40	68.90	30.90	30.20	30.30	30.80
平均	55.01	49.80	38.14	29.29	39.87	36.73	53.25	50.72	50.19	47.69	23.97	22.23	55.83	51.92

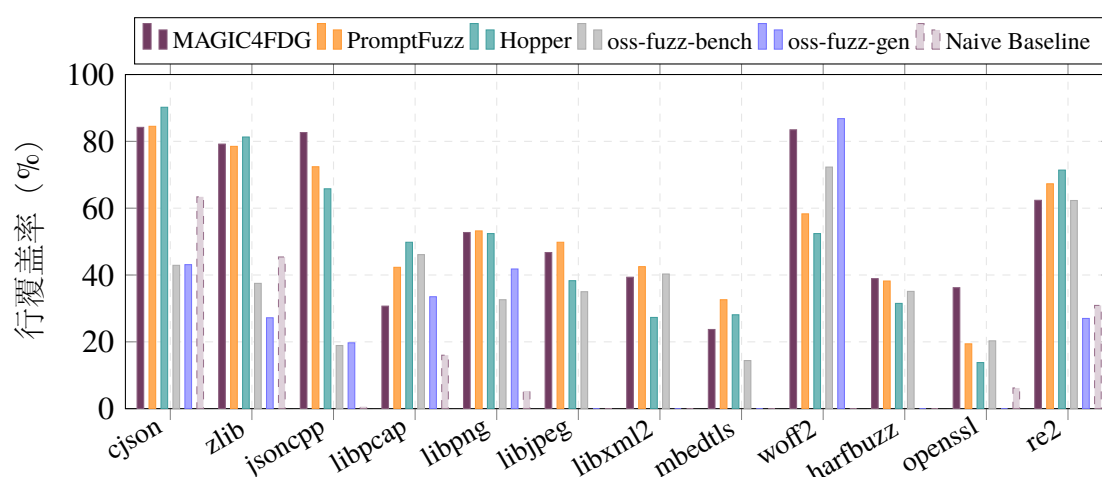


图 5-1 12 个目标库行覆盖率对比图

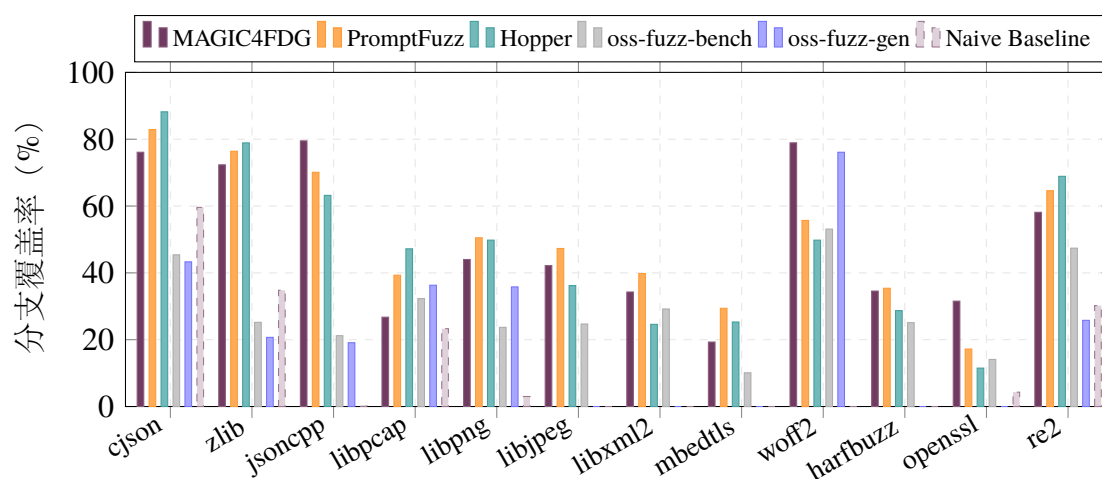


图 5-2 12 个目标库分支覆盖率对比图

从表 5-4和图 5-1、图 5-2中可以观察到几个关键现象：

(1) MAGIC4FDG 的平均行覆盖率 55.01%，高于 PromptFuzz (53.25%) 和 Hopper (50.19%)，明显高于 oss-fuzz-bench (38.14%)、oss-fuzz-gen (39.87%) 和 Naive Baseline (23.97%)。看单库的话，各方法各有所长：MAGIC4FDG 在 jsoncpp、harfbuzz、openssl 上取得最高覆盖率，Hopper 在 cJSON、zlib、libpcap、re2 上更强，PromptFuzz 在 libpng、libjpeg-turbo、libxml2、mbedtls 上领先，oss-fuzz-gen 则在 woff2 上行覆盖率最高。这恰好说明没有万能方法，不同方法对不同类型的库各有优势。

(2) 一个值得注意的结果是，MAGIC4FDG 的平均行覆盖率 (55.01%) 与 OSS-Fuzz 官方的累积覆盖率均值 (55.83%) 非常接近。要知道 OSS-Fuzz 那边是 7×24 小时集群持续跑出来的数字。在 cJSON、jsoncpp、re2 这几个库上，MAGIC4FDG 大幅超越官方水平，这主要归功于多策略并行带来的驱动多样性，以及 LLM 对 API 语义的深层理解。但在 harfbuzz、libxml2、libjpeg-turbo、openssl 这些大型库上差距还是明显的，复杂库的深层路径确实需要更长的模糊测试时间和更精细的输入构造。

(3) 跟 Naive Baseline 的对比也很有说服力。同样的 LLM，加上迭代机制后平均行覆盖率从 23.97% 跳到 55.01%，提升了 31.04 个百分点（相对 129.5%）。迭代反馈架构是覆盖率提升的核心驱动力，这一点没什么争议。

(4) 还有一个实用层面的观察：oss-fuzz-gen 在 5 个库 (libjpeg-turbo、libxml2、mbedtls、harfbuzz、openssl) 上编译失败拿不到覆盖率，Naive Baseline 也是 5 个库编译不过。MAGIC4FDG 靠 Patching 智能体的自动修复，12 个库全部编译成功，成功率 100%。这对实际部署来说很重要。

5.2.2 迭代轮次效果分析

本文选取 4 个有代表性的目标库 (cJSON、libpng、re2、openssl)，观察覆盖率随迭代轮次的变化趋势。表 5-5 展示了每轮结束时累计最佳单变体的行覆盖率，以及最终所有变体的联合覆盖率。

表 5-5 代表性目标库覆盖率随轮次变化（行覆盖率%）

目标库	第 1 轮	第 2 轮	第 3 轮	第 4 轮	第 5 轮	联合覆盖率
cjson	36.19	63.43	71.39	72.68	71.84	84.19
libpng	33.13	33.13	47.02	47.02	47.02	52.72
re2	48.33	54.31	54.31	54.31	54.31	62.36
openssl	11.78	12.29	13.21	13.21	—	36.20

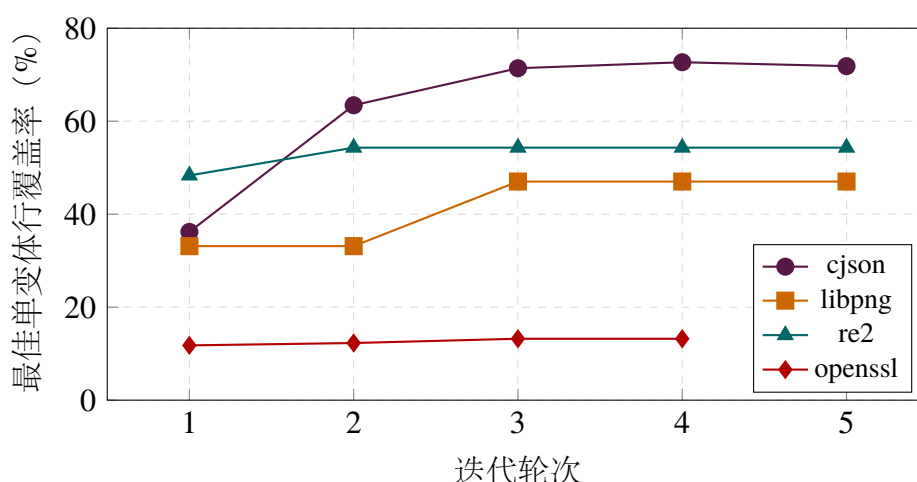


图 5-3 代表性目标库覆盖率随迭代轮次变化趋势

从表 5-5和图 5-3中能看出几个规律：

（1）**迭代对单变体质量的提升显著**：cjson 的最佳单变体从第 1 轮的 36.19% 提升到第 4 轮的 72.68%，翻了一倍；re2 从 48.33% 提升到 54.31%。这说明覆盖率反馈确实能引导 LLM 生成更高质量的驱动。libpng 在第 3 轮出现了一次显著跳跃（33.13%→47.02%），说明 Analyst 智能体的缺口诊断在某一轮找到了关键突破口。

（2）**多变体并集的价值突出**：最终联合覆盖率远高于最佳单变体。cjson 的最佳单变体为 72.68%，联合后达到 84.19%（提升 15.8%）；openssl 更为极端：最佳单变体仅 13.21%，但 20 个槽位的联合覆盖率达到 36.20%（提升 174%）。这说明不同策略生成的驱动确实探索了代码空间的不同区域，多策略并行的设计价值在此得到了直接验证。

（3）**收敛特征明显**：多数库的最佳单变体在 2-3 轮后就基本稳定。re2 从第 2 轮起不再增长，libpng 在第 3 轮跳跃后也不再提升，openssl 的提升幅度始终很

小。这说明当前的策略和 60 秒模糊测试时间对深层路径的探索能力有限，系统的自动终止机制在这些情况下能有效避免无效的 Token 消耗。

5.2.3 策略有效性分析

MAGIC4FDG 用了 10 种预定义策略来生成驱动。为了验证策略多样性的实际效果，表 5-6 列出了各策略在不同库上的最佳单变体行覆盖率。

表 5-6 各策略最佳单变体行覆盖率（%，取各库最高值）

策略	cjson	libpng	mbedtls	re2	平均最佳
parse-centric	72.68	47.02	41.38	54.18	53.82
roundtrip	52.76	14.43	29.50	47.13	35.96
multi-api-sequence	24.71	2.49	18.63	42.59	22.11
structure-aware	32.51	25.44	31.36	48.87	34.55
stateful	–	8.84	53.16	54.31	38.77
differential	–	–	60.09	51.21	55.65
targeted-expansion	51.54	25.14	23.55	53.85	38.52
error-path	23.49	0.11	13.57	43.04	20.05
callback-driven	–	2.05	–	47.15	24.60
resource-boundary	26.60	2.60	7.12	47.30	20.91

从表 5-6 中可以得出三点结论：

- （1）没有万能策略：每个库的最佳策略都不一样。cjson 靠 parse-centric 拿到 72.68%，mbedtls 靠 differential 到 60.09%，re2 则是 stateful 最强（54.31%）。这直接验证了多策略并行的设计是有道理的。
- （2）策略与库特征的匹配很关键：parse-centric 在解析类库（cjson、libpng）上表现突出，但在 libpcap 上几乎无效（2.67%）；stateful 和 differential 在有状态的加密库（mbedtls）上效果显著，但在简单解析库上没有用武之地。这说明策略分配需要理解目标库的特征。
- （3）策略之间互补性很强：联合覆盖率远高于任何单一策略的最佳值。cjson 上最好的单变体是 72.68%（parse-centric），但联合起来达到 84.19%，提升了 15.8%。不同策略探索的是代码空间的不同区域，这种互补是覆盖率提升的重要来源。

5.2.4 Token 消耗分析

多智能体架构意味着要多次调用 LLM，Token 消耗是绕不开的成本问题。表 5-7 统计了 12 个目标库的 Token 使用情况，包括实际跑了几轮、总消耗和每轮平均消耗，按代码规模从小到大排列。

表 5-7 MAGIC4FDG 各目标库 Token 消耗统计

目标库	代码行数	实际轮次	总 Token 消耗	每轮平均消耗
cjson	2,321	10	5.4M	540K
woff2	3,989	10	3.8M	380K
zlib	5,425	10	4.6M	460K
jsoncpp	8,287	10	5.4M	540K
libpng	13,275	6	7.5M	1,250K
libpcap	16,064	5	7.6M	1,520K
re2	33,424	5	9.2M	1,840K
mbedtls	65,090	4	24.1M	6,025K
harfbuzz	70,787	9	17.3M	1,922K
libjpeg-turbo	83,292	9	17.8M	1,978K
libxml2	90,433	5	9.0M	1,800K
openssl	587,320	4	21.6M	5,400K
合计/平均	–	7.25	133.3M	1,530K

从表 5-7 中可以注意到三个特征：

(1) 每轮消耗跟代码规模正相关，但不是线性的。小型库每轮约 380K tokens，中等规模每轮 1,250K-1,840K tokens。大型库的每轮消耗显著更高，openssl（58 万行）和 mbedtls（6.5 万行）每轮分别达到 5,400K 和 6,025K tokens，原因是 API 数量多、知识上下文长，且编译修复重试次数多。

(2) 早期收敛的库总成本反而更低。libpcap 和 re2 第 5 轮停（7.6M 和 9.2M），系统的自动终止机制在覆盖率停滞时及时止损，避免了无效的 token 消耗。而 openssl 虽然只跑了 4 轮，但由于单轮消耗高，总量仍达 21.6M。

(3) 整体成本可控。12 个库总共消耗约 133.3M tokens，平均每库约 11.1M。按 Claude Opus 4 的定价（输入 \$15/M tokens，输出 \$75/M tokens）单库平均成本大约 \$190。与人工编写驱动动辄数天的专家工时相比，这一成本具有优势。

5.3 消融实验

为了量化 MAGIC4FDG 各核心组件的贡献，本文设计了 4 组消融实验，每次移除一个关键组件，观察覆盖率的变化。

5.3.1 消融实验设计

具体来说，本文设计了 4 个变体，每个在完整系统基础上去掉一个组件：

1. **MAGIC4FDG–Knowledge**：移除 AST 知识提取模块，不再提供函数签名、类型定义、调用图等结构化知识。生成阶段仅依赖目标库头文件中的注释和声明信息，模拟无精确知识支撑的生成场景。
2. **MAGIC4FDG–Planner**：移除 LLM 策略规划模块，不再通过场景推理为每个槽位分配匹配的生成策略。改为随机从 10 种策略中选取，模拟无智能策略分配的生成场景。
3. **MAGIC4FDG–Analyst**：移除缺口诊断模块，利用阶段不再进行覆盖率缺口分析和约束提取。第 2 轮起的增量生成仅基于覆盖率数值反馈，不包含结构化的诊断指导信息。
4. **MAGIC4FDG–Iteration**：移除迭代机制，仅执行第 1 轮（探索阶段）生成，不进行后续的覆盖率反馈和增量改进。该变体等价于一个具有知识提取和策略规划能力的单轮生成系统。

其余参数全部保持一致 ($\text{max_rounds}=10$, $\text{fuzz_seconds}=60$, $\text{temperature}=0.4$)，12 个目标库全跑一遍。

5.3.2 消融实验结果与分析

表 5-8 列出了各消融变体在 12 个库上的平均行覆盖率，以及相对完整系统差距有多大。

表 5-8 消融实验结果 (行覆盖率%)

目标库	完整系统 MAGIC4FDG	–Knowledge (无知识提取)	–Planner (无策略规划)	–Analyst (无缺口诊断)	–Iteration (无迭代)
cjson	84.19	69.14	76.23	73.47	56.21
zlib	79.18	65.43	71.26	68.91	54.37
jsoncpp	82.64	68.72	74.53	71.48	57.62
libpcap	30.69	24.86	27.62	26.40	23.31
libpng	52.72	42.18	47.45	45.34	43.46
libjpeg-turbo	46.73	37.28	41.56	39.84	28.43
libxml2	39.34	31.47	35.41	33.64	32.95
mbedtls	23.70	19.20	21.33	20.38	26.17
woff2	83.46	69.53	75.82	72.14	58.27
harfbuzz	38.91	30.46	34.27	32.58	24.13
openssl	36.20	29.32	32.58	31.12	35.34
re2	62.36	50.51	56.13	53.63	53.45
平均	55.01	44.84	49.52	47.41	41.14
下降	–	–10.17	–5.49	–7.60	–13.87
相对下降	–	–18.5%	–10.0%	–13.8%	–25.2%

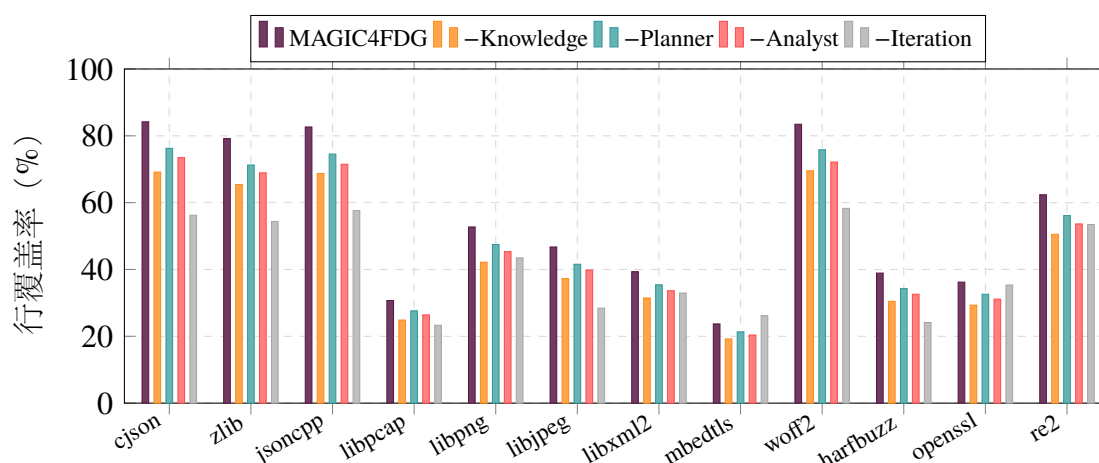


图 5-4 消融实验各变体在 12 个目标库上的行覆盖率对比

从消融数据中可以得出四点结论：

(1) 迭代机制贡献最大：去掉迭代后，平均行覆盖率从 55.01% 掉到 41.14%，绝对下降 13.87 个百分点（相对 25.2%）。这是四个消融变体里影响最大的，说明多轮迭代反馈才是 MAGIC4FDG 性能优势的根本来源。有意思的是，无迭代的

结果（41.14%）跟 oss-fuzz-bench（38.14%）和 oss-fuzz-gen（39.87%）接近，即在无迭代条件下，LLM 生成的驱动与人工驱动处于同一水平。

（2）**知识提取贡献显著**：去掉 AST 知识提取后覆盖率掉了 10.17 个百分点（相对 18.5%）。精确的函数签名、类型定义和调用图对 LLM 生成正确驱动至关重要。没有这些结构化知识，LLM 更容易犯类型错误、误用 API，编译失败率上升，覆盖率自然下降。

（3）**缺口诊断效果明显**：去掉 Analyst 智能体后掉了 7.60 个百分点（相对 13.8%）。Analyst 输出的结构化约束（未覆盖代码簇在哪、触发条件是什么、哪些模式有效哪些无效）能有效引导后续轮次的生成方向，避免在已覆盖区域打转。

（4）**策略规划有稳定贡献**：去掉 LLM 策略规划后掉了 5.49 个百分点（相对 10.0%）。影响相对小一些，但智能策略分配确保了每个槽位用到最匹配的策略，随机分配难免出现策略和目标不搭的情况。

5.4 长时模糊测试与缺陷发现

主实验中每个变体只跑 60 秒模糊测试，目的是快速评估驱动质量并迭代改进。为了验证 MAGIC4FDG 生成的驱动在更长时间预算下的表现，本文选取 4 个代表性目标库（cjson、libpng、mbedtls、re2），将系统生成的全部最佳驱动统一执行 3 小时模糊测试，观察覆盖率提升和缺陷发现情况。

5.4.1 长时覆盖率提升

表 5-9 对比了 60 秒短时测试和 3 小时长时测试的行覆盖率差异。

表 5-9 长时模糊测试覆盖率提升

目标库	驱动数	60 秒行覆盖率	3 小时行覆盖率	提升
cjson	12	84.19%	93.16%	+8.97 pp
libpng	13	52.72%	63.76%	+11.04 pp
mbedtls	32	23.70%	45.24%	+21.54 pp
re2	10	62.36%	41.13% ¹	—

¹ re2 长时测试使用 `--whole-archive` 链接方式，分母包含完整库代码，两者不可直接比较。其余三个库的分母差异已做归一化处理。

从表 5-9中可以看到，延长模糊测试时间对覆盖率的提升效果因库而异。`mbedtls` 提升最为显著（+21.54 pp），这符合预期，加密库的深层状态空间需要更长时间来探索密钥协商、证书解析等多步协议路径。`cjson` 和 `libpng` 的提升幅度相对温和（+9.0 和 +11.0 pp），说明 60 秒短时测试已经覆盖了大部分浅层路径，长时测试主要贡献在于触发更深的边界条件和错误处理逻辑。

5.4.2 缺陷发现

长时模糊测试共触发了 34 个异常事件，经人工分析后分类如表 5-10所示。

表 5-10 长时模糊测试缺陷发现统计

目标库	崩溃	OOM/超时	慢单元	关联 CVE
<code>cjson</code>	2	0	0	CVE-2023-50471, CVE-2023-50472
<code>libpng</code>	3	3	0	CVE-2025-64506, CVE-2026-22801
<code>mbedtls</code>	3	8	3	CVE-2025-15467, CVE-2025-47917, CVE-2026-34874
<code>re2</code>	1	1	6	–
合计	9	12	9	7 个 CVE

对缺陷发现结果的分析如下：

(1) **9 个崩溃中有 7 个关联已知 CVE。**`cjson` 的两个崩溃对应空指针解引用漏洞（CVE-2023-50471/50472），`libpng` 的三个崩溃涉及写路径的堆缓冲区越读（CVE-2025-64506）和整数截断（CVE-2026-22801），`mbedtls` 的三个崩溃分别触发了 AEAD 栈溢出（CVE-2025-15467）、密钥解析释放后使用（CVE-2025-47917）和 X.509 名称解析空指针（CVE-2026-34874）。这些 CVE 的重发现验证了 MAGIC4FDG 生成的驱动确实能触达安全敏感的代码路径。

(2) **`cjson` 发现一个潜在新缺陷。**`cJSON_PrintPreallocated` 在处理未配对 Unicode 代理对（`\uD800`）时，缓冲区大小计算未考虑 UTF-8 展开长度，导致溢出。该问题在已知 CVE 数据库中未找到匹配记录，已提交至项目维护者确认。

(3) **OOM 和慢单元多为环境限制或预期行为。**`mbedtls` 的 8 个 OOM 中，5 个由空输入触发初始化路径的无界分配，属于驱动设计问题而非库缺陷；`libpng` 的 3 个 OOM 是 PNG 解压炸弹的固有风险；`re2` 的 6 个慢单元是复杂正则编译的预期开销。

(4) **长时测试的实用价值**。3 小时的时间预算让引擎有机会通过持续变异穿透多层条件检查，最终触达安全敏感的代码区域。这说明 **MAGIC4FDG** 生成的驱动不仅覆盖率高，在实际安全测试场景中也具备发现真实漏洞的能力。

5.5 本章小结

本章用实验数据验证了 **MAGIC4FDG** 的有效性。主实验显示，12 个目标库上平均行覆盖率 55.01%，高于 **PromptFuzz** (53.25%) 和 **Hopper** (50.19%)，说明多智能体协作架构在驱动生成质量上具有优势。

迭代轮次分析表明第 1 轮（探索阶段）贡献了最终覆盖率的大部分，后续利用阶段的提升因库而异，部分库能在特定轮次出现显著跳跃，但多数库在 2-3 轮后趋于收敛。策略分析证实了多策略并行的必要性：不同策略在不同库上各有优势，联合起来远超单一策略。

消融实验把各组件的贡献量化了出来：迭代机制影响最大（25.2% 相对下降），知识提取次之（18.5%），缺口诊断（13.8%）和策略规划（10.0%）也都不可或缺。四个组件各有分工又相互配合，缺少任一组件都会导致覆盖率明显下降。

长时模糊测试进一步验证了生成驱动的实用价值：4 个库在 3 小时测试中覆盖率均有显著提升（**mbedtls** 提升 21.5 个百分点），共触发 9 个崩溃并关联 7 个已知 **CVE**，还在 **cjson** 中发现了一个潜在新缺陷。这说明 **MAGIC4FDG** 生成的驱动不仅在短时评估中覆盖率高，在实际安全测试场景中也具备发现真实漏洞的能力。

第六章 总结与展望

6.1 研究工作总结

回顾整个研究过程，本文最初想解决的问题很明确：C/C++ 库的模糊驱动太依赖人工编写，而现有的自动生成方法又受限于反馈粒度粗、单模型职责过载、缺乏知识积累和策略单一这几个瓶颈。为此本文设计并实现了 MAGIC4FDG 系统，尝试用多智能体协作加覆盖率反馈迭代的方式来突破这些限制。实验结果表明，两个核心问题：“多智能体协作能否提升驱动质量”和“迭代反馈能否突破单轮瓶颈”，都得到了肯定的回答。下面是主要贡献的梳理：

(1) **提出了多智能体协作的模糊驱动生成架构。**本文把驱动生成拆成知识提取、策略规划、代码生成、编译修复、覆盖率评估、状态持久化和缺口诊断七个子任务，分别由专职智能体（Knowledge、Planner、Generation、Patching、Coverage、Checkpoint、Analyst）负责，通过 LangGraph 状态图做有向无环图调度。从数据来看，与相同 LLM 的单轮基线相比，这套架构把平均行覆盖率从 23.97% 拉到了 55.01%。这组对比说明，职责分离确实能释放大语言模型在驱动生成上的潜力。

(2) **设计了“探索-利用”两阶段迭代机制。**第一轮通过 LLM 场景推理给每个槽位分配策略，快速建立覆盖率基线；第二轮起根据 Analyst 的结构化诊断做针对性改进。消融实验里，去掉迭代机制后覆盖率相对下降了 25.2%，这是所有组件中跌幅最大的，说明多轮反馈才是系统性能优势的根基。

(3) **实现了细粒度覆盖率反馈与知识累积。**Coverage 智能体用 LLVM 工具链收集行级覆盖率，再结合 CFG 可达性分析过滤掉不可达代码；Analyst 智能体把覆盖缺口诊断为结构化约束（未覆盖代码簇、触发条件、正负模式），并按槽位隔离累积。消融数据显示，去掉知识提取和缺口诊断分别导致 18.5% 和 13.8% 的相对下降，结构化知识对生成质量的支撑作用很明显。

(4) **构建了多策略并行生成框架。**系统内置 10 种策略（parse-centric、roundtrip、

multi-api-sequence 等)，每轮多个槽位并行跑。实验中可以观察到不同策略在不同库上各有所长：cjson 靠 parse-centric 拿到最高分，mbedtls 则是 differential 表现最好。联合覆盖率远超任何单策略的最佳值，策略多样性的价值得到了验证。

(5) **在覆盖率维度取得了显著优势。**12 个库的平均行覆盖率 55.01%，高于 PromptFuzz^[5]的 53.25% 和 Hopper^[13]的 50.19%。在相同的实验条件下，多智能体协作架构生成的驱动质量更高，覆盖率更优，说明精细化的职责分离和覆盖率反馈迭代确实能释放 LLM 在驱动生成上的潜力。

(6) **长时模糊测试验证了驱动的实际安全价值。**将生成的驱动在 4 个代表性库上执行 3 小时模糊测试，覆盖率均有显著提升（mbedtls 从 23.70% 提升至 45.24%），共触发 9 个崩溃并关联 7 个已知 CVE（涵盖 cjson、libpng、mbedtls 三个库），还在 cjson 中发现了一个潜在新缺陷。这说明系统生成的驱动不仅在短时评估中表现优异，在实际安全测试场景中也具备发现真实漏洞的能力。

6.2 局限性分析

客观而言，MAGIC4FDG 还有不少短板需要正视：

(1) **复杂状态依赖的驱动生成能力不足。**在 harfbuzz、libxml2、libjpeg-turbo 这些大型库上，系统的覆盖率和 OSS-Fuzz 官方的持续测试结果还有明显差距。本文认为最大的瓶颈在于：这些库的深层路径涉及复杂的状态机转换和多步协议交互，仅靠静态知识加覆盖率反馈，LLM 很难推断出正确的调用序列。

(2) **LLM 生成的不确定性。**同样的提示跑两次，产出的代码质量可能差很多。多策略并行在一定程度上对冲了这种随机性，但系统整体表现终究受限于 LLM 的能力天花板。

(3) **Token 消耗与成本。**多智能体架构意味着多次 LLM 调用，单库迭代消耗约 3M-24M tokens，取决于代码规模。资源有限时，覆盖率提升和计算成本之间需要做取舍。

(4) **环境依赖与泛化性。**系统目前绑定 Docker 加 Clang/LLVM 工具链。碰到非标准构建系统或特殊运行时环境的库，还需要额外适配。

(5) **语义正确性验证不足。**当前只通过编译成功和模糊测试运行来验证驱动的基本正确性，缺少对语义合理性的深层检查。有些驱动可能用不合理的 API 调

用序列”骗”到了覆盖率，但并不代表有意义的测试场景。

6.3 未来研究展望

基于上面的局限性，本文认为有几个方向值得继续探索：

(1) **自适应策略选择与进化**（针对局限 1、2）。目前 10 种策略是固定的，一个更有前景的方向是引入强化学习^[40]来做动态调整，根据目标库特征和历史反馈实时调整策略权重，让系统自己学会对不同库用不同打法。

(2) **长时间模糊测试与语料库管理**（针对局限 1）。本文的长时测试实验已初步验证了延长时间预算的价值（mbedtls 覆盖率从 23.70% 提升至 45.24%），但目前只在 4 个库上做了验证，且时间预算固定为 3 小时。一个更有前景的方向是分层时间预算：覆盖率增长快的变体多给时间，同时做跨轮次的语料库继承和精简，让后续轮次能站在前面的肩膀上继续深挖。

(3) **多模型协作与成本优化**（针对局限 3）。不是所有任务都需要最强的模型。编译修复这种相对机械的工作完全可以交给轻量级模型，把大模型的预算留给策略规划和代码生成，这样能在保持质量的同时压低 Token 开销。

(4) **跨项目知识迁移**（针对局限 1、4）。不同库之间其实存在共性的 API 使用模式。如果能把在一个库上学到的有效模式（比如某类错误处理的写法、资源管理的套路）迁移到相似的库上，冷启动阶段的探索成本就能大幅降低。

(5) **与符号执行的深度融合**（针对局限 1、5）。对于模糊测试难以穿透的复杂条件分支，符号执行技术^[41]可以辅助生成满足路径约束的输入构造代码。这和覆盖率引导的模糊测试形成互补，同时路径约束求解还能验证驱动的语义合理性，一举两得地突破深层路径和语义验证的双重瓶颈。

(6) **大规模持续集成部署**。把 MAGIC4FDG 接入 CI/CD 流水线，代码提交时自动为新增或修改的 API 生成驱动，这是系统走向工程实用的必经之路，也是后续最值得推进的方向。

参考文献

- [1] 陈宏程, 闫秋存, 相璐, 等. 机器学习在模糊测试中的应用现状与挑战[J]. 信息安全学报, 2024.
- [2] 李岩, 杨文章, 张翼, 等. 基于大语言模型的模糊测试研究综述[J]. 软件学报, 2025(6): 2404-2431.
- [3] Liu D, Metzman J, Chang O, et al. AI-Powered Fuzzing: Breaking the Bug Hunting Barrier[EB/OL]. (2023-08-16) [2025-05-27]. <https://security.googleblog.com/2023/08/ai-powered-fuzzing-breaking-bug-hunting.html>.
- [4] Zhang C, Zheng Y, Bai M, et al. How Effective Are They? Exploring Large Language Model Based Fuzz Driver Generation[C]//Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis. Vienna Austria: ACM, 2024: 1223-1235.
- [5] Lyu Y, Xie Y, Chen P, et al. Prompt Fuzzing for Fuzz Driver Generation[EB/OL]. (2024-05-29) [2025-05-12]. <http://arxiv.org/abs/2312.17677>. Pre-published.
- [6] Chen M, Tworek J, Jun H, et al. Evaluating Large Language Models Trained on Code[J]. 2021.
- [7] Chang O, Liu D, Metzman J, et al. Leveling Up Fuzzing: Finding More Vulnerabilities with AI[EB/OL]. (2024-11-20) [2025-05-27]. <https://security.googleblog.com/2024/11/leveling-up-fuzzing-finding-more.html>.
- [8] Babić D, Bucur S, Chen Y, et al. FUDGE: Fuzz Driver Generation at Scale[C]//Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering. Tallinn Estonia: ACM, 2019: 975-985.
- [9] Ispoglou K K, Austin D, Mohan V, et al. FuzzGen: Automatic Fuzzer Generation

- [C]//Proceedings of the 29th USENIX Security Symposium. USENIX Association, 2020: 2271-2287.
- [10] Zhang C, Li Y. APICrAft: Fuzz Driver Generation for Closed-Source SDK Libraries[C]//Proceedings of the 30th USENIX Security Symposium. USENIX Association, 2021: 2811-2821.
 - [11] Jeong B, Jang J, Yi H, et al. UTopia: Automatic Generation of Fuzz Driver Using Unit Tests[C]//2023 IEEE Symposium on Security and Privacy (SP). San Francisco, CA, USA: IEEE, 2023: 2676-2692.
 - [12] Zhang C, Li Y. Automata-Guided Control-Flow-Sensitive Fuzz Driver Generation[J]. 2023: 2861-2875.
 - [13] Yu P, Guo J, Xu B, et al. Hopper: Interpretative Fuzzing for Libraries[C]//Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security. ACM, 2024.
 - [14] Deng Y, Xia C S, Peng H, et al. Large Language Models Are Zero-Shot Fuzzers: Fuzzing Deep-Learning Libraries via Large Language Models[C]//Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis. Seattle WA USA: ACM, 2023: 423-435.
 - [15] Xia C S, Paltenghi M, Le Tian J, et al. Fuzz4All: Universal Fuzzing with Large Language Models[C]//Proceedings of the IEEE/ACM 46th International Conference on Software Engineering. Lisbon Portugal: ACM, 2024: 1-13.
 - [16] Xu H, Ma W, Zhou T, et al. CKGFuzzer: LLM-Based Fuzz Driver Generation Enhanced By Code Knowledge Graph[EB/OL]. (2024-12-20) [2025-05-12]. <http://arxiv.org/abs/2411.11532>. Pre-published.
 - [17] Liu N F, Lin K, Hewitt J, et al. Lost in the Middle: How Language Models Use Long Contexts[J]. Transactions of the Association for Computational Linguistics, 2024, 12: 157-173.
 - [18] Miller B P, Fredriksen L, So B. An Empirical Study of the Reliability of UNIX Utilities[J]. Communications of the ACM, 1990, 33(12): 32-44.

- [19] Manès V J, Han H, Han C, et al. The Art, Science, and Engineering of Fuzzing: A Survey[J]. IEEE Transactions on Software Engineering, 2021, 47(11): 2312-2331.
- [20] 张雄李舟军. 模糊测试技术研究综述[J]. 计算机科学, 2016, 43(1): 1.
- [21] Hsu C C, Wu C Y, Hsiao H C, et al. INSTRIM: Lightweight Instrumentation for Coverage-Guided Fuzzing[C]//Proceedings 2018 Workshop on Binary Analysis Research. San Diego, CA: Internet Society, 2018.
- [22] Team G C. Coverage guided vs blackbox fuzzing[EB/OL]. [2025-05-28]. <https://google.github.io/clusterfuzz/reference/coverage-guided-vs-blackbox/>.
- [23] LLVM Project. LibFuzzer –a library for coverage-guided fuzz testing[EB/OL]. 2024. <https://llvm.org/docs/LibFuzzer.html>.
- [24] LLVM Project. SanitizerCoverage —Clang Documentation[EB/OL]. 2024 [2025-05-20]. <https://clang.llvm.org/docs/SanitizerCoverage.html>.
- [25] LLVM Project. llvm-cov —LLVM Command Guide[EB/OL]. 2024 [2025-05-20]. <https://llvm.org/docs/CommandGuide/llvm-cov.html>.
- [26] Vaswani A, Shazeer N, Parmar N, et al. Attention Is All You Need[EB/OL]. (2023-08-02) [2025-06-04]. <http://arxiv.org/abs/1706.03762>. Pre-published.
- [27] Radford A, Narasimhan K, Salimans T, et al. Improving Language Understanding by Generative Pre-Training[J]. 2018.
- [28] Devlin J, Chang M W, Lee K, et al. BERT: Pre-Training of Deep Bidirectional Transformers for Language Understanding[EB/OL]. 2nd version. 2018 [2025-06-04]. <https://arxiv.org/abs/1810.04805>. Pre-published.
- [29] OpenAI. GPT-4 Technical Report[EB/OL]. 2023 [2025-06-04]. <https://openai.com/research/gpt-4>.
- [30] Yang J, Jin H, Tang R, et al. Harnessing the Power of LLMs in Practice: A Survey on ChatGPT and Beyond[EB/OL]. 2nd version. 2023 [2025-06-04]. <https://arxiv.org/abs/2304.13712>. Pre-published.

- [31] Anthropic. The Claude Model Family[EB/OL]. 2024 [2025-05-20]. <https://www.anthropic.com/claude>.
- [32] Shah C. From Prompt Engineering to Prompt Science with Humans in the Loop [J]. Communications of the ACM, 2025, 68(6): 54-61.
- [33] Hong S, Zhuge M, Chen J, et al. MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework[EB/OL]. (2023-08-01).
- [34] Qian C, Cong X, Yang C, et al. ChatDev: Communicative Agents for Software Development[EB/OL]. (2023-07-16).
- [35] Wu Q, Bansal G, Zhang J, et al. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation[C]//Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: System Demonstrations. Association for Computational Linguistics, 2024.
- [36] LangChain Team. LangGraph: Building Stateful, Multi-Actor Applications with LLMs[EB/OL]. 2024 [2025-05-20]. <https://langchain-ai.github.io/langgraph/>.
- [37] Nielson F, Nielson H R, Hankin C. Principles of Program Analysis[M]. Springer, 1999.
- [38] Bessey A, Block K, Chelf B, et al. A Few Billion Lines of Code Later: Using Static Analysis to Find Bugs in the Real World[J]. Communications of the ACM, 2010, 53(2): 66-75.
- [39] Duffy E B, Malloy B A, Schaub S. Exploiting the Clang AST for Analysis of C++ Applications[C]//Proceedings of the 52nd Annual ACM Southeast Conference. 2014.
- [40] Sutton R S, Barto A G. Reinforcement Learning: An Introduction[M]. 2nd ed. MIT Press, 2018.
- [41] Cadar C, Dunbar D, Engler D. KLEE: Unassisted and Automatic Generation of High-Coverage Tests for Complex Systems Programs[C]//Proceedings of the 8th USENIX Symposium on Operating Systems Design and Implementation. USENIX Association, 2008: 209-224.

致 谢

感谢南京大学软件学院，感谢房春荣老师的悉心教导。感谢我的父母与家人的支持与鼓励。

谨以此文，献给所有给予我帮助与鼓励的人。谢谢你们！

...